



MATERIA

Base de Datos

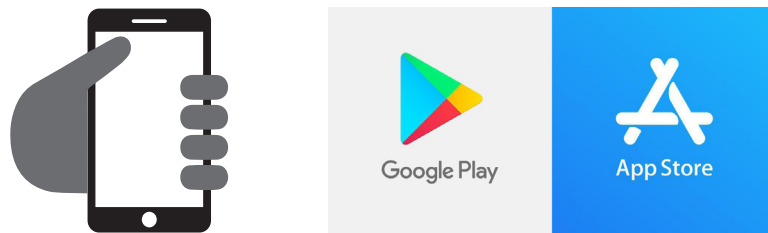
BD · PRIMER CUATRIMESTRE

CÓDIGOS QR

Estimado Alumno:

Esta materia puede contener **Códigos QR** para agilizar el acceso a los contenidos audiovisuales que el docente ha desarrollado especialmente para Usted.

Para poder visualizar los videos utilice el lector de código QR que trae por defecto su teléfono o dispositivo móvil. En caso de no disponer de uno, puede descargarlo desde *Google Play* si su sistema operativo corresponde a Android o desde la *App Store* si utiliza IOS.



Una vez instalado el software en su dispositivo móvil:

- 1) Abra la aplicación
- 2) Apunte con la cámara de su dispositivo hacia los códigos QR que encuentre en el interior de su materia.



De esta manera estará visualizando los videos que el docente ha desarrollado y/o seleccionado especialmente para usted.

Índice

Bienvenida	4
Propósito	4
Competencias	5
Objetivos	5
Contenidos mínimos	10
Contenidos	10
Bibliografía	10
Planificación	11
Metodología	11
Evaluación	12
Mapa conceptual	13
Módulos	
› Módulo 1	14
› Módulo 2	52
› Módulo 3	81
› Módulo 4	119

Importante

Lecturas básicas y complementarias disponibles desde plataforma.

Impresión total del documento 127 páginas

Bienvenida

Bienvenidos a la materia:

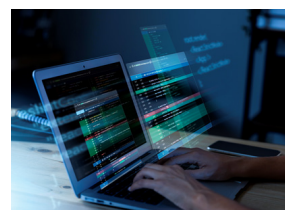
"Base de Datos"

Los invitamos a ver el siguiente video del docente:



Propósito

Estimado/a estudiante, bienvenido/a, es un gusto poder acompañarlo en este primer acercamiento al mundo de las bases de datos. En la era digital, los datos son el activo más valioso. Cualquier sistema, aplicación móvil, página web o proceso de negocio que pueda imaginar genera, consume y depende por completo de datos. Sin una gestión eficiente de estos, todo se detiene. No importa si su futuro trabajo se trata sobre el análisis, la visualización o la inteligencia artificial: las habilidades que aprenderá aquí serán fundamentales para cualquiera de estas tareas.



Vamos a estructurar nuestro viaje en cuatro módulos clave, que se construyen uno sobre el otro, y culminan en un proyecto integrador donde todo cobrará vida. Comenzaremos por el principio: ¿cómo evolucionó el almacenamiento de datos desde archivos de texto hasta sistemas complejos? Aquí entenderá los conceptos fundamentales: tablas, relaciones y constraints. En el primer módulo, aprenderá por qué una llave primaria es tan crucial y cómo la normalización evita el caos y la redundancia. También hablaremos de transacciones y ese acrónimo mágico ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), que garantiza que nuestras operaciones sean confiables y seguras.

Con estos cimientos puestos, es hora de aprender a comunicarnos con la base de datos. Para ello en el Módulo 2 utilizaremos SQL, el lenguaje universal para gestionar datos relacionales. Nos sumergiremos en dos grandes áreas: el DDL (Lenguaje de Definición de Datos), para crear y modificar las estructuras de las tablas; y el DML (Lenguaje de Manipulación de Datos), para interactuar con los datos dentro de ellas. Desde el sencillo select, where hasta consultas más potentes con join, group by, having y order by. Aquí es donde la teoría se vuelve práctica.

Sin embargo, el mundo de los datos no es solo SQL. Cuando los datos son masivos, no tienen una estructura fija o son inherentemente interconectados, surgen otras herramientas especializadas.

En el tercer módulo exploraremos las bases NoSQL. Trabajaremos con MongoDB para entender las bases de datos documentales, veremos las bases de grafos para analizar relaciones complejas y echaremos un vistazo a soluciones propietarias en la nube como Cosmos DB de Azure.

Finalmente, una vez que tenemos los datos cabe preguntarse ¿cómo los analizamos y extraemos insights? En el último módulo de la materia, cambiaremos un poco el chip y nos adentraremos en el ecosistema de Python. Usaremos Jupyter Notebooks, un entorno interactivo fantástico para experimentar, y la biblioteca Pandas. De esta manera, aprenderá a manipular grandes conjuntos de datos, filtrar, limpiar y aplicar agregaciones para descubrir patrones ocultos.

Ahora, sin más preámbulos, ¡comencemos!

Objetivos

- › Conocer los conceptos fundamentales de las bases de datos, para sentar las bases teóricas necesarias para el diseño y consulta de estructuras de almacenamiento eficientes.
- › Aplicar los comandos del lenguaje SQL, tanto DDL como DML, para la creación y manipulación de datos, con el fin de gestionar de manera autónoma la información contenida en una base de datos relacional.
- › Diferenciar los principales tipos de bases de datos NoSQL, como documentales y de grafos, y sus casos de uso, para saber cómo extraer y consultar la información de manera eficiente.
- › Manipular y analizar datos con Pandas en entornos de notebooks para extraer información valiosa que apoye la toma de decisiones.

Competencias

Relación de la asignatura con las competencias de egreso

En la tabla siguiente se establece la relación de la asignatura con las competencias de egreso: Específicas, Genéricas Tecnológicas, Sociales, Políticas y Actitudinales de la carrera. Se incluyen las competencias de egreso a las que tributa su nivel de aporte (ninguno, bajo, medio y alto)

1. Competencias Específicas

CÓDIGO	COMPETENCIAS	APORTE
CE1	Desarrollar propuestas innovadoras en modelos y estrategias.	Medio (la materia introduce enfoques modernos en gestión de datos, pero no exige innovación creativa avanzada).
CE2	Identificar problemas y diseñar acciones fundamentadas para responder a ellos.	Alto (toda la materia gira en torno a resolver problemas de almacenamiento, consulta y análisis de datos).

CE3	Emitir juicios evaluativos basados en criterios fundamentados y contextualizados.	Alto (se analiza cuándo usar SQL, cuándo NoSQL, qué modelo conviene, ACID, normalización).
CE4	Detectar tendencias y necesidades del entorno, incorporando nuevos conocimientos.	Medio (hay exposición a tecnologías actualizadas y reflexión sobre ecosistemas de datos).
CE5	Construir conocimientos y desarrollar propuestas integrando saberes diversos.	Medio (la materia integra modelos conceptuales, relacionales, NoSQL y análisis con Python).
CE6	Desenvolverse con flexibilidad ante situaciones nuevas y problemas emergentes.	Medio (la comparación entre tecnologías y la capacidad de elegir modelos aporta a la adaptación).
CE7	Trabajar con método, definiendo objetivos claros, pasos, procesos e indicadores de resultados.	Alto (la materia enseña procesos sistemáticos: modelado → normalización → consultas → análisis).

2. Competencias Genéricas Tecnológicas

CÓDIGO	DESCRIPCIÓN	APORTE
CG1	Buscar, obtener, evaluar, organizar y compartir información con herramientas digitales.	Alto (la gestión de datos estructurados y no estructurados es el núcleo de la materia).
CG2	Comunicarse, relacionarse y colaborar con otros utilizando tecnologías.	Medio (la materia implica prácticas en entornos colaborativos como notebooks, bases compartidas).
CG3	Emplear tecnologías propias de la disciplina con criterio pertinente.	Alto (SQL, NoSQL, Python, Pandas, Jupyter, modelos de datos).

3. Competencias Sociales – Políticas – Actitudinales

CÓDIGO	DESCRIPCIÓN	APORTE
CSPA1	Usar adecuadamente la lengua oral y escrita para comunicar con eficacia.	Medio (se requiere formular consultas, describir modelos y documentar procesos con claridad técnica).
CSPA2	Vincularse con empatía, negociar posiciones y trabajar por propósitos comunes.	Bajo (la materia no se centra en dinámicas socioemocionales, aunque puede promover trabajo en equipo).
CSPA3	Actuar responsablemente considerando impacto de decisiones.	Medio (la toma de decisiones sobre datos afecta seguridad, integridad y calidad de información).
CSPA4	Comprender necesidades y expectativas del entorno laboral.	Medio (el propósito menciona aplicaciones reales y necesidades actuales del mercado de datos).

CSPA5	Relacionarse constructivamente con otros, generando visiones compartidas.	Bajo (no es central, aunque puede abordarse en el proyecto integrador).
CSPA6	Mantener actitud abierta al cambio y adoptar nuevos enfoques.	Alto (la materia expone al estudiante a SQL, NoSQL, Python y nuevas tecnologías).
CSPA7	Desempeñarse con personas de diversas culturas, integrando aprendizajes.	Bajo (no se aborda en forma directa, aunque el ecosistema digital es global por naturaleza).

CÓDIGO	RESULTADO DE APRENDIZAJE
RA1	Reconocer los conceptos fundamentales de las bases de datos para comprender los principios teóricos necesarios en el diseño y consulta de estructuras de almacenamiento eficientes.
RA2	Aplicar comandos del lenguaje SQL, tanto DDL como DML, para crear, modificar y manipular datos en bases de datos relacionales de manera autónoma y eficaz.
RA3	Diferenciar los principales tipos de bases de datos NoSQL—documentales, de grafos y otras variantes— para seleccionar el modelo adecuado según el caso de uso.
RA4	Manipular y analizar datos con la biblioteca Pandas en entornos de notebooks para obtener información valiosa que apoye procesos de toma de decisiones.

Relación de los RA y las competencias

En la tabla siguientes se indica el aporte de cada RA con la competencia de egreso: Específicas, Genéricas Tecnológicas, Sociales Políticas y Actitudinales.

1. Matriz De Aporte – Competencias Específicas (CE)

CÓDIGO	RESULTADO DE APRENDIZAJE	CE1	CE2	CE3	CE4	CE5	CE6	CE7
RA1	Reconocer los conceptos fundamentales de las bases de datos para comprender los principios teóricos necesarios en el diseño y consulta de estructuras de almacenamiento eficientes.	Medio	Alto	Alto	Medio	Medio	Medio	Alto

RA2	Aplicar comandos del lenguaje SQL, tanto DDL como DML, para crear, modificar y manipular datos en bases de datos relacionales de manera autónoma y eficaz.	Medio	Alto	Medio	Medio	Bajo	Medio	Alto
RA3	Diferenciar los principales tipos de bases de datos NoSQL documentales, de grafos y otras variantes para seleccionar el modelo adecuado según el caso de uso.	Medio	Alto	Alto	Medio	Bajo	Medio	Medio
RA4	Manipular y analizar datos con la biblioteca Pandas en entornos de notebooks para obtener información valiosa que apoye procesos de toma de decisiones.	Medio	Alto	Medio	Medio	Bajo	Medio	Medio

Notas clave:

- › CE2 y CE3 reciben aporte alto por la naturaleza analítica y de resolución de problemas de la materia.
- › CE7 también recibe aporte alto porque todos los RA requieren método, procesos y pasos estructurados.

2. Matriz De Aporte – Competencias Genéricas Tecnológicas (CG)

CÓDIGO	RESULTADO DE APRENDIZAJE	CG1	CG2	CG3
RA1	Reconocer los conceptos fundamentales de las bases de datos para comprender los principios teóricos necesarios en el diseño y consulta de estructuras de almacenamiento eficientes.	Medio	Bajo	Medio
RA2	Aplicar comandos del lenguaje SQL, tanto DDL como DML, para crear, modificar y manipular datos en bases de datos relacionales de manera autónoma y eficaz.	Alto	Medio	Alto
RA3	Diferenciar los principales tipos de bases de datos NoSQL —documentales, de grafos y otras variantes— para seleccionar el modelo adecuado según el caso de uso.	Medio	Bajo	Medio
RA4	Manipular y analizar datos con la biblioteca Pandas en entornos de notebooks para obtener información valiosa que apoye procesos de toma de decisiones.	Alto	Medio	Alto

Notas clave:

- › RA2 y RA4 aportan alto porque implican manejo avanzado de tecnologías disciplinares (SQL, Pandas).
- › CG2 mantiene aporte medio por el trabajo en entornos digitales colaborativos (notebooks, bases compartidas).

3. Matriz De Aporte – Competencias Sociales, Políticas y Actitudinales (CSPA)

CÓDIGO	RESULTADO DE APRENDIZAJE	CSPA1	CSPA2	CSPA3	CSPA4	CSPA5	CSPA6	CSPA7
RA1	Reconocer los conceptos fundamentales de las bases de datos para comprender los principios teóricos necesarios en el diseño y consulta de estructuras de almacenamiento eficientes.	Medio	Bajo	Medio	Medio	Bajo	Medio	Bajo
RA2	Aplicar comandos del lenguaje SQL, tanto DDL como DML, para crear, modificar y manipular datos en bases de datos relacionales de manera autónoma y eficaz.	Medio	Bajo	Medio	Medio	Bajo	Medio	Bajo
RA3	Diferenciar los principales tipos de bases de datos NoSQL documentales, de grafos y otras variantes para seleccionar el modelo adecuado según el caso de uso.	Medio	Bajo	Medio	Medio	Bajo	Medio	Bajo
RA4	Manipular y analizar datos con la biblioteca Pandas en entornos de notebooks para obtener información valiosa que apoye procesos de toma de decisiones.	Medio	Bajo	Medio	Medio	Bajo	Alto	Bajo

Notas clave:

- › CSPA6 recibe aporte alto en RA4 por el enfoque en herramientas modernas y adaptación tecnológica.
- › El resto mantienen niveles medio o bajo, ya que la materia se centra en aspectos técnicos.

Sistema De Correlatividades

En la siguiente tabla se muestra los prerrequisitos y la correlativa posterior este espacio curricular:

CUATRIMESTRE	CÓDIGO	ASIGNA-TURA	ABREVI-A-TURA	HORAS SEMANALES	HORAS TOTALES	ANTERIOR	POSTERIOR
I	0103	Base de Datos	BD	6	90	--	

Contenidos mínimos

Sistemas de información. Evolución de las tecnologías de bases de datos, conceptos básicos de bases de datos. Diseño de bases de datos: diseño conceptual, lógico y físico, modelo entidad/relación. Modelo relacional, lenguajes de consulta, SQL, normalización.

Contenidos

Módulo 1: Bases de datos relacionales

- › Historia de las bases de datos. Tablas, relaciones, constraints, primary keys, normalización, transacciones, concepto ACID.

Módulo 2: SQL

- › DML, DDL, Querys iniciales, select, where, group by, join, order by, having.

Módulo 3: NO-SQL

- › MongoDB. Bases de Datos de Grafos. Ejemplos propietarios (Cosmos DB Azure).

Módulo 4: Pandas y notebooks en Python

- › Manipulación de datos en notebooks con pandas, aplicar agregaciones, descubrir el poder de Python y Pandas para la exploración y la ciencia de datos.

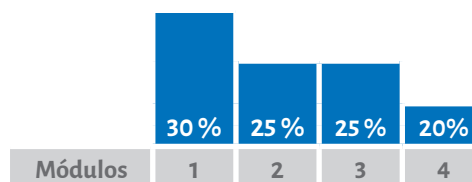
Bibliografía

Obligatoria

- › Material didáctico de producción institucional diseñado y desarrollado especialmente para cada módulo.

Planificación

Porcentaje estimativo por módulo según la cantidad y complejidad de contenidos y actividades:



Representación de porcentajes en semanas:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
MÓDULO 1				MÓDULO 2				✓	MÓDULO 3			MÓDULO 4		✓

✓ Evaluación integradora

Metodología

Forma en que se llevará adelante el proceso de enseñanza y aprendizaje, buscando alcanzar los objetivos de aprendizaje y el cumplimiento de las competencias de la materia.

Speech general que cada uno pueda agregar alguna particularidad de la asignatura:

A) Estrategias de enseñanza: Este espacio curricular fue diseñado según lo previsto en el Sistema Institucional de Educación a Distancia (SIED) de la UBP (Resolución MECCYT 197/2019), las características de los destinatarios, las necesidades de esta disciplina, competencias que se espera promover en esta asignatura y el perfil del egresado.

En relación con lo anterior, para llevar a cabo la enseñanza se cuenta con:

- › Contenidos y actividades de aprendizaje desarrollados en la plataforma virtual miUBP.
- › Material bibliográfico especialmente seleccionado.
- › Clases en video, en las que el docente expone contenidos que considera relevantes y que pueden ser visualizadas en cualquier momento.
- › Tutorías virtuales que implican diferentes espacios de interacción.

B) Materiales curriculares: Durante el cursado, se fomenta el estudio de los contenidos (presentados en diferentes soportes y lenguajes) que componen el programa, y la resolución de las actividades de aprendizaje. De esta forma, se introduce al alumno en los conceptos que se deben dominar en todo lo que respecta a Base de

Datos. También, se invita a leer bibliografía que enriquece y profundiza dicho desarrollo. Por otra parte, se propone la resolución de actividades de aprendizaje tanto de carácter optativo como obligatorio que se encuentran debidamente identificadas en la plataforma. Algunas de ellas se centran en la apropiación de conceptos teóricos básicos de la asignatura y otras en la transferencia mediante un abordaje práctico a través del análisis de casos, ejercitaciones y situaciones problemáticas.

C) Modalidad de agrupamiento: En esta asignatura se proponen principalmente instancias de producción individual, en coherencia con los objetivos y contenidos a trabajar. De esta manera, el alumno puede desenvolverse en forma autónoma, reconociendo sus fortalezas y debilidades, a la vez que puede desarrollar sus propias estrategias de aprendizaje. Es importante destacar que para el estudio de la asignatura los alumnos pueden optar por agruparse libremente y resolver las actividades de aprendizaje planteadas.

D) Interacción: La comunicación con el docente tutor se realiza a través de los diferentes medios disponibles en la plataforma miUBP.

E) Formación Práctica: En lo que se refiere a la formación práctica se hace foco en la resolución de problemas que implique el uso de los contenidos estudiados en la asignatura contextualizados en situaciones que tengan que ver con la carrera.

Evaluación

La metodología de evaluación propuesta se presenta conforme al Reglamento Académico General de la UBP.

Cada instrumento de evaluación contiene sus criterios de evaluación explicitados y el puntaje otorgado a cada consigna.

En cuanto a los criterios de acreditación, la escala cuantitativa utilizada para evaluar es del 1 al 100 y el puntaje mínimo que se requiere para la aprobación es de 50.

Regularidad

Para alcanzar la condición de alumno regular usted deberá aprobar la Evaluación Integradora dentro de los plazos previstos para el cursado.

Si reprueba esta instancia podrá reelaborar la evaluación según los requerimientos del docente.

Examen Final

Deberá rendir, en condición de regular, una evaluación final que se realizará a través de un examen escrito integrador, presencial, en el periodo que disponga la Universidad.

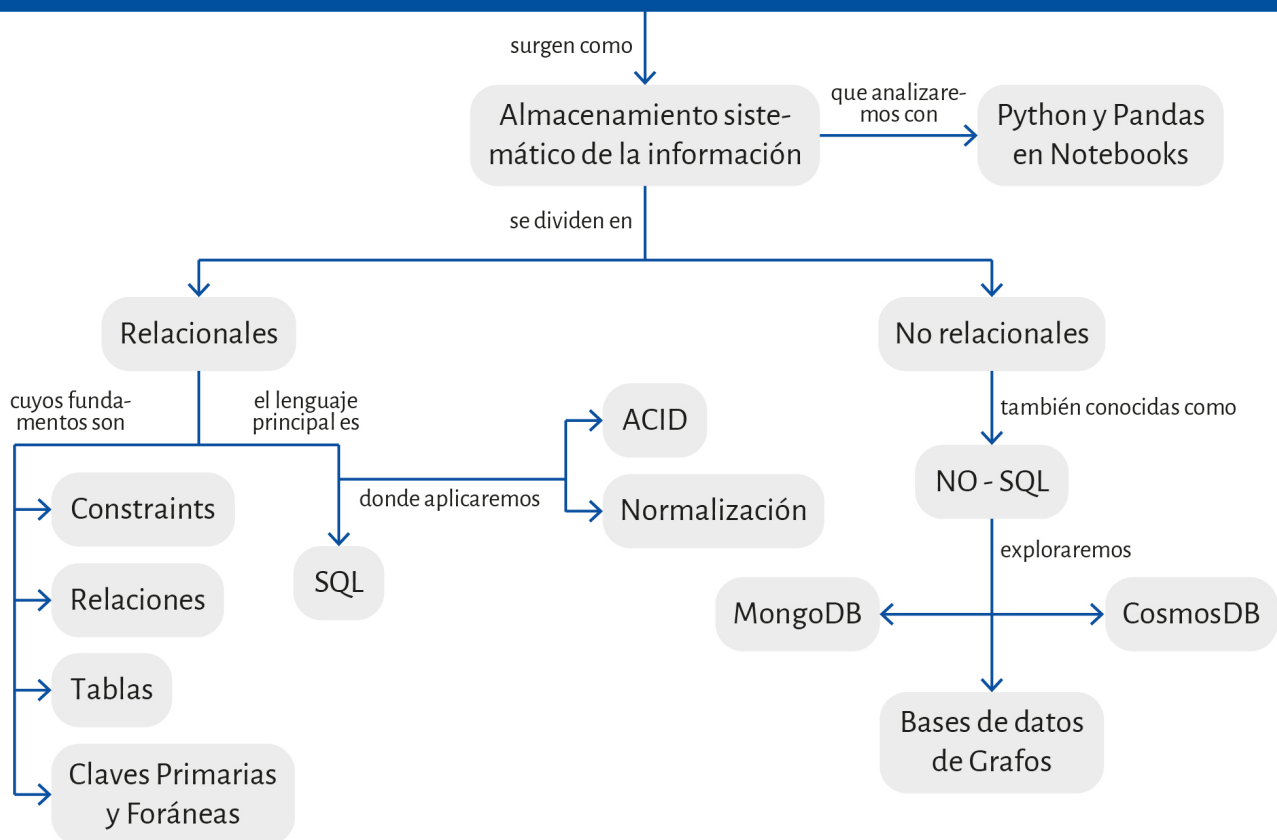
Mapa conceptual



A continuación, lo/a invitamos a escuchar el recorrido por el mapa conceptual de la materia propuesto por el docente.



Base de Datos



MÓDULO 1

Microobjetivos

- › Conocer la evolución histórica y los fundamentos del modelo relacional, para comprender el contexto y los principios que rigen las bases de datos modernas.
- › Diferenciar los componentes estructurales del modelo relacional (relaciones, atributos, tuplas) y saber aplicar los conceptos de claves e integridad referencial, para interpretar esquemas de bases de datos y garantizar la identificación única y consistencia de los datos.
- › Diseñar diagramas entidad-relación en sus niveles conceptual, lógico y físico, para plasmar visualmente la estructura de una base de datos antes de su implementación.
- › Normalizar esquemas de bases de datos aplicando las formas normales desde 1FN hasta 3FN, con el fin de eliminar redundancias y anomalías, asegurando un diseño robusto y eficiente.
- › Comprender el significado y los principios ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) que rigen las transacciones en bases de datos, para garantizar la procesamiento confiable y seguro de los datos en cualquier sistema.



VIDEO DOCENTE

Contenidos

- Bases de Datos Relacionales



Fuente: <https://www.freepik.es/>

¡Bienvenido/a al módulo 1!

En este módulo, abordaremos el estudio del modelo relacional, y por consiguiente, el mundo de las bases de datos relacionales. Aprenderemos qué es lo que hace que este modelo sea tan utilizado, y veremos sus principales características. Al finalizar este módulo, tendrán todas las herramientas para poder entender un diagrama entidad-relación y qué significa cada una de sus partes.

› Historia de las bases de datos

Para empezar, podemos dividir la historia de las bases de datos en décadas.



Para empezar, podemos dividir la historia de las bases de datos en décadas, lo/a invitamos a ver el siguiente material: <https://view.genially.com/691216f452ec5b3db17057ba>

• Modelo relacional

Tal como venimos planteando, en este módulo nos centraremos en el *modelo relacional*. En la sección anterior mencionamos que dicho modelo fue inventado por Edgar Frank Codd en 1970. Lo que Codd buscaba era una forma de modelar los datos que no implique atar el modelo a la estructura física en la que se guardaban los datos, es decir, él buscaba desacoplar la estructura física de los datos (los archivos donde se guardaban), de la estructura lógica.

Para hacer esto, Codd se basó en un concepto matemático llamado relaciones, que viene de la teoría de conjuntos. Podemos pensarlo de la siguiente manera: una relación contiene atributos y tuplas.

Puede sonar complicado de esta manera, pero esto es simplemente la teoría que Codd desarrolló, en la práctica, solemos verlo plasmado de esta forma: una tabla (relación) está compuesta por un conjunto de columnas (atributos) y contiene un conjunto de filas o registros (tuplas).

› Tablas

Avancemos con un ejemplo:

Tabla Estudiantes					
Atributos o Columnas					
dni	nombre	apellido	carrera	fecha inicio	materias aprobadas
12345678	Ana	García	Ingeniería Informática	01/09/2022	12
87654321	Carlos	Martínez	Administración de Empresas	15/09/2021	20
11223344	Elena	López	Derecho	10/09/2020	35
44332211	David	Rodríguez	Medicina	05/09/2023	5
55667788	María	Sánchez	Psicología	01/09/2022	15
99887766	Javier	Fernández	Ingeniería Civil	15/09/2021	22
66554433	Laura	Pérez	Ciencia de Datos	05/09/2023	8
22334455	Pablo	González	Biología	10/09/2020	40
33445566	Sara	Díaz	Educación Infantil	01/09/2022	18
78787878	Daniel	Ruiz	Economía	15/09/2021	25

Fuente: elaboración propia.

En la imagen se observa una tabla llamada Estudiantes, que representa una relación de base de datos. Esta tabla está organizada en seis columnas o atributos, que son DNI, nombre, apellido, carrera, fecha de inicio y materias aprobadas. Cada una de estas columnas define el tipo de información que se almacena sobre los estudiantes. A su vez, aparecen diez filas o tuplas, que corresponden a los registros concretos de distintos estudiantes, es decir, cada fila reúne en un mismo conjunto los datos particulares de una

persona. De esta forma, la tabla combina la estructura general de los atributos con los valores específicos de cada estudiante, mostrando cómo se organizan los datos en el modelo relacional.

- **Grado y cardinalidad**

A la cantidad de atributos que una relación posee, se le llama “*grado*”, y a la cantidad de filas, se le llama “*cardinalidad*”.

Siguiendo entonces con el ejemplo, nuestra tabla Estudiantes tiene *grado* 6 y *cardinalidad* 10.

- **Dominio**

Ahora, ¿qué pasaría si encontráramos una fila que en la columna “nombre” contiene el siguiente valor “01/09/2021”? Sería raro, ¿no? Ya que este no es un nombre de una persona, es una fecha.

Por ello, cada columna posee un “*dominio*”, es decir, una lista de posibles valores que esa columna puede tomar. En el caso de la columna “nombre”, su dominio sería “cadenas de caracteres”, si hablamos de la columna “fecha inicio”, su dominio sería “fechas”.

- › **Clave primaria (Primary Key)**

Una *clave primaria* es un atributo (o un conjunto de atributos) que identifica de manera única e inequívoca cada fila dentro de una tabla.

Para que un atributo o conjunto de atributos sea una clave primaria, debe cumplir dos condiciones fundamentales:

- › No pueden existir dos tuplas en la tabla con el mismo valor en la clave primaria. Cada valor debe ser único. Por ejemplo, si en una tabla de estudiantes el atributo dni es la clave primaria, no es posible que aparezcan dos estudiantes con el mismo número de documento, como “12345678” en dos filas distintas.
- › La clave primaria no puede contener valores NULL (nulos). Es obligatorio que cada fila tenga un valor en este campo para garantizar su identificación. Volviendo al ejemplo anterior, si el campo dni está vacío en una fila, ese estudiante no tendría forma de ser identificado de manera inequívoca, lo que rompe la condición de clave primaria.

Consideremos el siguiente ejemplo, la tabla tutores:

numero_legajo	nombre	apellido	email	fecha_nacimiento
1001	Ana	García	ana.garcia@mail.com	15/05/2000
1002	Luis	Martínez	luis.m@mail.com	22/08/1999
1003	Carlos	López	c.lopez@mail.com	10/02/2001
1004	Ana	García	ana.g2@mail.com	30/11/2000

Fuente: elaboración propia.

Si nosotros quisiéramos saber cuál es el email de la tutora “Ana García”, ¿cómo sabríamos a cuál nos referimos? La respuesta es que, con los datos que tenemos a nuestra disposición para realizar la búsqueda (nombre y apellido) nos sería imposible, ya que estos valores no cumplen las condiciones necesarias para identificar de manera única a una fila. En cambio, si supiéramos el número de legajo, si podríamos saber a cuál nos referimos, ya que el número de legajo si identifica de manera única a cada fila.

› **Clave alternativa o clave candidata (Alternate/Candidate Key)**

Siguiendo el ejemplo de arriba, vemos que la columna de email también podría ser una clave primaria, ya que cumple los requisitos para serlo, pero por cuestiones de diseño, se eligió a la columna de numero legajo para ser esa clave primaria. Esto no quita que email cumpla las condiciones, por esto es por lo que podemos llamar a esa columna como “clave alternativa” o “clave candidata”. Es una columna o conjunto de columnas (que no sean la clave primaria) que identifican de manera única a una fila. Otro ejemplo de clave candidata sería Nombre y Fecha de Nacimiento, ya que juntas nos permiten identificar a cada fila distintivamente.

› **Clave foránea (Foreign Key)**

Una **clave foránea** es un atributo (o conjunto de atributos) dentro de una tabla que actúa como una referencia a la clave primaria de otra tabla. Su propósito fundamental es establecer una relación entre los datos de dos tablas distintas, garantizando lo que se conoce como integridad referencial.

Las claves foráneas se rigen por tres características que garantizan su correcto funcionamiento. En primer lugar, una clave foránea debe hacer referencia específicamente a una clave primaria de otra tabla. Esto significa que no puede apuntar a cualquier columna; por ejemplo, en una tabla Cursos, la columna legajo_tutor debe referenciar inequívocamente al atributo numero_legajo de la tabla Tutores para identificar al tutor responsable.

En segundo lugar, ambos atributos deben pertenecer al mismo dominio, es decir, compartir el mismo tipo de dato y significado. Si la clave primaria numero_legajo se define como un entero, la clave foránea legajo_tutor también debe ser un valor entero, asegurando así la consistencia de la relación.

Finalmente, a diferencia de la clave primaria, una clave foránea sí puede contener valores NULL. Esto nos permite modelar relaciones opcionales, donde la asociación entre los registros puede no ser obligatoria. Por ejemplo, un registro en la tabla Cursos podría tener NULL en la columna legajo_tutor, indicando que ese curso en particular aún no tiene un tutor asignado, lo cual es una situación válida dentro del modelo. La decisión final de si la clave foránea contiene o no valores NULL, dependerá de la relación que se está modelando.



Énfasis

La función más crítica de una clave foránea es mantener la *consistencia de los datos*. La regla de integridad referencial establece que no puede existir un valor en una clave foránea que no aparezca como valor en la clave primaria a la que referencia. En otras palabras, la base de datos nos impedirá cometer inconsistencias.

Siguiendo nuestro ejemplo, podemos tener un curso con `legajo_tutor = 205` solo si existe previamente un tutor con `numero_legajo = 205` en la tabla Tutores; si intentáramos insertar un curso con `legajo_tutor = 999` sin que exista un tutor con ese legajo, la base de datos rechazaría la operación.



En este momento, lo/a invitamos a participar de la primera actividad del módulo 1: “*Conceptos básicos del modelo relacional*”.

- **Modelo entidad-relación**

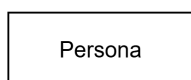
En este módulo, nos centraremos en el **modelo entidad-relación (E-R)**. Este modelo fue propuesto por Peter Chen en 1976 como una forma de representar conceptualmente los datos antes de implementarlos en una base de datos relacional.

Lo que Chen buscaba era una forma de visualizar y estructurar la información de manera lógica, enfocándose en cómo los datos se relacionan entre sí, sin preocuparse aún por cómo se almacenan físicamente. Es decir, el Modelo E-R nos permite diseñar la base de datos desde una perspectiva conceptual, antes de pasar a su implementación técnica.

- **Entidad**

En el contexto del modelo entidad-relación, una entidad representa cualquier elemento del mundo real o conceptual que deseemos incluir en nuestro sistema de información. Puede tratarse de algo físico, como una persona, un libro o un vehículo; o de algo abstracto, como un cargo laboral, una asignatura o una reserva.

Se lo suele representar como un rectángulo con el nombre de la entidad dentro:



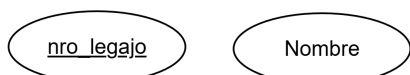
- **Atributo**

Cada entidad está compuesta por atributos, que son las características que la describen. Los atributos permiten identificar, clasificar o cuantificar a una entidad, y son esenciales para distinguir entre diferentes instancias de una misma entidad.

Por ejemplo, la entidad “estudiante” podría tener los siguientes atributos:

- › Nombre
- › Edad
- › Género
- › Número de legajo

Se suele representar como un óvalo. Además, si el nombre está subrayado significa que este atributo es una clave primaria:



- **Relación**

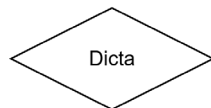
Una relación describe cómo dos o más entidades están conectadas entre sí dentro del sistema. Es decir, nos permite representar interacciones o asociaciones entre entidades.

Por ejemplo:

- › Un estudiante se inscribe en una materia.
- › Un profesor dicta un curso.

Estas relaciones también pueden tener *atributos propios*, como la fecha de inscripción o el horario de clase, y son fundamentales para entender el comportamiento del sistema de información.

Se suele representar como un rombo:



Nota importante: El concepto de “relación” en el modelo entidad-relación no debe confundirse con el de “relación” en el modelo relacional. En el modelo relacional, una “relación” se refiere a una tabla compuesta por filas y columnas. En cambio, en el modelo entidad-relación, una “relación” representa una conexión lógica entre entidades, que luego puede ser implementada como una tabla en el modelo relacional, pero no son equivalentes en su definición conceptual.

- **Cardinalidad**

La cardinalidad indica cuántas veces una entidad puede participar en una relación. Existen tres tipos principales:

- › **Uno a uno (1:1):** Un estudiante tiene un único tutor.
- › **Uno a muchos (1: N):** Un profesor puede dictar muchas materias.
- › **Muchos a muchos (N: N):** Un estudiante puede inscribirse en varias materias, y cada materia puede tener varios estudiantes.

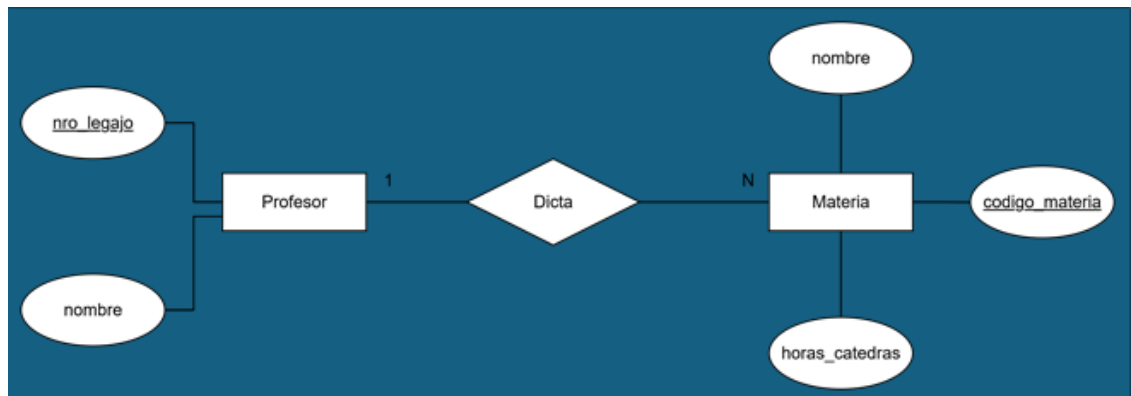
La cardinalidad es clave para entender la estructura de los datos y cómo se deben modelar las relaciones. Por ejemplo, hagamos una demostración de cómo se ve un diagrama entidad-relación con todas las partes ensambladas.

Nuestra problemática es la siguiente: debemos realizar un diagrama donde se modele el sistema de asignación de materias a profesores de la universidad. Un profesor tiene un número de legajo que lo identifica y su nombre. La materia tiene un nombre, un código de materia que la identifica y la cantidad de horas cátedras asignadas a esa materia. Igualmente, un profesor puede dictar muchas materias.

Como primer ejercicio, podemos identificar los atributos, entidades y relaciones que participan en este sistema:

- › **Primera entidad:** profesor. Sus atributos son: número de legajo y nombre.
- › **Segunda entidad:** materia. Sus atributos son: nombre, código de materia y horas cátedras.
- › **Primera relación:** dicta. Un Profesor puede dictar muchas materias.
- › **Cardinalidad:** Uno (profesor) a muchos (materia).

Con este análisis hecho, nuestro diagrama podría verse de la siguiente forma:



Fuente: elaboración propia.

Aquí podemos ver cómo los atributos *nro_legajo* y *codigo_materia* identifican a la entidad Profesor y Materia respectivamente. Vemos también cómo es la relación entre un Profesor y una Materia, y cómo las cardinalidades son aplicadas. Empezando el diagrama en Profesor y leyéndolo hacia la derecha, podría leerse de la siguiente manera:

- › Un Profesor → (Entidad)
- › Dicta → (Relación)
- › Muchas → (Cardinalidad)
- › Materias → (Entidad)

Ahora, leyéndolo de izquierda a derecha, podemos verlo de la siguiente manera.

- › Una Materia → (Entidad)
- › Es dictada → (Relación)
- › Por un → (Cardinalidad)
- › Profesor → (Entidad)

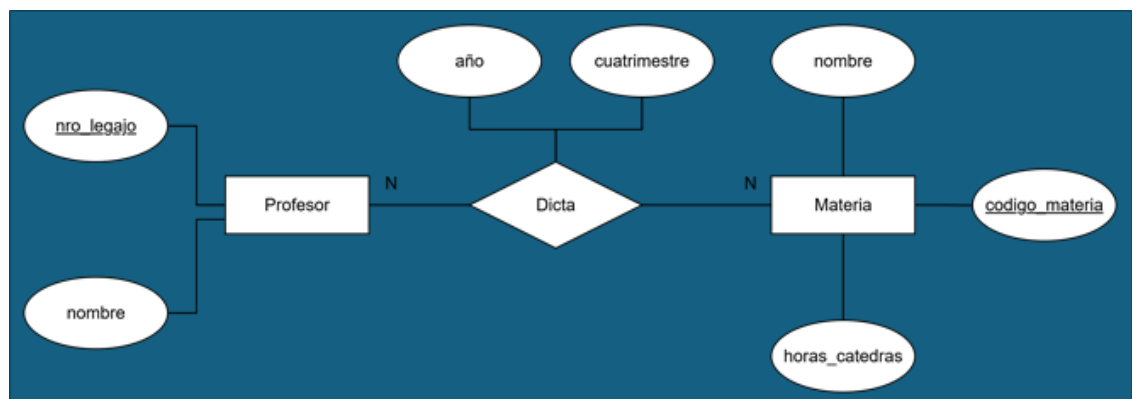
Ahora, recibimos un pedido de modificación al sistema debido a que un profesor dejará de dictar una materia a partir del próximo cuatrimestre. Sin embargo, es necesario conservar el registro histórico de que fue docente de esa asignatura en un determinado año y cuatrimestre. Este cambio requiere ajustar la relación Dicta, que vincula a los profesores con las materias, para que el sistema pueda almacenar tanto los vínculos actuales como los del pasado.

Además, se debe contemplar que un profesor puede dictar varias materias y que una misma materia puede haber sido impartida por distintos docentes en diferentes períodos, configurando así una relación de tipo muchos a muchos entre las entidades Profesor y Materia.

El cambio podría ser el siguiente:

- › **Primera relación:** dicta. Un Profesor puede dictar muchas materias.
- › **Cardinalidad:** Muchos (profesor) a muchos (materia).
- › **Atributos:** Año en que se dicta y cuatrimestre en el que se dicta.

De esta manera, el diagrama actualizado podría verse así:



Fuente: elaboración propia.

Vemos como ahora la cardinalidad paso a ser “muchos a muchos”, y la relación adquirió 2 atributos: año y cuatrimestre.

• Diagrama entidad-relación

Finalmente, todo esto se representa gráficamente en un **Diagrama E-R**, el mismo es una herramienta poderosa para diseñar bases de datos relacionales de forma clara y estructurada antes de implementarlas.

Comúnmente, hablamos de **3 niveles de abstracción** a la hora de crear Diagramas E-R, iremos explorando los diferentes niveles y para qué sirve cada uno.

1. Nivel conceptual

Este es el nivel más alto de abstracción. Su propósito principal es mostrar el alcance general del sistema y representar la arquitectura de la información sin entrar en detalles técnicos.

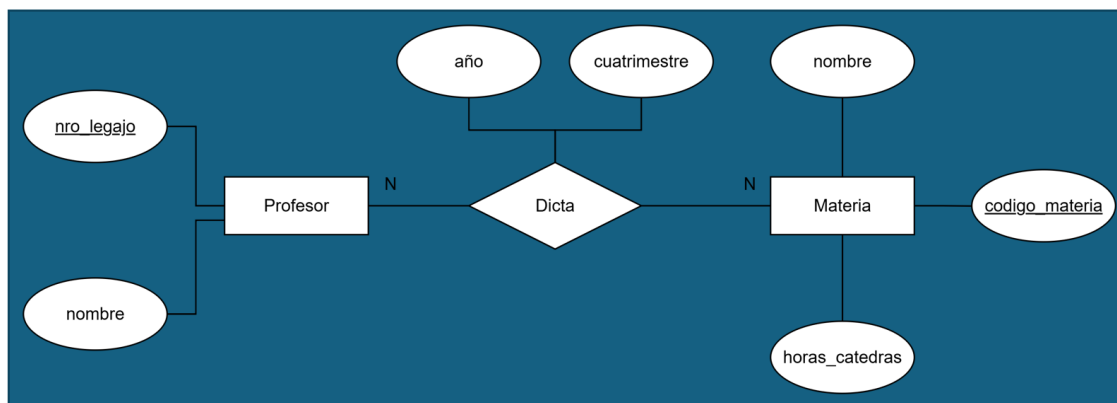
En este nivel:

- › Se identifican las entidades principales, sus atributos clave y las relaciones fundamentales.
- › No se especifican tipos de datos ni estructuras físicas.

- › Se comunica la idea general del sistema a usuarios no técnicos o en etapas tempranas del proyecto.

Además, en sistemas de menor complejidad, este nivel puede omitirse y comenzar directamente con el modelo lógico.

Como ejemplo, podemos tomar el que hicimos anteriormente:



Fuente: elaboración propia.

2. Nivel lógico

En este nivel se comienza a pensar en cómo el modelo conceptual se puede traducir a una base de datos relacional.

Así, podemos decir que:

- › Las entidades se transforman en tablas.
- › Los atributos se convierten en columnas.
- › Se definen las claves primarias y foráneas.
- › Se ajustan las relaciones para que puedan ser representadas en el modelo relacional.

Este nivel es clave para preparar el modelo para su implementación en un sistema de gestión de bases de datos (DBMS).

También vemos que las relaciones pasan de utilizar la notación "N:N" o "1:N" a utilizar figuras como las siguientes:



Esta figura indica una relación de muchos opcional, es decir, la entidad puede participar, cero, una o muchas veces de la relación. Por ejemplo: un cliente puede realizar ninguna, una o varias compras.



Esta figura, en cambio, indica una relación de muchos obligatoria, es decir, la entidad debe participar al menos una vez en la relación, y puede hacerlo muchas veces. Por ejemplo: un curso debe tener al menos un estudiante inscrito, pero puede tener muchos.



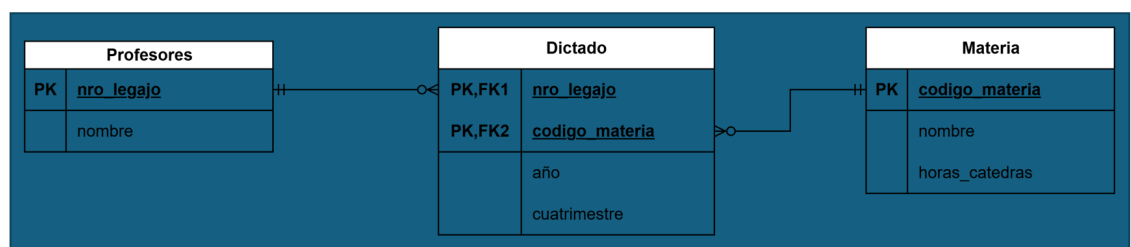
Por su parte, esta figura indica una relación de uno a uno opcional, es decir, la entidad puede participar, cero o una vez en la relación. Por ejemplo: un empleado puede tener cero o un supervisor.



Finalmente, esta figura indica una relación de uno a uno obligatoria, es decir, la entidad debe participar exactamente una vez en la relación. Un ejemplo de este caso puede ser: cada persona debe tener un número de documento único.

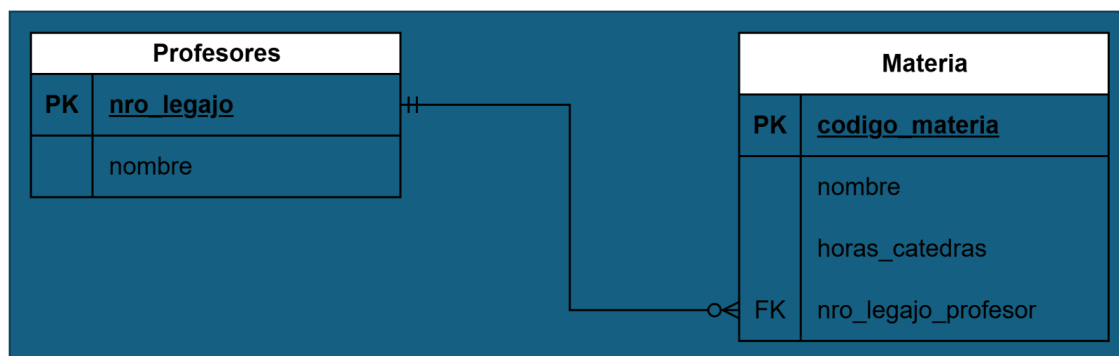
Cabe aclarar que existen muchos tipos de notaciones, la que utilizaremos para el nivel lógico será la notación explicada anteriormente, llamada *"Crow's Foot"*.

En nuestro ejemplo, la relación "Dicta" se convierte en una nueva tabla "Dictado", ya que las relaciones "Muchos a muchos" no se pueden implementar directamente en el modelo relacional. Y gráficamente se vería de la siguiente manera:



Fuente: elaboración propia.

En cambio, si hubiéramos mantenido el diagrama anterior, donde la relación era "Uno a muchos", este podría diagramarse así:



Fuente: elaboración propia.

En este caso, como la entidad “Materia” solo se relaciona con un “Profesor”, podemos agregar el atributo “nro_legajo_profesor” a la tabla de “Materia” para poder referenciar al número de legajo del profesor que dicta esta materia.



Énfasis

Es importante destacar 2 cosas:

- › Las claves primarias se destacan del resto, indicándolas con las siglas **PK (Primary Key)** y ubicándolas en la zona superior de la tabla.
- › Las claves foráneas se indican con las siglas **FK (Foreign Key)**, y la unión de la relación se realiza entre la Primary Key de la tabla origen y la Foreign Key de la tabla destino.

3. Nivel físico

Finalmente, el nivel físico se enfoca en cómo se almacenarán los datos en el sistema. Aquí:

- › Se definen los tipos de datos específicos (por ejemplo, VARCHAR, INT, DATE).
- › Se adapta el modelo lógico a las limitaciones y características del DBMS elegido.

Así podría verse un diagrama E-R en su nivel físico:



Fuente: elaboración propia.

En el siguiente módulo, exploraremos más en profundidad este diagrama y que significa cada detalle que se ha agregado.



Actividad

Luego de leer el desarrollo conceptual de este apartado, se encuentran en condiciones de realizar la Actividad 2: “Modelo Entidad Relación”.

- **Normalización**

La normalización es un proceso dentro del diseño de bases de datos que busca organizar los datos de manera eficiente, disminuyendo la redundancia y mejorando la integridad. Se realiza a través de una serie de reglas llamadas formas normales, que se aplican progresivamente.

Cada forma normal tiene requisitos específicos que deben cumplirse para que una tabla esté correctamente estructurada. A medida que avanzamos en las formas normales, el diseño se vuelve más refinado y robusto.

1. Primera forma normal



Requisito: Cada celda debe contener un solo valor atómico, es decir, no debe poseer más de un valor por celda.

Supongamos que tenemos la siguiente tabla inscripciones:

Estudiante	Cursos	Profesor(es)	Legajo Profesor	Fecha(s)
Ana	Matemática, Física	Gómez, Pérez	101, 102	03/03, 02/03
Luis	Química	Torres	103	03/03

Aquí tenemos múltiples valores en una sola celda, lo que genera redundancia y dificulta el manejo de los datos. Si quisiera filtrar la tabla por las inscripciones que se realizaron el “03/03” sería muy complicado, ya que tendría que hacer un procesamiento extra para conseguir los datos que están en una misma celda.

Su transformación para llevarla a primera forma normal debe separar los valores no atómicos de las celdas. Esto sería:

Estudiante	Curso	Profesor	Legajo Profesor	Fecha
Ana	Matemática	Gómez	101	03/03
Ana	Física	Pérez	102	02/03
Luis	Química	Torres	103	03/03

Ahora cada celda contiene un único valor, cumpliendo con 1FN (Primera Forma Normal).

2. Segunda forma normal



Requisito: Estar en Primera Forma Normal y eliminar dependencias parciales (atributos que dependen solo de parte de la clave candidata).

Siguiendo el ejemplo de la Primera Forma Normal, la transformación parte de identificar que la clave candidata está compuesta por los atributos Estudiante y Curso, formando así una clave candidata (Estudiante, Curso). Al analizar la estructura, se observa que los atributos Profesor y Legajo Profesor dependen únicamente del atributo Curso, podemos denotarlo así:

- › (Curso) -> (Profesor) Curso determina a Profesor
- › (Curso) -> (Legajo Profesor) Curso determina a Legajo Profesor

Esto evidencia la existencia de una dependencia parcial dentro de la relación.

La solución es separar las columnas en 2 tablas de la siguiente manera:

› Inscripciones

Estudiante	Curso	Fecha
Ana	Matemática	03/03
Ana	Física	02/03
Luis	Química	03/03

› Cursos

Curso	Profesor	Legajo Profesor
Matemática	Gómez	101
Física	Pérez	102
Química	Torres	103

Ahora, los datos del profesor dependen solo del curso, no del estudiante, cumpliendo con la Segunda Forma Normal.

3. Tercera forma normal



Requisito: Estar en Segunda Forma Normal y eliminar dependencias transitivas (atributos que dependen de otros atributos no clave).

Una dependencia transitiva se puede explicar de la siguiente forma:
Tenemos las siguientes dependencias:

- › (A) -> (B) A determina a B
- › (B) -> (C) B determina a C

En este esquema, se puede decir que A *determina a C de manera indirecta*.

- › (A) -> (C) A determina a C

Esto es a lo que llamamos dependencia transitiva.

En este caso, se observa que en la tabla “Cursos” el “Legajo de Profesor” determina el nombre del “Profesor”; aquí nos damos cuenta de que la dependencia (Curso) $\rightarrow$ (Profesor) es una dependencia transitiva. Para eliminar esta dependencia, la transformación consiste en crear una nueva tabla denominada “Profesores”, en la cual el atributo “Legajo Profesor” funcione como clave primaria y se incluya junto a él el nombre del “Profesor”.

› *Cursos*

<i>Curso</i>	<i>Legajo Profesor</i>
Matemática	101
Física	102
Química	103

› *Profesores*

<i>Legajo Profesor</i>	<i>Profesor</i>
101	Gómez
102	Pérez
103	Torres

De esta forma, el nombre del “Profesor” depende del “Legajo Profesor” de su propia tabla, en vez de depender de el “Legajo Profesor” que se encuentra en la tabla “Cursos”, que no es un atributo clave.

4. *Formas normales avanzadas*

Estas formas normales abordan problemas muy específicos que son raros en aplicaciones prácticas. La mayoría de las bases de datos bien diseñadas que cumplen con 3FN ya satisfacen implícitamente estos niveles superiores.

› *Forma normal Boyce-Codd*



Énfasis

Para que una tabla esté en la Forma Normal Boyce-Codd (FNBC), debe encontrarse en la Tercera Forma Normal (3FN) y cumplir además con que, para cualquier dependencia funcional no trivial $(X) \rightarrow (Y)$, el conjunto de atributos X que funciona como determinante sea una clave candidata. Esto garantiza que dicho conjunto actúe como un identificador único de cada registro. En otras palabras, si un atributo determina a otro, debe ser una clave candidata.

Supongamos una tabla en la que cada aula está asignada a un único profesor. Un profesor puede tener varias aulas, pero cada aula solo tiene un profesor.

En este caso, la clave primaria está compuesta por los atributos Curso y Aula, y se verifican las siguientes dependencias funcionales:

- › $(Curso, Aula) \rightarrow Profesor$
- › $(Aula) \rightarrow Profesor$

Tabla asignaciones (en 3FN, pero no en FNBC)

Curso	Aula	Profesor
Matemática	A101	Gómez
Física	A102	Pérez
Química	A101	Gómez

En este ejemplo, la tabla se encuentra en tercera forma normal, pero no cumple con los requisitos de la forma normal Boyce-Codd. El problema radica en la dependencia funcional $Aula \rightarrow Profesor$, que infringe el principio fundamental de la FNBC. Este principio establece que todo determinante debe ser una clave candidata, condición que el atributo Aula no cumple, ya que por sí solo no identifica de manera única cada registro de la tabla.

- **Transformación**

- › *Curso_Aula:*

Curso	Aula
Matemática	A101
Física	A102
Química	A101

- › *Aula_Profesor:*

Aula	Profesor
A101	Gómez
A102	Pérez

Mediante esta transformación se eliminó la dependencia funcional $Aula \rightarrow Profesor$, logrando que ambas tablas cumplan con los requisitos de la Forma Normal Boyce-Codd y evitando la redundancia de datos.

- **Cuarta forma normal (4FN)**

La cuarta forma normal tiene como objetivo eliminar dependencias multivaluadas independientes. Este tipo de dependencias aparece cuando una tabla contiene dos o más atributos multivaluados que son independientes entre sí, pero se representan generando combinaciones redundantes.

Por ejemplo, si un curso tiene asociados varios profesores (Gómez y López) y también distintos libros (Álgebra y Cálculo) de forma independiente, la tabla original podría combinar innecesariamente todas las opciones posibles. Para resolver este problema y evitar redundancias, en 4FN se propone dividir la información en dos tablas separadas:

- › *Curso_Profesor* (Curso, Profesor)
- › *Curso_Libro* (Curso, Libro)

- **Quinta forma normal (5FN)**

La quinta forma normal busca garantizar que una tabla pueda ser reconstruida sin pérdida de información a partir de todas sus proyecciones. Esto asegura que al descomponer una tabla en varias partes y luego unir las, no aparezcan registros “ficticios” que no existían en la tabla original.

Este principio resulta especialmente útil en casos donde existen relaciones complejas entre múltiples entidades, como profesores, cursos e inscripciones. La 5FN garantiza que, al combinar las diferentes tablas, solo se reconstruyan las combinaciones reales y válidas de los datos.



Multimedia

Como se mencionó anteriormente, tanto la cuarta como la quinta forma normal no se aplican con frecuencia en el diseño cotidiano de bases de datos. Sin embargo, para quienes deseen profundizar en estos conceptos, se recomienda complementar esta lectura con los siguientes videos sugeridos, donde se explican sus fundamentos y ejemplos de aplicación de manera más detallada:

Cuarta Forma Normal	Quinta Forma Normal
	

También disponible desde:

<https://tinyurl.com/mpppum5t>

<https://tinyurl.com/3nrwpvk8>



Actividad

En este momento los/las invitamos a realizar la **Actividad 3: “Normalización”**. Recuerden que ante cualquier duda también pueden consultar a su tutor/a.

- **Principios ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad)**

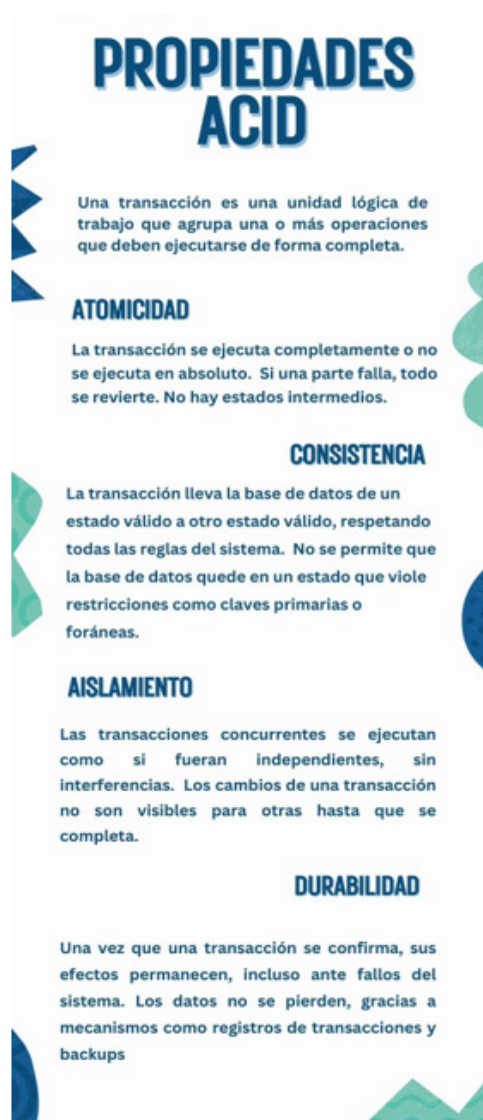
Los principios **ACID** garantizan que las transacciones se comporten de manera segura y predecible. Pero antes de avanzar con los principios, debemos ahondar en qué significa la palabra transacción en un contexto de bases de datos.

¿Qué es una transacción? Una transacción en el contexto de bases de datos es una *unidad lógica de trabajo* que agrupa una o más operaciones que deben ejecutarse de forma completa y coherente.

Por ejemplo, si un estudiante se inscribe en un curso, esa acción puede implicar:

- › Registrar la inscripción.
- › Actualizar el cupo disponible.
- › Registrar la fecha de inscripción.

Todas estas operaciones forman parte de una única transacción, lo que significa que si alguna de ellas falla, el sistema debe revertir todas las demás para evitar inconsistencias en la base de datos. Las transacciones resultan esenciales en sistemas donde la precisión, la integridad y la confiabilidad son prioritarias, como en entornos bancarios, plataformas académicas o sitios de comercio electrónico. Su correcta implementación permite asegurar que la información almacenada refleje siempre un estado válido y coherente, incluso ante errores o interrupciones.



Fuente: Elaboración propia.

MÓDULO 1 | GLOSARIO

Atomicidad: Propiedad que garantiza que una transacción se ejecuta completamente o no se ejecuta nada.

Atributo: Característica o propiedad de una entidad (ej.: nombre, edad, legajo).

Base de datos: Conjunto de datos organizados y almacenados sistemáticamente.

Cardinalidad: Número de filas en una tabla o el tipo de relación entre entidades (uno a uno, uno a muchos, muchos a muchos).

Clave alternativa/candidata: Atributo que podría ser clave primaria pero no fue seleccionado como tal.

Clave foránea: Atributo que referencia a la clave primaria de otra tabla para establecer relaciones.

Clave primaria: Atributo que identifica de forma única cada fila en una tabla. Debe ser única y no nula.

Consistencia: Propiedad que asegura que una transacción lleva la base de datos de un estado válido a otro.

Diagrama Entidad-Relación: Representación gráfica de entidades, atributos y relaciones en una base de datos.

Dominio: Conjunto de valores válidos que puede tomar un atributo (ej.: solo fechas, solo números).

Durabilidad: Propiedad que garantiza que los cambios de una transacción confirmada sean permanentes.

Entidad: Objeto o concepto del mundo real que se representa en la base de datos (ej.: Estudiante, Materia).

Forma normal Boyce-Codd (FNBC): Versión reforzada de 3FN donde todo determinante debe ser clave candidata.

Grado: Número de columnas (atributos) de una tabla.

Integridad referencial: Regla que asegura que los valores de una clave foránea existan en la tabla referenciada.

Modelo Entidad-Relación: Modelo conceptual para representar la estructura lógica de una base de datos.

Modelo relacional: Modelo de datos donde la información se organiza en tablas bidimensionales.

Normalización: Proceso de organizar los datos para reducir redundancias y mejorar la integridad.

Primera forma normal (1FN): Cada celda de una tabla contiene un solo valor (no listas ni conjuntos).

Relación: Conexión lógica entre entidades en el modelo E-R, o sinónimo de tabla en el modelo relacional.

Segunda forma normal (2FN): Elimina dependencias parciales (atributos que dependen solo de parte de la clave).

SQL: Lenguaje estándar para gestionar y consultar bases de datos relacionales.

Tabla: Estructura bidimensional compuesta por filas y columnas que almacena datos.

Tercera forma normal (3FN): Elimina dependencias transitivas (atributos no clave que dependen de otros no clave).

Transacción: Unidad lógica de trabajo que agrupa operaciones que deben ejecutarse como un todo.

Tupla: Fila o registro individual en una tabla.

MÓDULO 1 | ACTIVIDADES



ACTIVIDAD N° 1:

Conceptos básicos del modelo relacional

En esta actividad, aplicaremos y repasaremos los conceptos básicos del modelo relacional. Para ello dividimos la consigna en dos secciones.

Sección 1: Identificando elementos del modelo relacional.

› Ejercicio 1

Analice la siguiente tabla:

ISBN	Título	Autor	Año	Editorial	Precio
978-0141439518	Orgullo y Prejuicio	Jane Austen	1813	Penguin	12.50
978-8466338141	Don Quijote	Cervantes	1605	Anaya	15.00
978-9151339588	Orgullo y Prejuicio	Jane Austen	1813	Hachette	15.50
978-8420412146	Cien años de soledad	García Márquez	1967	Alfaguara	18.00

Preguntas:

1. ¿Cuál es el grado de esta tabla?
2. ¿Cuál es la cardinalidad de esta tabla?
3. Proponga un dominio para cada columna (ej: "cadenas de texto", "números enteros", "números decimales").
4. ¿Qué columna o conjunto de columnas podrían actuar como clave primaria? ¿Existe alguna otra clave candidata? Justifique tu respuesta.

› Ejercicio 2

Analice la siguiente tabla:

ID_Producto	Nombre_Producto	Categoría	Precio	Stock	Fecha_Lanzamiento
PROD-001	Laptop Gamer	Tecnología	1200.00	15	15/03/2023
PROD-002	Mesa de Oficina	Hogar	299.99	8	01/08/2022
PROD-003	Silla Ergonómica	Hogar	450.50	0	10/01/2024
PROD-004	Headset	Tecnología	49.99	20	09/04/2025

Preguntas:

1. ¿Cuál es el grado y la cardinalidad?
2. ¿Qué dominio asignarías a la columna "Precio"?
3. ¿Por qué la columna "Stock" no sería buena clave primaria?
4. Identifique que columna o conjunto de columnas podrían actuar como clave primaria.

› Ejercicio 3

Analice la siguiente tabla:

Nro_ Historia	Paciente	Fecha_ Nacimiento	Tipo_ Sangre	Última_Visita	Fecha_ Lanzamiento
HCL-1258	María González	1985-07-22	O+	2024-01-15	15/03/2023
HCL-3390	Carlos Rojas	1992-11-03	AB-	2024-02-28	01/08/2022
HCL-893	Mario Martinez	1989-04-21	B+	2024-09-10	10/01/2024
HCL-4412	Ana Silva	1978-04-18	A+	2024-06-17	09/04/2025

Preguntas:

1. ¿Qué representa cada tupla en este contexto?
2. ¿Cuál es el grado y la cardinalidad?
3. ¿Cuál sería una buena clave primaria? ¿Por qué?
4. Proponga un dominio para cada columna.

› Sección 2: Identificación de claves en tablas existentes

› Ejercicio 1

En una base de datos de biblioteca existen estas dos tablas:

- › Tabla Socios: (id_socio, dni, nombre, apellido, teléfono)
- › Tabla Préstamos: (id_prestamo, fecha_prestamo, fecha_devolucion, id_socio, isbn_libro)

Preguntas:

1. Identifique qué columna o conjunto de columnas podrían actuar como clave primaria en la tabla Socios.
2. ¿Qué columna en la tabla Socios calificaría como clave candidata? Explique qué condición cumple.

3. Identifique qué columna o conjunto de columnas podrían actuar como clave primaria en la tabla Préstamos.
4. Identifique la clave foránea en Préstamos y explique:
 - a) A qué tabla y columna referencia.
 - a) Por qué cumple con las características de una FK.
 - a) Qué representa esta relación.
5. ¿Qué error evitaría la base de datos si intenta registrar un préstamo con un id_socio que no existe en la tabla Socios?

› *Ejercicio 2*

En base a las siguientes tablas, conteste las preguntas.

› *Tabla clientes*

<i>id_cliente</i>	<i>nombre</i>	<i>email</i>
1	Juan	juan@mail.com
2	María	maria@mail.com

› *Tabla pedidos*

<i>id_pedido</i>	<i>fecha</i>	<i>id_cliente</i>
101	2025-09-01	1
102	2025-09-02	3

Preguntas:

1. ¿Cuál es la clave primaria de cada tabla?
2. ¿Qué atributo es la clave foránea?
3. ¿Qué problema hay con el valor “3” en la columna id_cliente?

› *Ejercicio 3*

En base a las siguientes tablas, conteste las preguntas

› *Tabla clientes*

<i>numero_cliente</i>	<i>nombre</i>	<i>edad</i>
1001	Pedro	25
1002	Lucía	30

› **Tabla turnos**

fecha	hora	numero_cliente
2025-09-01	19:00	1001
2025-09-01	19:30	1002
2025-09-01	20:00	NULL

Preguntas:

1. ¿Qué representa la clave foránea numero_cliente en la tabla Turnos?
2. ¿Por qué es válido que el turno del 2025-09-01 a las 20:00hs tenga NULL en numero_cliente?
3. ¿Qué pasaría si quiero asignar "1005" al numero_cliente de la tabla Turnos que actualmente está en NULL?

ACTIVIDAD 1 | **Clave de corrección** **CC 1**

Sección 1: Identificando elementos del modelo relacional

› **Ejercicio 1**

1. Grado: 6 columnas
2. Cardinalidad: 4 filas
3. Dominios:
 - a) ISBN: cadena de texto
 - b) Título: cadena de texto
 - c) Autor: cadena de texto
 - d) Año: número entero
 - e) Editorial: cadena de texto
 - f) Precio: número decimal
4. Clave primaria: La columna ISBN, porque identifica de manera única cada libro. Clave candidata: El conjunto {Título, Autor, Año, Editorial} podría ser clave candidata.

› **Ejercicio 2**

1. Grado: 6 columnas, Cardinalidad: 4 filas
2. Dominio "Precio": número decimal positivo
3. Stock no es buena PK: Porque no es único (puede repetirse) y puede cambiar con el tiempo
4. Clave Primaria: ID_Producto, ya que identifica de manera única cada producto

› **Ejercicio 3**

1. Qué representa cada tupla: El registro médico de un paciente en el sistema
2. Grado: 5 columnas, Cardinalidad: 4 filas
3. Clave Primaria: Nro_Historia, porque es el identificador único de cada paciente
4. Dominios:
 - a) Nro_Historia: cadena de texto
 - b) Paciente: cadena de texto
 - c) Fecha_Nacimiento: fecha
 - d) Tipo_Sangre: cadena de texto (grupo sanguíneo)
 - e) Última_Visita: fecha

Sección 2: Identificación de claves en tablas existentes

› **Ejercicio 1**

1. PK en Socios: id_socio (identificador único y controlado)
2. Clave candidata: dni, porque también identifica de manera única a cada persona
3. PK en Préstamos: id_prestamo
4. Clave foránea:
 - a) Referencia: id_socio en Préstamos → id_socio en Socios
 - b) Características: referencia a PK, mismo dominio
 - c) Representa: la relación "cada préstamo está asociado a un socio"
5. Error evitado: La integridad referencial impediría el registro porque no existe el socio referenciado

› **Ejercicio 2**

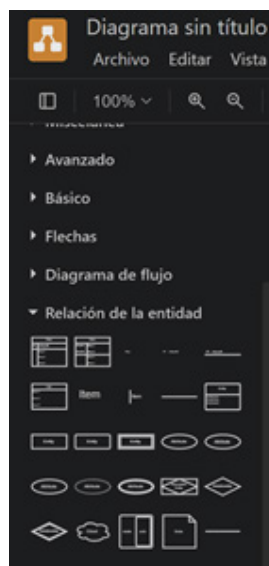
1. PK de cada tabla:
 - a) Clientes: id_cliente
 - b) Pedidos: id_pedido
2. Clave foránea: id_cliente en la tabla Pedidos
3. Problema con "3": Rompe la integridad referencial porque no existe un cliente con id_cliente = 3 en la tabla Clientes

› **Ejercicio 3**

1. Qué representa la FK: Relaciona cada turno con un cliente específico
2. NULL válido: Representa un turno disponible que no está asignado a ningún cliente
3. Asignar "1005": La base de datos rechazaría la operación por integridad referencial, ya que no existe un cliente con numero_cliente = 1005

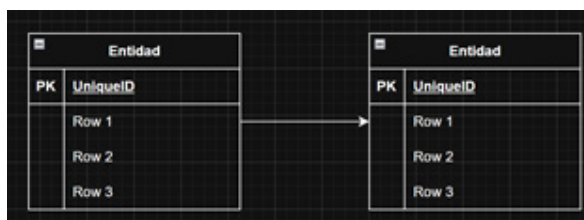
ACTIVIDAD N° 2: Modelo Entidad Relación

Para resolver los ejercicios de este módulo, usted puede hacerlo en papel. Si prefiere realizarlos en la computadora, le recomiendo utilizar la aplicación Diagrams. En la barra lateral izquierda encontrará las formas necesarias para realizar las actividades, dentro de la pestaña “Relación de la entidad”:



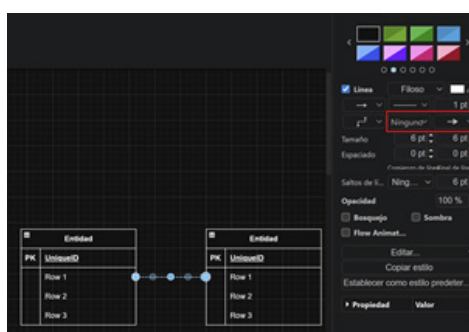
Fuente: elaboración propia.

Para las flechas especiales que utilizamos en el modelo lógico, o las flechas sin punta del modelo conceptual, pueden crear una flecha como la siguiente:



Fuente: elaboración propia.

Al hacer click en ella, podremos seleccionar qué tipo de punta queremos usar:



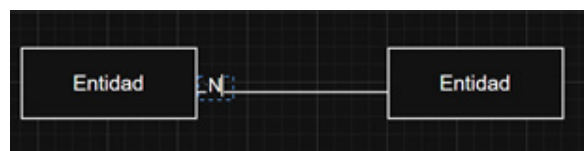
Fuente: elaboración propia.

En este caso, podemos ver cómo la punta de la izquierda muestra "ninguno", ya que no tiene ninguna punta, mientras que la de la derecha muestra una punta de flecha normal. Por último, para agregar las cardinalidades del modelo conceptual ("1:1", "1:N", "N:N"), podremos seleccionar la línea que creamos.



Fuente: elaboración propia.

Y, al hacer doble click cerca de uno de sus extremos, se abrirá la posibilidad de editar un campo de texto y agregar las cardinalidades en ese espacio:



Fuente: elaboración propia.

Con esta información, ya tiene las herramientas necesarias para realizar los siguientes ejercicios.

Construir el modelo de datos conceptual y el modelo lógico para los siguientes problemas.

› Ejercicio 1

Un videoclub necesita guardar información sobre sus películas y los socios que las alquilan.

Requerimientos:

- › De cada película se conoce: código, título, género y año.
- › Cada socio tiene un número de socio, nombre y teléfono.
- › Un alquiler relaciona un socio con una película e incluye: fecha de alquiler y fecha de devolución.

› Ejercicio 2

Una escuela necesita registrar la información de sus alumnos y las materias que cursan.

Requerimientos:

- › Cada alumno tiene un legajo, nombre y fecha de nacimiento.
- › Cada materia tiene un código, nombre y carga horaria.
- › Los alumnos pueden cursar varias materias, y en cada caso se guarda la nota final.

› Ejercicio 3

Una farmacia quiere almacenar información sobre sus productos y las ordenes realizadas por los clientes.

Requerimientos:

- › Cada producto se identifica por un código, y tiene nombre, precio y stock.
- › Cada cliente se identifica por un número de cliente, y tiene nombre y teléfono.
- › Una orden incluye el número de orden, la fecha y una lista de los productos ordenados (con su cantidad).

› Ejercicio 4

Una biblioteca necesita mantener información acerca de sus préstamos y reservas. Para ello se requiere construir un modelo de datos que refleje el funcionamiento de la misma.

La biblioteca solo administra libros. Por cada libro se tiene un número único, título, autores, temas, año de edición, editorial, idioma y ejemplares. Cada libro puede tener varios ejemplares idénticos, identificados por un número único, además de registrar su estado.

- › Cada autor tiene un número único y nombre.
- › Los temas se identifican por un número único y tienen una descripción.
- › La biblioteca tiene socios, los cuales se identifican con un número de socio y se describen por nombre, dirección y teléfono.
- › Los préstamos se registran con la siguiente información: socio, ejemplar, fecha de préstamo, fecha pactada de devolución y fecha de devolución.
- › Por su parte, las reservas contienen la siguiente información: socio, libro, fecha de reserva, fecha pactada de préstamo.

› Ejercicio 5

Una empresa con varias sucursales comercializa diferentes productos que se almacenan en dichas sucursales. La empresa desea mantener información acerca de sus sucursales, productos, clientes, vendedores y ventas a clientes, para lo cual se necesita diseñar una base de datos basada en el modelo relacional teniendo en cuenta la siguiente información:

- › Las sucursales están identificadas por un número único y se requieren los siguientes datos: nombre, dirección, localidad, teléfono.
- › Los productos tienen un código único y se describen a través de la siguiente información: nombre, precio de venta, precio de costo

- › Los clientes tienen un código único y se describen con la siguiente información: nombre, dirección, teléfono.
- › Los vendedores tienen un número único y se describen con la siguiente información: nombre y comisión.
- › Los datos de las ventas a registrar son los siguientes: cliente, vendedor, número de factura, sucursal, producto y cantidad vendida.
- › **Ejercicio 6**

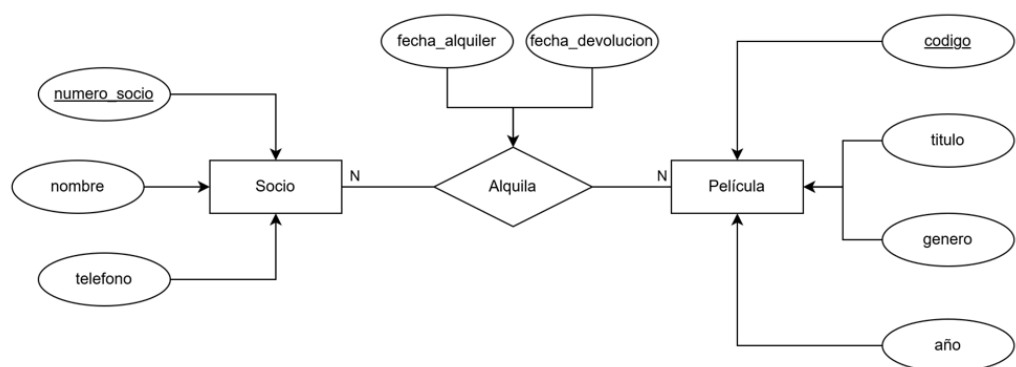
Un banco desea registrar información acerca de sus cuentas de cajas de ahorros. Se deberá desarrollar un modelo de datos relacional que permita mantener la información requerida

Los datos para registrar son los siguientes:

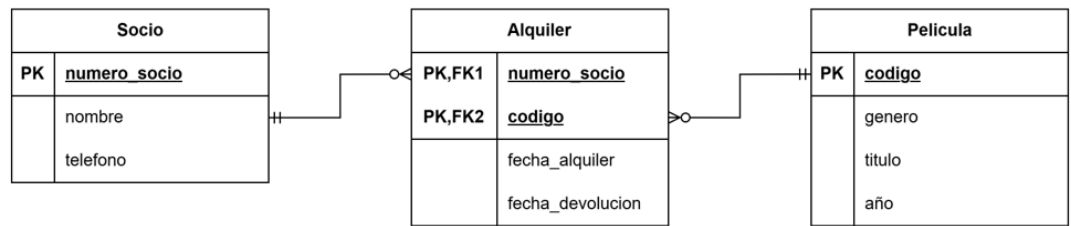
- › **Clientes:** identificados por un número y con la siguiente información: apellido y nombres, documento de identidad, domicilio
- › **Cuentas de cajas de ahorros:** identificadas por un número de cuenta, y con la siguiente información: tipo de cuenta, saldo en dólares, saldo en pesos, cliente, estado (*abierta, cerrada*)
- › **Movimientos de cuentas:** cuenta, fecha de contabilización, fecha de aplicación, código de movimiento (extracción, depósito, acreditación de intereses), número de movimiento, importe de movimiento, moneda
- › **Cheques depositados:** movimiento con el cual se depositó, nro. cheque, banco, plaza, fecha de cheque, importe de cheque, moneda

ACTIVIDAD 2 | Clave de corrección CC1

- › **Ejercicio 1**
- › **Modelo conceptual**

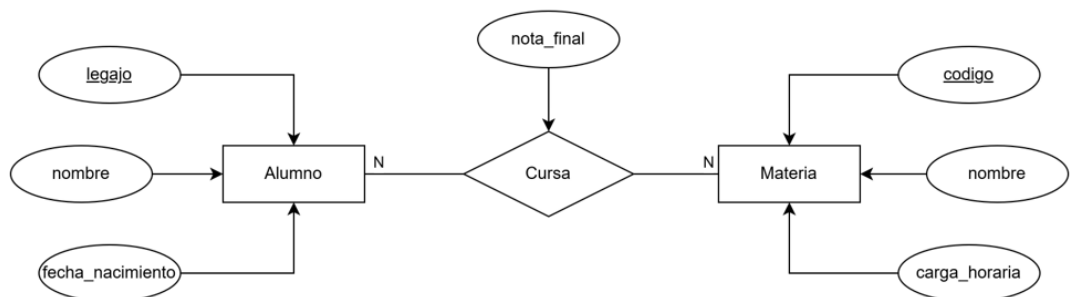


› *Modelo lógico*

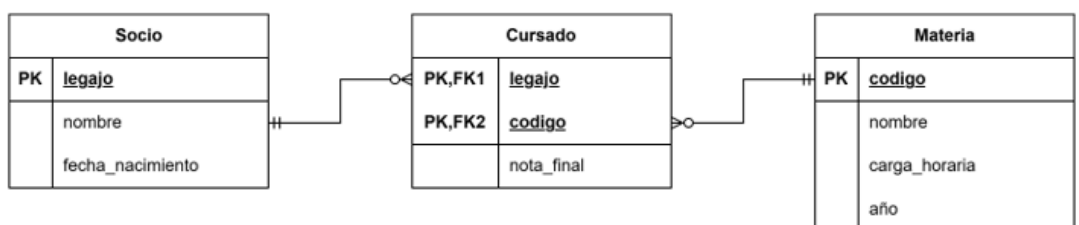


› *Ejercicio 2*

› *Modelo conceptual*

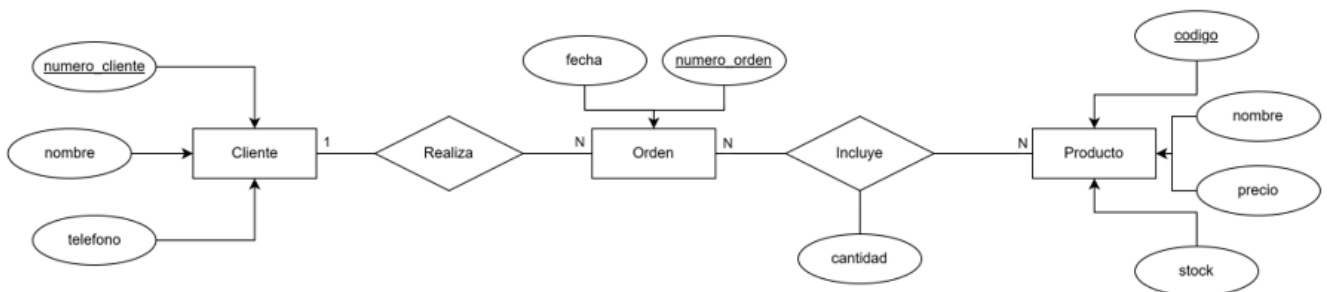


› *Modelo lógico*

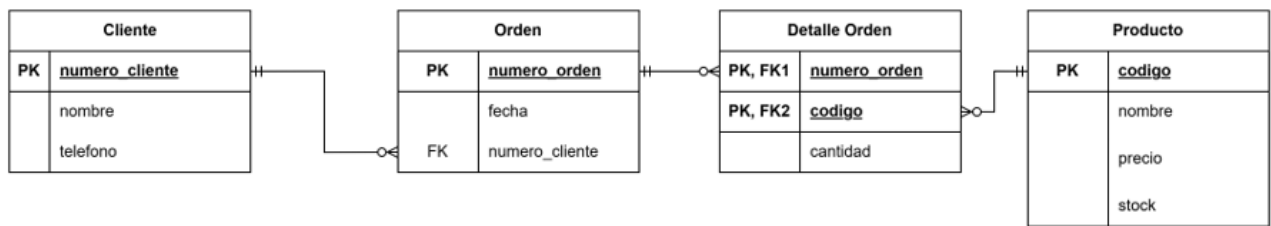


› *Ejercicio 3*

› *Modelo conceptual*

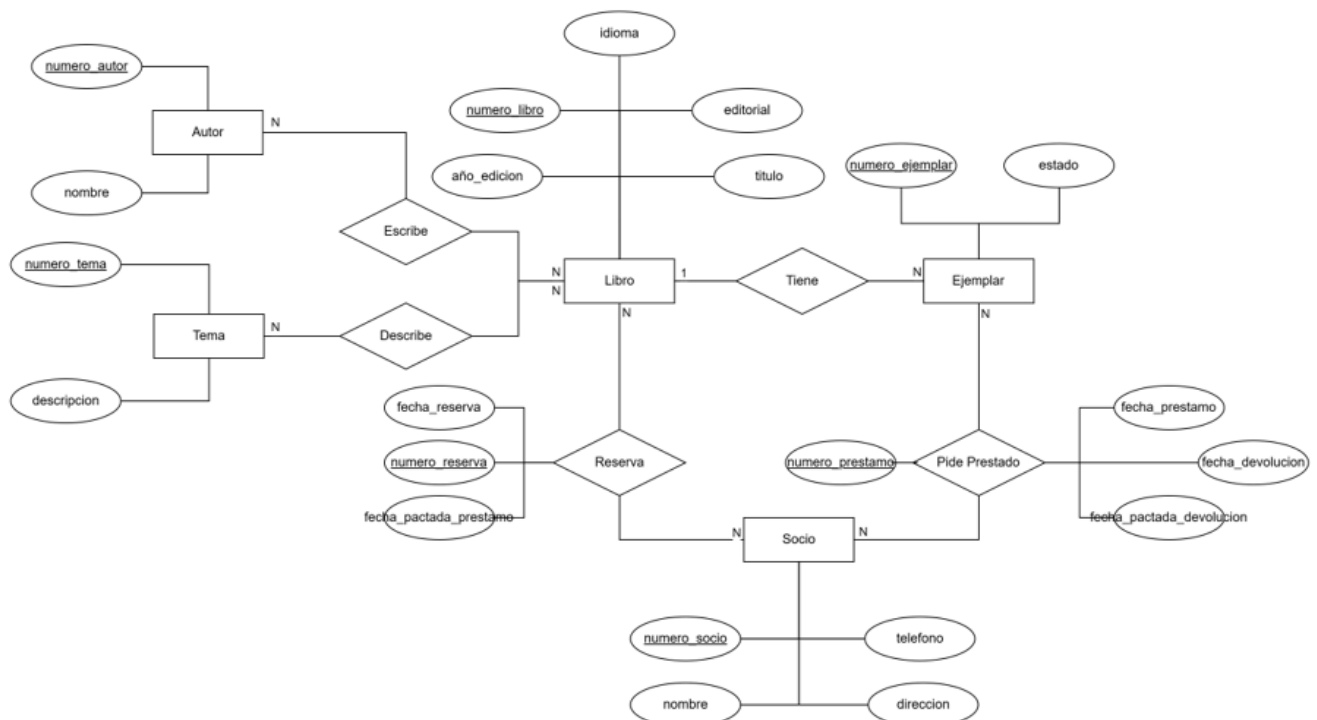


› *Modelo lógico*

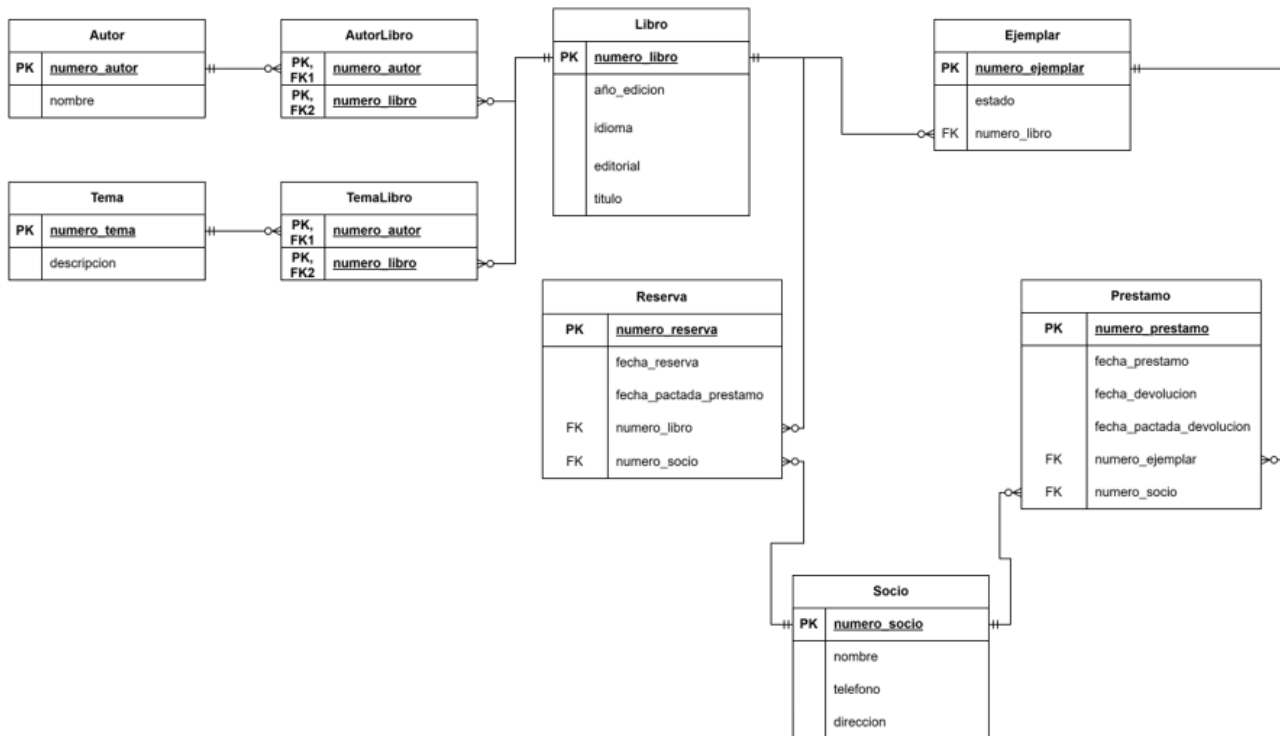


› *Ejercicio 4*

› *Modelo conceptual*

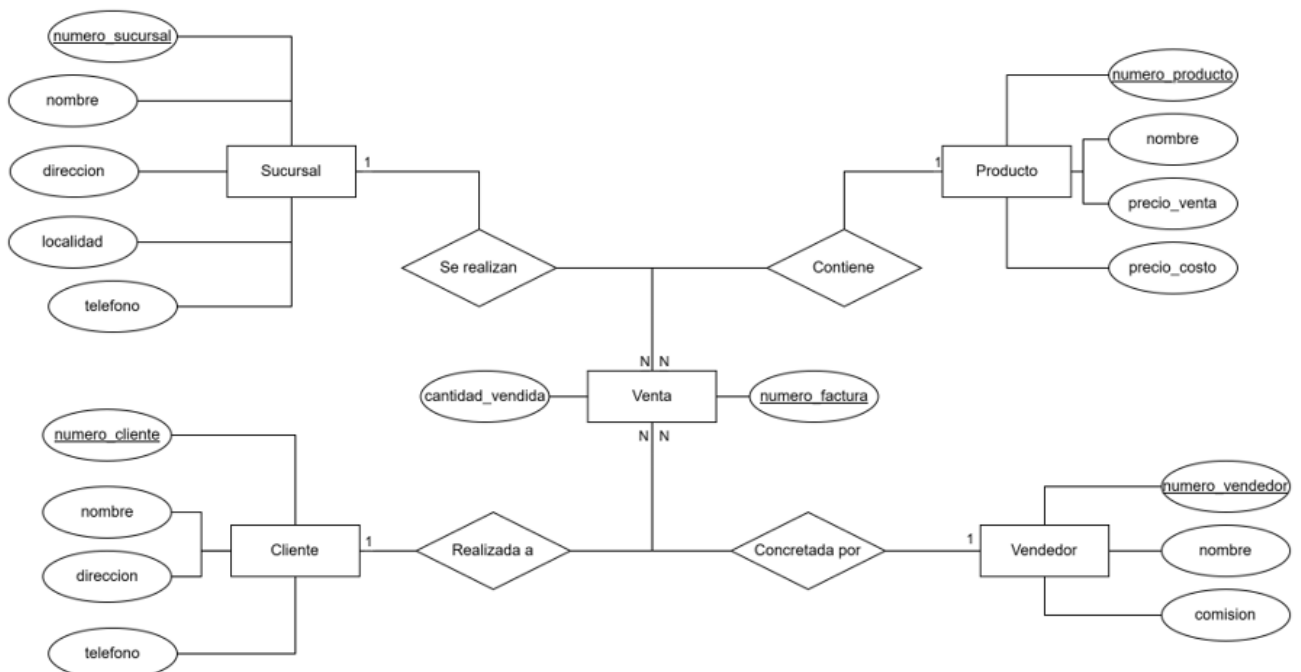


› *Modelo lógico*



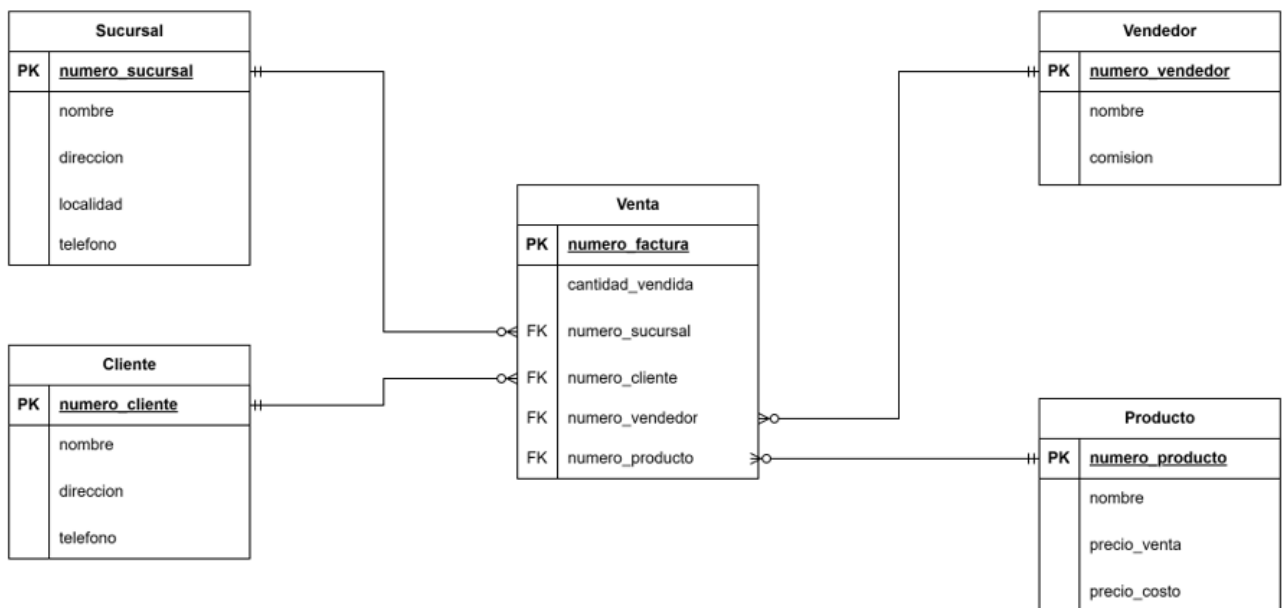
› *Ejercicio 5*

› *Modelo conceptual*



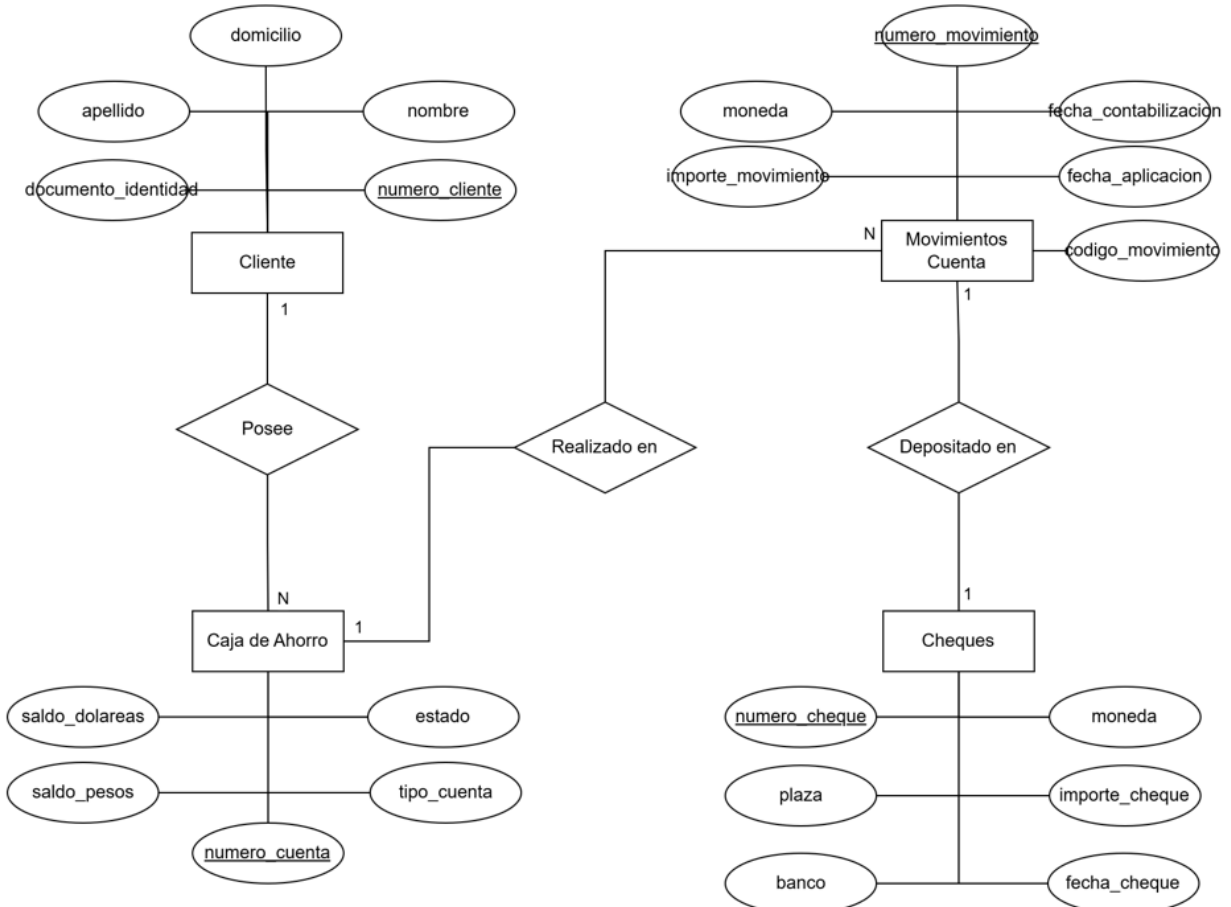
›

› *Modelo lógico*

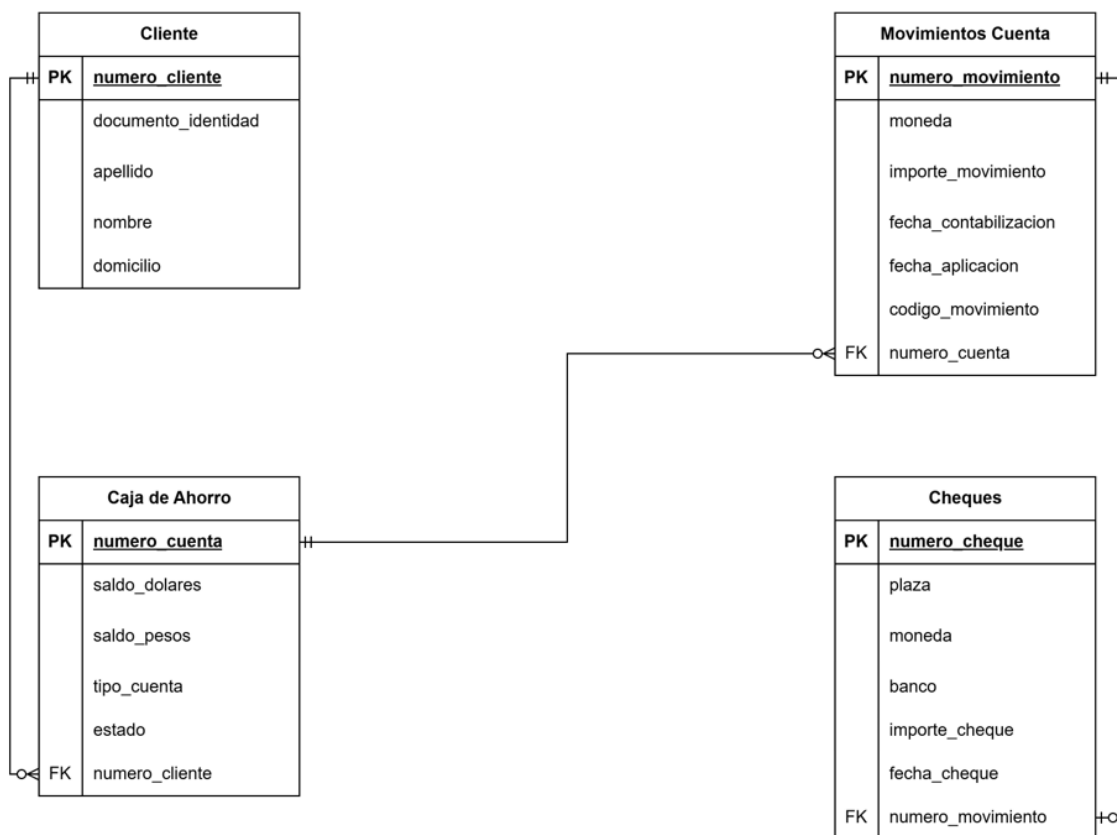


› *Ejercicio 6*

› *Modelo conceptual*



› Modelo lógico



ACTIVIDAD N° 3:
Normalización

1. Analice la siguiente tabla y señale qué columnas no cumplen la Primera Forma Normal (1FN). Explique por qué.

Empleado	Correo(s)	Proyecto(s)
Carla	carla@empresa.com, c.rojas@empresa.com	App Móvil, Sistema Web
Pedro	pedro@empresa.com	Sistema Web
Laura	laura@empresa.com, l.martinez@empresa.com	Base de Datos, App Móvil

2. Use la tabla del ejercicio anterior y reescríbala de forma que cumpla con la 1FN.
3. Observe la siguiente tabla que ya está en 1FN:

Cliente	Producto	Precio	Fecha
Sofía	Laptop	800	10/04
Sofía	Mouse	20	11/04
Marcos	Laptop	800	10/04

La clave candidata es (Cliente, Producto). Indique si hay alguna dependencia parcial. Si existe, separe las tablas para que cumplan con la Segunda Forma Normal (2FN).

4. Considere la siguiente tabla en 2FN:

Producto	Código Proveedor	Nombre Proveedor
Laptop	P01	TechCorp
Mouse	P02	ElectroMax
Teclado	P02	ElectroMax

- ¿Qué dependencia transitiva se puede identificar?
- Reorganice las tablas para alcanzar la Tercera Forma Normal (3FN).

5. Partiendo de la siguiente tabla:

Paciente	Médico	Código Médico	Especialidad	Fecha
Ana	Dr. Ruiz	MO1	Cardiología	05/05
Pedro	Dr. Gómez	MO2	Pediatría	06/05
Ana	Dr. Ruiz	MO1	Cardiología	07/05

Sabemos que:

- Cada médico tiene un código único y una especialidad.
- Un paciente puede atenderse con distintos médicos.

Realice el proceso completo de normalización hasta la Forma Normal 3, mostrando las tablas paso a paso.

ACTIVIDAD 3 | Clave de corrección CC 1

1. Las columnas Correo(s) y Proyecto(s) contienen múltiples valores en la misma celda (listas separadas por comas). Eso no es atómico y rompe la 1FN.

2.

Empleado	Correo	Proyecto
Carla	carla@empresa.com	App Móvil Sistema Web
Carla	carla@empresa.com	Sistema Web
Carla	c.rojas@empresa.com	App Móvil
Carla	c.rojas@empresa.com	Sistema Web
Pedro	pedro@empresa.com	Sistema Web
Laura	laura@empresa.com	Base de Datos
Laura	laura@empresa.com	App Móvil
Laura	l.martinez@empresa.com	Base de Datos
Laura	l.martinez@empresa.com	App Móvil

3. Análisis de dependencias parciales

Identificamos atributos que dependen solo de parte de la clave:

- › Precio depende de Producto (el precio es del producto, no del cliente).
- › Fecha depende de la compra específica (Cliente, Producto) → bien asociado a la clave completa.
- › Entonces hay dependencias parciales: Producto → Precio

- **Transformación a 2FN**

Se separan las tablas para eliminar dependencias parciales:

- › Ventas (o Compras) — mantiene la clave (Cliente, Producto, Fecha)
- › Ventas(Cliente, Producto, Fecha)

Cliente	Producto	Fecha
Sofía	Laptop	10/04
Sofía	Mouse	11/04
Marcos	Laptop	10/04

- › **Productos** — atributos que dependen solo de Producto

- › Productos(Producto, Precio)

<i>Producto</i>	<i>Precio</i>
Laptop	800
Mouse	20

4.

- a) Observamos: Producto → Código Proveedor (producto está asociado a proveedor) y Código Proveedor → Nombre Proveedor.

Por ende, Producto → Nombre Proveedor por transitividad. Aquí Nombre Proveedor depende de Código Proveedor, que a su vez depende del Producto. Si Nombre Proveedor no depende directamente de la clave Producto sino de otro atributo (Código Proveedor), tenemos dependencia transitiva.

- b) Se separa la información del proveedor en su propia tabla:

- › Productos (mantiene relación con el código del proveedor):
- › Productos(Producto, CodigoProveedor)

<i>Producto</i>	<i>CodigoProveedor</i>
Laptop	P01
Mouse	P02
Teclado	P02

- › Proveedores (clave: CodigoProveedor):
- › Proveedores(CodigoProveedor, NombreProveedor)

<i>CodigoProveedor</i>	<i>NombreProveedor</i>
P01	TechCorp
P02	ElectroMax

Así NombreProveedor depende de la clave primaria CodigoProveedor de su tabla → cumple 3FN.

5.

- › **Paso 1: Chequeo 1FN**

Revisando la tabla, cada celda tiene un solo valor atómico (no hay listas en una celda), por tanto, sí cumple 1FN.

› **Paso 2: Chequeo 2FN**

Clave candidata: (Paciente, Fecha, CódigoMedico)

Dependencias funcionales observadas

- › CódigoMedico → Médico
- › CódigoMedico → Especialidad
- › (Paciente, Fecha, CódigoMedico) determina el registro completo de la cita.

¿Hay dependencias parciales?

Sí. Médico y Especialidad dependen solo de CódigoMedico (una parte de la clave compuesta), por lo tanto, hay dependencia parcial y la tabla NO cumple 2FN.

Transformación para 2FN

Separamos los atributos que dependen solo de CódigoMedico en una tabla Medicos. La tabla principal de citas queda con la información específica de la cita.

Tablas resultantes (2FN):

1. Citas

Paciente	Fecha	CodigoMedico
Ana	05/05	Mo1
Pedro	06/05	Mo2
Ana	07/05	Mo1

2. Medicos

Código Médico	Médico	Especialidad
Mo1	Dr. Ruiz	Cardiología
Mo2	Dr. Gómez	Pediatría

Con esto eliminamos las dependencias parciales: Médico y Especialidad ya no están en la tabla cuya clave es (Paciente, Fecha, CodigoMedico).

› **Paso 3: Chequeo 3FN**

Revisamos las tablas:

- Medicos(CodigoMedico, Médico, Especialidad)

- › Médico y Especialidad dependen directamente de la clave primaria `CodigoMedico`.
- › No hay atributo no clave que determine a otro atributo no clave dentro de esta tabla. → cumple 3FN.
- Citas(Paciente, Fecha, `CodigoMedico`)
 - › Todos los atributos no clave (ninguno aparte de la clave compuesta) dependen de la clave completa; `CodigoMedico` es parte de la PK en la tabla Citas. No hay atributo no clave que dependa de otro atributo no clave dentro de Citas. → cumple 3FN.

No existen dependencias transitivas en las tablas separadas, por lo tanto, el diseño ya está en 3FN.

MÓDULO 2

Microobjetivos

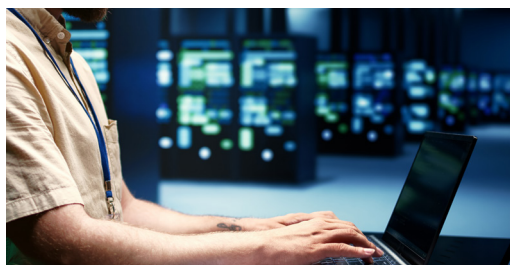
- › Comprender el proceso de instalación y configuración de un servidor de bases de datos MySQL y MySQL Workbench, para saber crear un entorno práctico de desarrollo que permita interactuar con bases de datos relacionales de forma directa.
- › Aplicar los fundamentos del Lenguaje de Definición de Datos (DDL) para crear, modificar y eliminar tablas, definiendo constraints, claves primarias y foráneas, con el fin de expresar el diseño conceptual de una base de datos en términos de su implementación real.
- › Asimilar el uso del Lenguaje de Manipulación de Datos (DML) para insertar, actualizar, eliminar y consultar registros mediante sentencias SELECT con filtros, ordenamientos y agrupaciones, con el objetivo de gestionar y extraer información específica de las bases de datos.
- › Fortalecer el dominio del uso de JOINS (INNER, LEFT, RIGHT) para relacionar tablas en una base de datos, con el fin de resolver consultas complejas que requieren integrar información distribuida en el esquema relacional.



VIDEO DOCENTE

Contenidos

- SQL



Fuente: <https://www.freepik.es/>

En el módulo anterior, hemos aprendido lo más importante sobre las bases de datos desde el lado de la teoría, en este nuevo módulo, nos meteremos de lleno con la lengua franca de las bases de datos relacionales: SQL. A lo largo de los contenidos, aprenderemos cómo descargar un servidor de bases de datos y aplicaremos los conceptos vistos con anterioridad en un entorno de bases de datos real. Este módulo está pensado para que puedan ir escribiendo sus primeras líneas de código SQL a medida que avanzan con el desarrollo conceptual de la asignatura.

¡Comencemos!

- *Instalación MySQL y MySQLWorkbench*

Para empezar, instalaremos nuestro servidor de base de datos MySQL y nuestra herramienta de administración de bases de datos, MySQLWorkbench.



Deberá ir al siguiente enlace para descargar MySQL: Descargar MySQL. Es importante descargar la versión que diga LTS. Y también descargue MySQLWorkbench, este es el enlace: Descargar MySQLWorkbench



Le sugerimos los siguientes recursos para acompañar el paso a paso de la instalación según el sistema operativo que cada uno/a disponga:

Tutorial Windows	Tutorial MacOS	Tutorial Linux
		

También disponible desde:

<https://tinyurl.com/mveytdhd>
<https://tinyurl.com/3zukur8c>
<https://tinyurl.com/yr25emxz>

- *Creando la base de datos de prueba*

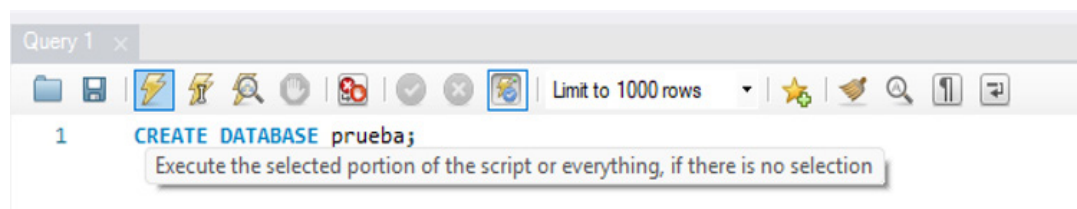
Una vez que hayamos instalado nuestro servidor de base de datos MySQL y la herramienta de administración MySQL Workbench, podremos comenzar a trabajar.

MySQL es un servidor de bases de datos, lo que implica que dentro del mismo servidor podemos tener múltiples bases de datos. Nuestro primer comando SQL será, entonces, crear la primera de ellas.

Lo haremos con el siguiente comando:

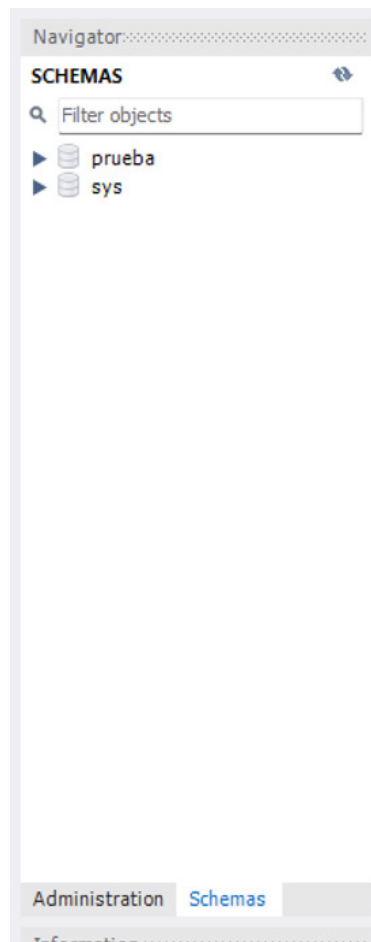
```
CREATE DATABASE prueba;
```

Para ejecutarlo, copiaremos este código y lo pegaremos en nuestro editor MySQL Workbench



Fuente: Elaboración propia.

Al hacer clic en el ícono del rayo, ejecutaremos el comando. Si vamos a la sección de schemas de la barra de la izquierda, podremos ver nuestra tabla “prueba” creada.



Fuente: Elaboración propia.

Por último, debemos decirle a MySQL que queremos que el código que creemos (*nuestras queries*) se ejecuten en la base de datos “prueba”. Para ello, debemos correr el siguiente comando:

```
USE prueba;
```

› DDL y DML

En el mundo de SQL, existen 2 grupos de consultas principales.



Énfasis

Por un lado, se encuentra el DDL (*Data Definition Language*) que significa Lenguaje de Definición de Datos. Estas son las consultas que nos ayudarán a crear, borrar o modificar una tabla. Por otro lado, está el DML (*Data Modelling Language*), o (*Lenguaje de Modelado de Datos*) que se utilizan para seleccionar, insertar, eliminar y modificar los registros o filas de una tabla.

› DDL: Primeros pasos

Utilizaremos un ejemplo que ya hemos visto anteriormente para poder explicar el código SQL.



Fuente: Elaboración propia.

```

CREATE TABLE Profesores (
    nro_legajo INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
CREATE TABLE Materia (
    codigo_materia VARCHAR(4) PRIMARY KEY,
    nombre VARCHAR(200) NOT NULL,
    horas_catedras INT NOT NULL
);
CREATE TABLE Dictado (
    nro_legajo INT,
    codigo_materia VARCHAR(4),
    año INT NOT NULL,
    cuatrimestre INT NOT NULL,
    CONSTRAINT PK_Dictado PRIMARY KEY (nro_legajo, codigo_materia),
    CONSTRAINT FK_nro_legajo_Dictado_Profesores FOREIGN KEY (nro_legajo) REFERENCES Profesores (nro_legajo),
    CONSTRAINT FK_nro_legajo_Dictado_Materia FOREIGN KEY (codigo_materia) REFERENCES Materia (codigo_materia)
);
  
```

Aquí podemos ver cómo se traslada el diagrama físico que habíamos creado con anterioridad a código SQL.

Podemos generalizarlo de la siguiente manera:

```

CREATE TABLE [IF NOT EXISTS] nombre_tabla (
    nombre_columna tipo_de_dato constraints,
    nombre_columna tipo_de_dato constraints,
    ...
)
  
```

Create Table indica a nuestra base de datos que estamos por crear una tabla, *IF NOT EXISTS* es un opcional que podemos agregar para que, si la tabla ya existe, la base de datos no tire un error. Como lo indica, `nombre_tabla` será el nombre que le pondremos a nuestra tabla. Dentro de ella, tenemos las columnas, las cuales están compuestas por un nombre de columna, un tipo de dato (Ej. *INT*, *VARCHAR*, *DATE*) y constraints o restricciones si corresponden (Ej. *PRIMARY KEY*, *NOT NULL*, *DEFAULT*).

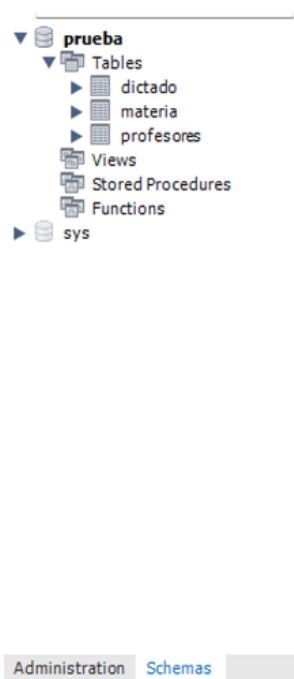
En el caso particular de la tabla “Dictado”, las constraints aparecen definidas al final de la declaración de la tabla. Esta es otra forma de definir constraints. Cuando la tabla posee una clave primaria compuesta, es decir, la clave primaria está compuesta por más de una columna, ya no podemos definirlas de la forma que vimos en las tablas Profesores o Materia. En este caso, debemos definirlas como se encuentran en la tabla de Dictado.

`[CONSTRAINT nombre_constraint] PRIMARY KEY (columnas_pk)`

Lo mismo sucede con las claves foráneas, debemos explicitarlas en la parte inferior como se encuentran en la tabla de Dictado.

`[CONSTRAINT nombre_constraint] FOREIGN KEY (columna) REFERENCES
tabla_referenciada (columna_tabla_referenciada)`

La estructura es la siguiente: luego de *FOREIGN KEY* ponemos entre paréntesis la columna de nuestra tabla que es una clave foránea y, luego de la palabra *REFERENCES*, va la tabla a la que estamos referenciando. Después de eso, debemos poner entre paréntesis la columna de la tabla referenciada. Con todo esto terminado, si le damos a refresh y expandimos las tablas de la base de datos prueba, veremos lo siguiente:



Fuente: Elaboración propia.

Para eliminar una tabla, se utiliza el siguiente comando:

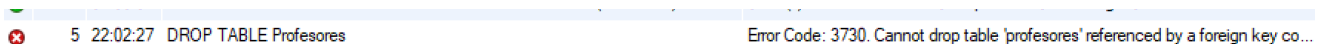
```
DROP TABLE [IF EXISTS] nombre_tabla;
```

Debido a la relación de la tabla Dictado con la tabla Profesores, si quisiéramos eliminar la tabla Profesores, no podríamos hacerlo, porque la tabla Dictado tiene una clave foránea.

Si ejecutamos el siguiente comando:

```
DROP TABLE Profesores;
```

Podremos ver este error:



5 22:02:27 DROP TABLE Profesores Error Code: 3730. Cannot drop table 'profesores' referenced by a foreign key co...

Fuente: Elaboración propia.

Para modificar una tabla, podemos hacerlo con el siguiente comando:

```
ALTER TABLE nombre_tabla ADD COLUMN nueva_columna tipo_dato [FIRST | AFTER columna_existente];
```

Supongamos que debemos agregar una nueva columna a la tabla Profesores, queremos tener registrada la edad de los profesores, haremos lo siguiente:

```
ALTER TABLE Profesores ADD COLUMN edad INT CHECK (edad > 18);
```

Aparte de agregar una nueva columna, hemos agregado un nuevo constraint “CHECK” que nos permite realizar validaciones de dato antes de agregar un nuevo registro a la tabla.

Finalmente, hablaremos sobre los tipos de datos más comunes en MySQL:

› Tipos de datos numéricos

Para empezar, exploraremos los tipos de datos diseñados para almacenar números. Utilizaremos el tipo **INT** o **INTEGER** para almacenar números enteros, siendo el más común para valores como identificadores únicos (IDs) o cantidades. En cambio, para almacenar valores decimales donde la precisión es crítica, como un precio o un saldo, utilizaremos el tipo **DECIMAL(M, N)**, donde M es el número total de dígitos y N es la cantidad de decimales. Por ejemplo, **DECIMAL(10, 2)** para guardar 99999999.99

Finalmente, cuando queremos representar simplemente un valor de Verdadero o Falso, usaremos **BOOLEAN**, que en realidad es un alias de **TINYINT(1)**, donde 1 es verdadero y 0 es falso.

› Tipos de datos de texto

Ahora, veremos los tipos de datos para almacenar cadenas de caracteres. Emplearemos **VARCHAR(n)** para almacenar texto de longitud variable, como un nombre, un email o

una dirección, donde 'n' define la longitud máxima de caracteres. Es el tipo de texto más utilizado por su eficiencia. Cuando necesitemos almacenar texto de una longitud fija y siempre predecible, como un código de país (ej: 'ES', 'AR') o un código postal usaremos **CHAR(n)**. Para textos más extensos, como una descripción de producto o un comentario largo, recurriremos al tipo **TEXT**, capaz de manejar grandes volúmenes de información.

› Tipos de datos de fecha y hora

Por último, tenemos los tipos de datos para manejar información temporal. Utilizaremos **DATE** cuando solo necesitemos almacenar una fecha, sin hora, en el formato 'AAAA-MM-DD'. Por ejemplo, una fecha de nacimiento. Usaremos **DATETIME** para almacenar un momento específico, combinando fecha y hora en el formato 'AAAA-MM-DD HH:MM:SS'. Este es ideal para registrar cuándo se realizó un pedido.

Para cerrar, un tipo particular de dato es **TIMESTAMP**, que también almacena un instante en el tiempo, pero con la particularidad de que se guarda en la zona horaria UTC y se convierte automáticamente a la zona horaria de la conexión. Es muy útil para registrar la hora de creación o modificación de un registro de forma automática.



En este momento, lo/a invitamos a realizar la Actividad 1 del módulo.

- **DML: Data Modelling Language**

Con nuestro ejemplo anterior ya creado, ahora aprenderemos a añadir datos a nuestro modelo. Para ello, utilizaremos la siguiente consulta SQL:

```
INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES (1, 'Jonathan', 27),
(2, 'Jeronimo', 30);
```

```
INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES (1, 'Jonathan', 27),
(2, 'Jeronimo', 30);
```

```
INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES (1, 'Jonathan', 27),
(2, 'Jeronimo', 30);
```

Al ejecutar este comando, habremos creado dos filas o registros en la tabla Profesores. En general, la estructura de la consulta SQL se ve así:

```
INSERT INTO nombre_tabla(columna1, columna2, ...)
VALUES (valor1, valor2, ...);
```

O si queremos agregar varios registros en una sola consulta podemos hacer lo siguiente:

```
INSERT INTO nombre_tabla(columna1, columna2, ...)
VALUES (valor1, valor2, ...),
VALUES (valor1, valor2, ...),
...
VALUES (valor1, valor2, ...);
```

La cantidad de valores tiene que ser igual a la cantidad de columnas, y el tipo de dato del valor tiene que corresponder con el tipo de dato de la columna, es decir, hacer lo siguiente sería incorrecto y nos daría un error:

```
INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES ('uno', 'Jonathan', 27);
```

El error se debe a que estamos intentando insertar una cadena de caracteres en una columna que fue declarada como INT, o sea que solo acepta enteros.

Para actualizar uno de los registros que acabamos de crear, podemos usar la siguiente consulta:

```
UPDATE Profesores
SET
nombre = 'Francisco'
WHERE
nro_legajo = 1;
```

En general, la consulta de UPDATE se ve así:

```
UPDATE nombre_tabla
SET
columna1 = valor1,
columna2 = valor2,
...
[WHERE condición];
```

De esta manera, la sentencia UPDATE se usa para modificar datos que ya están guardados en una tabla de una base de datos. Primero escribimos UPDATE, seguido del nombre de la tabla que queremos modificar. Luego usamos SET para indicar qué columnas vamos a cambiar y qué nuevos valores vamos a asignarle. Podemos cambiar varias columnas al mismo tiempo, separándolas por comas.

La parte más importante es el **WHERE**, que es opcional pero muy necesario. El **WHERE** nos permite decidir exactamente qué filas queremos modificar. Por ejemplo, podemos actualizar solo el registro con nro_legajo igual a 1. Es importante volver a aclarar que, si bien el **WHERE** es opcional, si no lo utilizamos la sentencia de **UPDATE** se ejecutará sobre todas las filas, lo que podría causar la pérdida de datos importantes.

Luego, tenemos la sentencia para eliminar datos, si queremos eliminar el Profesor 2, podríamos hacerlo de la siguiente manera:

```
DELETE FROM Profesores WHERE nro_legajo = 2;
```

En general, la estructura del **DELETE** se ve así:

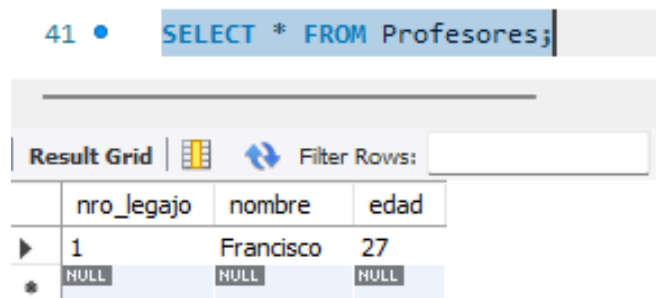
```
DELETE FROM nombre_tabla  
[WHERE condition];
```

Aquí, primero debemos agregar **DELETE FROM**, seguidamente explicitamos el nombre de la tabla de la cual vamos a eliminar datos. Al igual que el **UPDATE**, la cláusula **WHERE** es opcional, si no la agregamos, se eliminarán todos los datos que existen en la tabla. Esto es un punto muy importante, al menos que explícitamente se quiera eliminar todos los datos de una tabla, se recomienda siempre utilizar el **WHERE** para definir las filas o registros que se van a eliminar.

Por último, tenemos el **SELECT**, esta consulta es la que nos permite visualizar los datos y realizar el trabajo de extraer información útil de estos datos. Una sentencia SQL se ve así:

```
SELECT * FROM Profesores;
```

Si estuvo ejecutando los comandos en su servidor SQL hasta ahora, debería ver el siguiente resultado:



	nro_legajo	nombre	edad
▶	1	Francisco	27
*	NULL	NULL	NULL

Fuente: Elaboración propia.

La estructura general de la consulta **SELECT** es la siguiente:

```
SELECT lista_seleccionada  
FROM nombre_tabla;
```

El asterisco “*” lo que hace es seleccionar y mostrar todas las columnas que existen en la tabla, para el caso de Profesores, nos muestra las columnas nro_legajo, nombre y edad.

Si quisiéramos solo obtener el nombre y la edad, podríamos ejecutar la siguiente consulta:

43 • `SELECT nombre, edad FROM Profesores;`

Result Grid			Filter Rows:	Export:
	nombre	edad		
▶	Francisco	27		

Fuente: Elaboración propia.

No es que hayamos eliminado por completo la columna `nro_legajo` de nuestra tabla, sino que simplemente estamos explicitándole a la base de datos que solo necesitamos la columna `nombre` y la columna `edad`.

Ahora, agregaremos más datos en la tabla para poder realizar operaciones más complejas.

```
INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES (3, 'Francisco', 22),
       (4, 'Ana', 24),
       (5, 'Maria', 35),
       (6, 'Edgar', 53);
```

Una vez agregados estos datos, haremos un **SELECT *** nuevamente:

```
SELECT * FROM Profesores;
```

Veremos los nuevos registros agregados:

	nro_legajo	nombre	edad
▶	1	Francisco	27
	3	Francisco	22
	4	Ana	24
	5	Maria	35
	6	Edgar	53
⊞	NULL	NULL	NULL

Fuente: Elaboración propia.

Ejecutemos la siguiente sentencia:

```
SELECT DISTINCT nombre FROM Profesores;
```

Veremos un resultado como este:

	nombre
►	Francisco
	Ana
	Maria
	Edgar

Fuente: Elaboración propia.

La sentencia **DISTINCT** lo que hace es filtrar los valores duplicados de las columnas que especificamos, sólo trayendo los posibles valores sin repetir. Si agregáramos la columna edad a la sentencia veríamos lo siguiente:

```
SELECT DISTINCT nombre, edad FROM Profesores;
```

	nombre	edad
►	Francisco	27
	Francisco	22
	Ana	24
	Maria	35
	Edgar	53

Fuente: Elaboración propia.

Traemos la misma cantidad de filas que con un **SELECT** normal, esto sucede porque el **DISTINCT** ahora está filtrando por el conjunto de valores único, y si bien el nombre Francisco aparece 2 veces, la columna edad tiene un valor diferente en los 2 casos.

A continuación, aprenderemos a filtrar los resultados que obtenemos de un **SELECT** con la sentencia **WHERE**. Resulta necesario señalar que la estructura del **WHERE** en la consulta **SELECT** es idéntica a la que vimos anteriormente para el **UPDATE** y el **DELETE**:

```
SELECT columnas FROM nombre_tabla
WHERE condición;
SELECT nombre, edad FROM Profesores
WHERE nombre = 'Francisco';
```

Lo que está haciendo esta consulta SQL es filtrar la tabla Profesores y sólo traernos las filas en donde el nombre sea Francisco. Así, lo que obtendremos será lo siguiente:

	nombre	edad
►	Francisco	27
	Francisco	22

Fuente: Elaboración propia.

Es muy importante hacerse preguntas para aprender cómo traducir esas preguntas formuladas en lenguaje natural a código SQL. Por ejemplo, si nos preguntamos quiénes son

los profesores que tienen 25 años o más, podemos responder a esa pregunta ejecutando el siguiente comando:

```
SELECT nombre, edad FROM Profesores  
WHERE edad >= 25;
```

Al ejecutarlo, veremos el siguiente resultado:

	nombre	edad
▶	Francisco	27
	Maria	35
	Edgar	53

Fuente: Elaboración propia.

De esta forma empezamos a ver cómo los datos pueden sernos de utilidad para responder preguntas que las personas, o incluso nuestros clientes, nos hagan. Ahora pensemos en la siguiente pregunta: “¿Quién es el profesor más joven?”. ¿Cómo podríamos responderla con una consulta SQL? Para ello, podemos utilizar la siguiente consulta:

```
SELECT nombre, edad FROM Profesores  
ORDER BY edad ASC;
```

Si ejecutamos esta sentencia obtendríamos este resultado:

	nombre	edad
▶	Francisco	22
	Ana	24
	Francisco	27
	Maria	35
	Edgar	53

Fuente: Elaboración propia.

Lo que hace **ORDER BY** es, como su nombre indica, ordenar las filas en base a la columna o columnas que definamos. La estructura del **ORDER BY** en general es así:

```
SELECT columnas FROM nombre_tabla  
ORDER BY  
columna1 [ASC|DESC],  
columna2 [ASC|DESC],  
...;
```

Si después de la columna, ponemos **ASC**, los valores se ordenarán de forma ascendente, si ponemos **DESC**, se ordenarán de forma descendente. Por su parte, el **ORDER BY** puede combinarse con **WHERE**. Si nos hiciéramos la siguiente pregunta: “¿Cuántos años tiene el profesor Francisco de mayor edad?” podemos resolverla utilizando el siguiente código SQL:

```
SELECT nombre, edad FROM Profesores
WHERE nombre = "Francisco"
ORDER BY edad DESC;
```

De esta consulta SQL obtendríamos el siguiente resultado:

	nombre	edad
►	Francisco	27
	Francisco	22

Fuente: Elaboración propia.

Aquí podemos ver que el Profesor Francisco de mayor edad tiene 27 años.



Existen muchos tipos más de comparaciones que se pueden realizar con el **WHERE**, para aprender más de ellas, les recomiendo la siguiente página: [MySQL Basics - Sección 3: Filtering Data](#)

- **DML: Group By**

Ya tenemos las bases de cómo realizar consultas SQL, sabemos que utilizando el **SELECT** podemos obtener los datos de las bases de datos, y el **WHERE** nos permite filtrar estos datos y contestar preguntas que puedan surgir sobre ellos. Avancemos ahora hacia cómo realizar un análisis más profundo agrupando datos.

Podemos empezar por una sentencia GROUP BY básica:

```
SELECT nombre FROM Profesores
GROUP BY nombre;
```

Obtenemos lo siguiente:

	nombre
►	Francisco
	Ana
	Maria
	Edgar

Fuente: Elaboración propia.

A simple vista, el resultado es muy parecido a lo que obtuvimos cuando utilizamos el **DISTINCT**, pero la potencia del **GROUP BY** está en el hecho de que, por detrás, lo que la base de datos está haciendo es agrupar las filas en base a los valores de la columna (o columnas) que explicitamos. Sobre estos grupos, nosotros podemos aplicar agregaciones.

Por ejemplo:

```
SELECT nombre, COUNT(*) as cantidad_profesores
FROM Profesores
GROUP BY nombre;
```

Iremos punto por punto para explicar qué aporta esta consulta. `COUNT(*)` es una de las funciones de agregación mencionadas: cuenta la cantidad de filas que existen en la agrupación. La expresión “as cantidad_profesores” es un alias que permite renombrar la columna resultante para que la salida sea más clara. Por ejemplo, si quisiéramos que la columna nombre se muestre como nombre_profesores, podríamos ejecutar:

```
SELECT nombre as nombre_profesores, COUNT(*) as
cantidad_profesores FROM Profesores
GROUP BY nombre;
```

El resultado de las dos consultas, aunque cambie el nombre de la columna, es el mismo:

nombre_profesores	cantidad_profesores
Francisco	2
Ana	1
Maria	1
Edgar	1

Fuente: Elaboración propia.

Si quisiéramos sólo mostrar los nombres que aparecen más de una vez en la tabla podríamos ejecutar la siguiente consulta:

```
SELECT nombre, COUNT(*) as cantidad_profesores
FROM Profesores
GROUP BY nombre
HAVING COUNT(*) > 1;
```

La sentencia **HAVING** funciona como el **WHERE** para las agrupaciones, nos permite filtrar los datos ya agrupados aplicando condicionales con agregaciones sobre ellos.

› Introducción a los JOIN en SQL

Pasaremos, entonces, a los **JOIN** y comenzaremos a trabajar con las relaciones entre la tabla Profesores y las tablas Dictado y Materia.



Énfasis

Los **JOINS** son una de las operaciones más importantes en SQL, ya que nos permiten combinar datos de múltiples tablas basándonos en relaciones entre ellas. Hay diferentes tipos de **JOIN**. Nosotros exploraremos los siguientes: el **LEFT JOIN**, **INNER JOIN**, **RIGHT JOIN**.

Antes de avanzar con ejemplos, agregaremos más datos a nuestras tablas para contar con un escenario más completo:

```

INSERT INTO Profesores(nro_legajo, nombre, edad)
VALUES (7, 'Carlos', 28), (8, 'Laura', 31), (9, 'Pedro', 29);
INSERT INTO Materia(codigo_materia, nombre, horas_catedras)
VALUES ('MAT1', 'Matemáticas I', 4), ('FIS1', 'Física I', 4), ('QUI1', 'Química I', 3), ('PRO1',
'Programación I', 6), ('BD1', 'Bases de Datos', 5), ('BD2', 'Bases de Datos II', 6);
INSERT INTO Dictado(nro_legajo, codigo_materia, anio, cuatrimestre)
VALUES (1, 'MAT1', 2023, 1), (1, 'FIS1', 2023, 1), (3, 'QUI1', 2023, 1), (3, 'PRO1', 2023, 1),
(4, 'BD1', 2023, 1), (5, 'MAT1', 2023, 2), (6, 'FIS1', 2023, 2);

```

El **INNER JOIN** devuelve solo los registros que tienen coincidencias en ambas tablas. Si lo visualizáramos con un diagrama de Venn, el **INNER JOIN** sería algo así:



Fuente: Wikipedia.

El **círculo A** serían los registros de la tabla A, el **círculo B** serían los registros de la tabla B. La intersección de los 2 círculos es lo que nos devuelve un **INNER JOIN**.

Volviendo a nuestro ejemplo anterior, podríamos preguntar: ¿qué profesores dictan qué materias?

```

SELECT p.nombre AS profesor, d.anio, d.cuatrimestre, d.codigo_materia
FROM Profesores p
INNER JOIN Dictado d ON p.nro_legajo = d.nro_legajo;

```

profesor	anio	cuatrimestre	codigo_materia
Francisco	2023	1	FIS1
Francisco	2023	1	MAT1
Francisco	2023	1	PRO1
Francisco	2023	1	QUI1
Ana	2023	1	BD1
Maria	2023	2	MAT1
Edgar	2023	2	FIS1

Fuente: Elaboración propia.

Aquí podemos ver que recibimos información de las 2 tablas: Profesores y Dictado, pero no recibimos todos los datos de la tabla Profesores, ni tampoco recibimos todos los datos de la tabla Dictado, sólo recibimos los datos en los cuales se encontraron coincidencias de nro_legajo en las 2 tablas.

La estructura del JOIN es la siguiente:

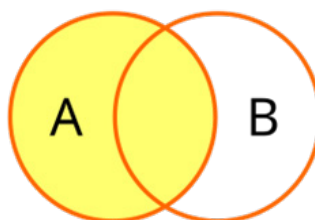
```
SELECT columnas  
FROM tabla1  
INNER JOIN tabla2 ON condición;
```

Comienza con un **SELECT ... FROM**, como ya vimos anteriormente, pero la diferencia aparece cuando incorporamos el **INNER JOIN**. Después de escribir **INNER JOIN**, debemos indicar la tabla con la que queremos unir la información. A continuación, luego de la palabra clave **ON**, establecemos la condición del **JOIN**. En nuestro ejemplo anterior, la condición era que el nro_legajo fuera igual en las 2 tablas “p.nro_legajo = d.nro_legajo”, esto le indica a MySQL que antes de unir una fila de la primera tabla con una fila de la segunda, tiene que cumplirse la condición de que nro_legajo de Profesores y nro_legajo de Dictado sean iguales.

- **LEFT JOIN**

Para entender el **LEFT JOIN**, primero debemos detenernos en qué significa el **LEFT** dentro de esta expresión. Cuando realizamos un **JOIN**, estamos combinando dos tablas. En el caso que vimos con el **INNER JOIN**, la tabla desde la que partíamos en el **SELECT** era Profesores, mientras que la tabla que estábamos uniendo era Dictado. Entonces, siguiendo ese ejemplo, la tabla que se encuentra a la izquierda, es decir, la primera mencionada, es Profesores, y esa será nuestra **LEFT** table. La tabla que se ubica a la derecha, la que se une a la primera, es Dictado, y será la **RIGHT** table.

Si visualizamos el **LEFT JOIN** con un diagrama de Venn, veríamos algo así:



Fuente: Wikipedia.

En este caso, lo que un **LEFT JOIN** nos devuelve es la intersección de los círculos A y B, y también el resto de A. Podemos verlo con el siguiente ejemplo:

```
SELECT p.nombre AS profesor, d.anio, d.cuatrimestre,  
d.codigo_materia  
FROM Profesores p  
LEFT JOIN Dictado d ON p.nro_legajo = d.nro_legajo;
```

Esta query (consulta) nos devolverá el siguiente resultado:

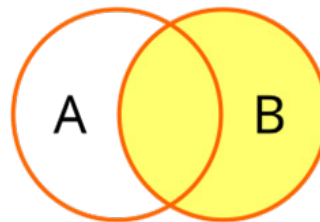
profesor	anio	cuatrimestre	codigo_materia
Francisco	2023	1	FIS1
Francisco	2023	1	MAT1
Francisco	2023	1	PRO1
Francisco	2023	1	QUI1
Ana	2023	1	BD1
Maria	2023	2	MAT1
Edgar	2023	2	FIS1
Carlos	NULL	NULL	NULL
Laura	NULL	NULL	NULL
Pedro	NULL	NULL	NULL

Fuente: Elaboración propia.

Si lo comparamos al **INNER JOIN**, veremos que nos trajo las mismas filas que en esa query, pero ahora agregó 3 filas más, que contienen a los profesores que no tienen asignada ninguna materia (Carlos, Laura y Pedro). Estas filas provienen de la tabla Profesores y no poseen coincidencias en Dictado, pero igualmente aparecen en el resultado porque estamos utilizando un **LEFT JOIN**.

- **RIGHT JOIN**

El **RIGHT JOIN** es la contraparte del **LEFT JOIN** que vimos anteriormente. Si observamos el diagrama de Venn, podemos notar con mayor claridad esta diferencia:



Fuente: Wikipedia.

Lo que un **RIGHT JOIN** nos devuelve es la intersección de los círculos A y B, y también el resto de B, es decir, la tabla de la derecha.

Si ejecutamos la siguiente query:

```
SELECT d.nro_legajo, d.anio, d.cuatrimestre, m.codigo_materia, m.nombre,
m.horas_catedras
FROM Dictado d
RIGHT JOIN Materia m ON d.codigo_materia = m.codigo_materia;
```

Obtendremos el siguiente resultado:

nro_legajo	anio	cuatrimestre	codigo_materia	nombre	horas_catedras
4	2023	1	BD1	Bases de Datos	5
NULL	NULL	NULL	BD2	Bases de Datos II	6
1	2023	1	FIS1	Física I	4
6	2023	2	FIS1	Física I	4
1	2023	1	MAT1	Matemáticas I	4
5	2023	2	MAT1	Matemáticas I	4
3	2023	1	PRO1	Programación I	6
3	2023	1	QUI1	Química I	3

Fuente: Elaboración propia.

Esta vez realizamos una unión entre Dictado y Materia. Podemos observar que, aunque la materia *Bases de Datos II* no tiene relación con ningún registro en Dictado en este momento (por eso los datos provenientes de Dictado aparecen en **NULL**), igualmente se muestra en los resultados. Esto ocurre porque el **RIGHT JOIN** trae todas las filas de la tabla Materia, incluso cuando alguna de ellas no tiene correspondencia en la tabla Dictado.

- **JOIN Múltiple**

Para cerrar este módulo, veremos cómo se pueden realizar operaciones con múltiples **JOINS**. Si quisiéramos unir las 3 tablas Profesores, Dictado y Materia para obtener la información completa sobre el dictado de materias, podríamos hacer lo siguiente:

```
SELECT p.nombre AS profesor, p.edad, m.nombre AS materia,
m.horas_catedras, d.anio, d.cuatrimestre
FROM Profesores p
INNER JOIN Dictado d ON p.nro_legajo = d.nro_legajo
INNER JOIN Materia m ON d.codigo_materia = m.codigo_materia;
```

Podemos ver que en esta consulta utilizamos dos **INNER JOIN**. Primero, unimos la tabla Profesores con la tabla Dictado. A partir de ese **JOIN**, obtendríamos un resultado similar al que vimos anteriormente en la sección de **INNER JOIN**:

profesor	anio	cuatrimestre	codigo_materia
Francisco	2023	1	FIS1
Francisco	2023	1	MAT1
Francisco	2023	1	PRO1
Francisco	2023	1	QUI1
Ana	2023	1	BD1
Maria	2023	2	MAT1
Edgar	2023	2	FIS1

Fuente: Elaboración propia.

Esto es lo que internamente queda almacenado en la memoria de MySQL. Luego, aplicamos el segundo **INNER JOIN** con la tabla Materia, utilizando como condición que “d.codigo_materia = m.codigo_materia”. Como resultado, obtenemos lo siguiente:

profesor	edad	materia	horas_catedras	anio	cuatrimestre
Ana	24	Bases de Datos	5	2023	1
Francisco	27	Física I	4	2023	1
Edgar	53	Física I	4	2023	2
Francisco	27	Matemáticas I	4	2023	1
Maria	35	Matemáticas I	4	2023	2
Francisco	22	Programación I	6	2023	1
Francisco	22	Química I	3	2023	1

Fuente: Elaboración propia.

De esta forma, logramos unir a los Profesores con las Materias y obtener toda la información relevante, aun cuando no exista una relación directa entre ambas tablas. La conexión se realiza a través de la tabla Dictado, tal como lo habíamos representado en el diagrama del módulo anterior.



Recursos

Para profundizar más sobre alguno de los temas presentados, lo/a invito a visitar la siguiente página web: MySQL Tutorial



Actividad

Dando por finalizado el desarrollo del contenido del módulo, lo/a invitamos a trabajar con la Actividad 2 del módulo. Tenga presente que cuenta con el espacio tutorial para resolver cualquier consulta.



Evaluación

Luego de haber estudiado los módulos 1 y 2, y de haber resuelto las actividades, Ud. está en condiciones de resolver la primera parte de la evaluación integradora.

MÓDULO 2 | GLOSARIO

Alter table: comando para modificar una tabla agregando, quitando o cambiando columnas.

Create table: comando para crear una tabla indicando columnas, tipos de datos y restricciones.

Constraint: restricción aplicada a una columna o tabla (PRIMARY KEY, NOT NULL, CHECK, etc.).

Create database: comando para crear una nueva base de datos.

DDL: conjunto de comandos para definir estructuras (crear, modificar o eliminar tablas).

Decimal: tipo de dato para números con precisión decimal fija.

Delete: comando para eliminar registros de una tabla.

Distinct: elimina valores duplicados en los resultados de un SELECT.

DML: conjunto de comandos para manipular datos (insertar, actualizar, eliminar, consultar).

Drop table: comando para eliminar una tabla existente.

Foreign key: columna que referencia una clave primaria de otra tabla para relacionarlas.

Group by: agrupa filas para realizar cálculos (sumas, promedios, conteos).

Having: filtra resultados después de una agrupación.

Inner join: devuelve sólo los registros que coinciden en ambas tablas.

Insert into: comando para insertar nuevos registros en una tabla.

Int: tipo de dato para números enteros.

Join: operación que combina datos de dos o más tablas.

Left join: devuelve todos los registros de la tabla izquierda y coincidencias de la derecha.

MySQL: servidor de bases de datos relacional donde podemos crear múltiples bases de datos.

MySQL Workbench: herramienta gráfica para administrar servidores MySQL y ejecutar consultas SQL.

Order by: ordena los resultados de una consulta de forma ascendente (ASC) o descendente (DESC).

Primary key: columna o conjunto de columnas que identifica de forma única cada registro.

Right join: devuelve todos los registros de la tabla derecha y coincidencias de la izquierda.

Select: comando para consultar y visualizar datos almacenados.

SQL: lenguaje utilizado para crear, modificar y consultar bases de datos relacionales.

Timestamp: tipo de dato que guarda fecha y hora en UTC y se ajusta según la zona horaria.

Update: comando para modificar datos existentes.

Use: comando para indicar en qué base de datos queremos trabajar.

Varchar: tipo de texto de longitud variable.

Where: filtro que permite seleccionar filas que cumplan una condición.

MÓDULO 2 | ACTIVIDADES



ACTIVIDAD Nº 1: DDL

Para esta actividad, utilizaremos los ejercicios de la actividad 2 del módulo 1, y pasaremos el modelo lógico que hemos creado a un modelo de bases de datos en MySQL utilizando sentencias de DDL (CREATE TABLE).

- › *Ejercicio 1:* Construir el modelo de datos en MySQL del Ejercicio 1 de la Actividad 2 del Módulo 1.
- › *Ejercicio 2:* Construir el modelo de datos en MySQL del Ejercicio 2 de la Actividad 2 del Módulo 1.
- › *Ejercicio 3:* Construir el modelo de datos en MySQL del Ejercicio 3 de la Actividad 2 del Módulo 1.
- › *Ejercicio 4:* Construir el modelo de datos en MySQL del Ejercicio 4 de la Actividad 2 del Módulo 1.
- › *Ejercicio 5:* Construir el modelo de datos en MySQL del Ejercicio 5 de la Actividad 2 del Módulo 1.
- › *Ejercicio 6:* Construir el modelo de datos en MySQL del Ejercicio 6 de la Actividad 2 del Módulo 1.

ACTIVIDAD 1 | Clave de corrección **CC 1**

› Ejercicio 1

CREATE TABLE Socio (

numero_socio INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
telefono VARCHAR(20) NOT NULL

);

CREATE TABLE Pelicula (

codigo INT PRIMARY KEY,
genero VARCHAR(50) NOT NULL,
titulo VARCHAR(150) NOT NULL,
año YEAR NOT NULL

);

CREATE TABLE Alquiler (

numero_socio INT NOT NULL,
codigo INT NOT NULL,
fecha_alquiler DATE NOT NULL,
fecha_devolucion DATE,
PRIMARY KEY (numero_socio, codigo),

CONSTRAINT fk_alquiler_socio
FOREIGN KEY (numero_socio) REFERENCES Socio(numero_socio),
CONSTRAINT fk_alquiler_pelicula
FOREIGN KEY (codigo) REFERENCES Pelicula(codigo)

);

› Ejercicio 2

CREATE TABLE Alumno (

legajo INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
fecha_nacimiento DATE NOT NULL

);

CREATE TABLE Materia (

```

codigo INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
carga_horaria INT NOT NULL,
año YEAR NOT NULL

);

CREATE TABLE Cursado (

legajo INT NOT NULL,
codigo INT NOT NULL,
nota_final DECIMAL(4,2),
PRIMARY KEY (legajo, codigo),

CONSTRAINT fk_cursado_socio
FOREIGN KEY (legajo) REFERENCES Alumno(legajo),
CONSTRAINT fk_cursado_materia
FOREIGN KEY (codigo) REFERENCES Materia(codigo)

);

```

► Ejercicio 3

```

CREATE TABLE Cliente (

id_cliente INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
telefono VARCHAR(20) NOT NULL

);

CREATE TABLE Orden ( id_orden INT PRIMARY KEY,

fecha DATE NOT NULL,
id_cliente INT NOT NULL,

CONSTRAINT fk_orden_cliente
FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)

);

CREATE TABLE Producto (

id_producto INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
precio DECIMAL(10,2) NOT NULL

);

```

```

CREATE TABLE DetalleOrden (

id_orden INT NOT NULL,
id_producto INT NOT NULL,
cantidad INT NOT NULL,

PRIMARY KEY (id_orden, id_producto),
CONSTRAINT fk_detalle_orden
FOREIGN KEY (id_orden) REFERENCES Orden(id_orden),
CONSTRAINT fk_detalle_producto
FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)

);

```

› Ejercicio 4

```

CREATE TABLE Autor (

numero_autor INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL

);

```

```

CREATE TABLE Libro (

numero_libro INT PRIMARY KEY,
año_edicion YEAR NOT NULL,
idioma VARCHAR(50) NOT NULL,
editorial VARCHAR(100) NOT NULL,
titulo VARCHAR(200) NOT NULL

);

```

```

CREATE TABLE AutorLibro ( numero_autor INT NOT NULL, numero_libro INT NOT NULL,
PRIMARY KEY (numero_autor, numero_libro),
FOREIGN KEY (numero_autor) REFERENCES Autor(numero_autor),
FOREIGN KEY (numero_libro) REFERENCES Libro(numero_libro)

);

```

```

CREATE TABLE Tema (

numero_tema INT PRIMARY KEY,
descripcion VARCHAR(200) NOT NULL

);

```

```

CREATE TABLE TemaLibro (

```

```

numero_tema INT NOT NULL,
numero_libro INT NOT NULL,
PRIMARY KEY (numero_tema, numero_libro),

FOREIGN KEY (numero_tema) REFERENCES Tema(numero_tema),
FOREIGN KEY (numero_libro) REFERENCES Libro(numero_libro)

);

CREATE TABLE Ejemplar (

numero_ejemplar INT PRIMARY KEY,
estado VARCHAR(50) NOT NULL,
numero_libro INT NOT NULL,

FOREIGN KEY (numero_libro) REFERENCES Libro(numero_libro)

);

CREATE TABLE Socio (

numero_socio INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
telefono VARCHAR(20) NOT NULL,
direccion VARCHAR(200) NOT NULL

);

CREATE TABLE Reserva (

numero_reserva INT PRIMARY KEY,
fecha_reserva DATE NOT NULL,
fecha_pactada_prestamo DATE NOT NULL,
numero_libro INT NOT NULL,

numero_socio INT NOT NULL,
FOREIGN KEY (numero_libro) REFERENCES Libro(numero_libro),
FOREIGN KEY (numero_socio) REFERENCES Socio(numero_socio)

);

CREATE TABLE Prestamo (

numero_prestamo INT PRIMARY KEY,
fecha_prestamo DATE NOT NULL,
fecha_devolucion DATE,
fecha_pactada_devolucion DATE NOT NULL,
numero_ejemplar INT NOT NULL,

```

```
numero_socio INT NOT NULL,  
  
FOREIGN KEY (numero_ejemplar) REFERENCES Ejemplar(numero_ejemplar),  
FOREIGN KEY (numero_socio) REFERENCES Socio(numero_socio)  
  
);
```

» Ejercicio 5

```
CREATE TABLE Sucursal (  
  
id_sucursal INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
direccion VARCHAR(200) NOT NULL,  
localidad VARCHAR(200) NOT NULL,  
telefono VARCHAR(30) NOT NULL  
  
);  
  
CREATE TABLE Vendedor (  
  
id_vendedor INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
comision INT NOT NULL  
  
);  
  
CREATE TABLE Cliente (  
  
id_cliente INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
direccion VARCHAR(200) NOT NULL,  
telefono VARCHAR(30) NOT NULL  
  
);  
  
CREATE TABLE Producto (  
  
id_producto INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
precio_venta DECIMAL(10,2) NOT NULL,  
precio_costo DECIMAL(10,2) NOT NULL  
  
);  
  
CREATE TABLE Venta (  
  
id_venta INT PRIMARY KEY,  
id_cliente INT NOT NULL,
```

```

id_vendedor INT NOT NULL,
id_producto INT NOT NULL,
id_sucursal INT NOT NULL,
cantidad_vendida INT NOT NULL,
CONSTRAINT fk_venta_cliente

FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente),
CONSTRAINT fk_venta_vendedor
FOREIGN KEY (id_vendedor) REFERENCES Vendedor(id_vendedor),
CONSTRAINT fk_venta_producto
FOREIGN KEY (id_producto) REFERENCES Producto(id_producto),
CONSTRAINT fk_venta_sucursal
FOREIGN KEY (id_sucursal) REFERENCES Sucursal(id_sucursal)

);

```

› Ejercicio 6

```

CREATE TABLE Cliente (

id_cliente INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
apellido VARCHAR(100) NOT NULL,
documento_identidad VARCHAR(20) UNIQUE NOT NULL,
domicilio VARCHAR(150)

);

CREATE TABLE Caja_Ahorro (

id_caja INT PRIMARY KEY,
tipo_cuenta VARCHAR(30) NOT NULL,
estado VARCHAR(10) NOT NULL,
saldo_dolares DECIMAL(18, 2) DEFAULT 0,
saldo_pesos DECIMAL(18, 2) DEFAULT 0,
id_cliente INT NOT NULL,

FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)

);

CREATE TABLE Movimientos_Cuenta (

id_movimiento INT PRIMARY KEY,
moneda VARCHAR(20) NOT NULL,
fecha_aplicacion DATETIME NOT NULL,
fecha_contabilizacion DATETIME NOT NULL,
importe_movimiento DECIMAL(18, 2) NOT NULL,

```

```

codigo_movimiento VARCHAR(20),

id_caja INT NOT NULL,

FOREIGN KEY (id_caja) REFERENCES Caja_Ahorro(id_caja)

);

CREATE TABLE Cheques (

id_cheque INT PRIMARY KEY,
plaza VARCHAR(30) NOT NULL,
moneda VARCHAR(20) NOT NULL,
banco VARCHAR(20) NOT NULL,
fecha_cheque DATE NOT NULL,
importe_cheque DECIMAL(18, 2) NOT NULL,
id_movimiento INT NOT NULL,

FOREIGN KEY (id_movimiento) REFERENCES Movimientos_Cuenta(id_movimiento)

);

```



Actividad

ACTIVIDAD N° 2: DML y Queries



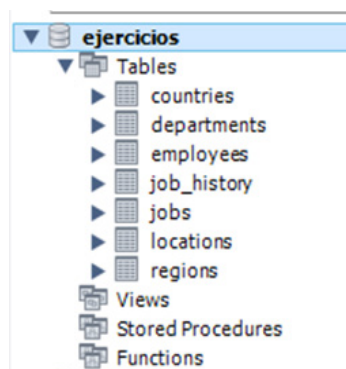
Recursos

Para esta actividad, utilizaremos una base de datos de HR. La puede descargar desde este link: [Script de creación](#)

Antes de comenzar los ejercicios de consultas y DML, lo que haremos será crear la base de datos que descargamos del link en nuestro motor de MySQL local de nuestra computadora.

En MySQLWorkbench, cree una base de datos llamada "ejercicios" y asegúrese de que los próximos comandos sean ejecutados contra esta. Ahora, copie y pegue el contenido del archivo descargado en una pestaña del editor de texto de MySQLWorkbench.

Proceda a ejecutar todo el contenido del editor. En la barra lateral, debería ver algo como lo siguiente:



Fuente: Elaboración propia.

Ahora, podemos pasar a los ejercicios de consultas. Cada ejercicio lo ejecutaremos en nuestro ambiente local.



Recursos

Para resolver los ejercicios, deberán ingresar a los enlaces previstos para cada uno.

› Ejercicio 1: Consultas SELECT básicas

Realizar los ejercicios del siguiente enlace: [Ejercicios](#)

› Ejercicio 2: Restringiendo y ordenando los datos

Realizar los ejercicios del siguiente enlace: [Ejercicios](#)

› Ejercicio 3: Funciones de agregación y Group By

Realizar los ejercicios del siguiente enlace: [Ejercicios](#)

› Ejercicio 4: Consultas JOIN

Realizar los ejercicios del siguiente enlace: [Ejercicios](#)



Recomendaciones

Dentro de cada ejercicio, podremos ver lo siguiente:

1. Write a query to display the names (first_name, last_name) using alias name "First Name", "Last Name"

Sample table: employees

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE
100	Steven	King	SKING	515.123.4567	1987-06-17
101	Neena	Kochhar	NKOCHHAR	515.123.4568	1987-06-18
102	Lex	De Haan	LDEHAAN	515.123.4569	1987-06-19
103	Alexander	Hunold	AHUNOLD	590.423.4567	1987-06-20

[Click me to see the solution](#)

Al hacer click en "Click me to see the solution", podremos ver la solución del ejercicio planteado.

MÓDULO 3

Microobjetivos

- › Conocer las operaciones fundamentales en bases de datos documentales con MongoDB, manejando la estructura de documentos, colecciones y consultas BSON a través de MongoDB Compass, para saber gestionar eficientemente datos semi-estructurados.
- › Aplicar el lenguaje Cypher para consultas en bases de datos de grafos con Neo4j, diseñando patrones de búsqueda que incluyan nodos, relaciones, propiedades y filtros complejos, con el objetivo de analizar y explotar las conexiones entre los datos.
- › Analizar el concepto de bases de datos multi-modelo mediante Azure CosmosDB, comprendiendo su capacidad para emular diferentes APIs (SQL, MongoDB, Gremlin) en una sola infraestructura, con el fin de determinar su flexibilidad en escenarios de datos diversos.



VIDEO DOCENTE

Contenidos

- NO-SQL



Fuente: Elaboración propia.

En este módulo, daremos un vistazo general a las bases de datos no relacionales, comúnmente llamadas NoSQL.

Algo importante a destacar es que lo único que tienen en común las bases de datos NoSQL entre sí es que no son relacionales, el resto es completamente diferente. Tenemos BD Documentales, es decir, que se organizan en Documentos, como MongoDB, y tenemos también BD de Grafos, como Neo4j. Por último, haremos una breve introducción teórica a las bases de datos multi-modelo, ilustrando el concepto con el caso de CosmosDB de Azure.

- **MongoDB**

MongoDB es una base de datos NoSQL, clasificada como una base de datos de tipo documental. Para comprender MongoDB es esencial familiarizarse con su terminología y cómo se relaciona con los conceptos tradicionales de bases de datos relacionales. La estructura fundamental comienza con la base de datos, que mantiene el mismo concepto que en SQL, sirviendo como contenedor principal. Dentro de cada base de datos, en lugar de tablas, MongoDB organiza los datos en colecciones, que actúan como agrupaciones lógicas de documentos.

Este documento equivale a una fila en SQL, pero con la diferencia crucial de que almacena datos en formato BSON (una extensión binaria del formato JSON). Cada documento contiene campos que serían análogos a las columnas en una base de datos relacional, pero con la flexibilidad de que diferentes documentos en la misma colección pueden tener distintos conjuntos de campos. Cada documento se identifica únicamente por un `_id`, que funciona como clave primaria y generalmente se implementa como un "ObjectId" automáticamente generado por MongoDB.

Un documento en una colección podría verse así:

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nombre": "Juan Pérez",
  "edad": 30,
  "email": "juan@ejemplo.com",
  "direccion": {
    "calle": "Av. Principal 123",
    "ciudad": "Buenos Aires",
    "pais": "Argentina"
  },
  "hobbies": ["fútbol", "lectura", "música"]
}
```

- **MongoDB en la práctica**

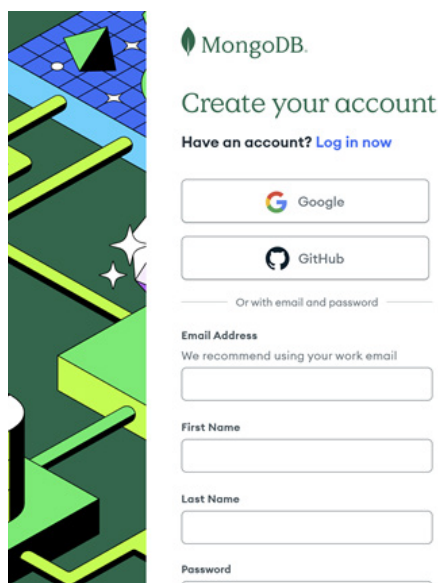
Ahora, exploraremos un poco de la interfaz de MongoDB y sus funcionalidades básicas a través de una instancia de MongoDB en la nube, llamada MongoDB Atlas.



Recursos

Ingresaremos al siguiente enlace y crearemos una cuenta:
[Register MongoDB Atlas](#)

Veremos una página parecida a esta:



MongoDB.

Create your account

Have an account? [Log in now](#)

 Google

 GitHub

Or with email and password

Email Address
We recommend using your work email

First Name

Last Name

Password

Fuente: Elaboración propia.

Luego de registrarnos y validar nuestro mail, veremos una pantalla como la siguiente:

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ M10	☐ Flex	☒ Free
\$0.12/hour Dedicated cluster for development environments and low-traffic applications.	From \$0.011/hour Up to \$30/month For development and testing, with on-demand burst capacity for unpredictable traffic.	Free For learning and exploring MongoDB in a cloud environment.
STORAGE 10 GB	STORAGE 5 GB	STORAGE 512 MB
RAM 2 GB	RAM Shared	RAM Shared
vCPU 2 vCPUs	vCPU Shared	vCPU Shared

✓ **Free forever!** Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name

You cannot change the name once the cluster is created.

Provider

☒ AWS

Quick setup

☒ Automate security setup ⓘ

☒ Preload sample dataset ⓘ

Fuente: Elaboración propia.

Seleccionaremos la opción free y activaremos solo la opción de "Preload sample dataset" del quick setup. La otra opción está relacionada con configuraciones de seguridad, pero para nuestro caso de uso podemos dejarla desmarcada. En la parte de Name, usted podrá cambiar el nombre de su instancia de MongoDB.

Con estas configuraciones hechas, podemos darle a Create Deployment. Veremos la siguiente pantalla:

Connect to Cluster0



You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

1. Add a connection IP address

✓ Your current IP address (170.244.190.162) has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's clusters. Add more later in [Network Access](#).

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.

We autogenerated a username and password. You can use this or create your own.

i You'll need your database user's credentials in the next step. Copy the database user password.

Username

mcermak_db_user

Password

.....

SHOW

Copy

Create Database User

Close

Choose a connection method

Fuente: Elaboración propia.

En esta pantalla, asegúrese de copiar la contraseña y su username en un lugar seguro. Una vez hecho esto, presione "Create Database User" y, a continuación, seleccione "Choose a connection method".

En seguida, veremos esta pantalla:

Connect to Cluster0



Connecting with MongoDB Compass

I don't have MongoDB Compass installed

I have MongoDB Compass installed

1. Select your operating system and download MongoDB Compass

macOS x64 (Intel) (11+)



Download Compass (1.47.1)

or



Copy download URL

Compass is an interactive tool for querying, optimizing, and analyzing your MongoDB data.

2. Copy the connection string, then open MongoDB Compass

☒ Show Password ⓘ

Use this connection string in your application

mongodb+srv:// : @cluster0.bwuzhpp.mongodb.net/



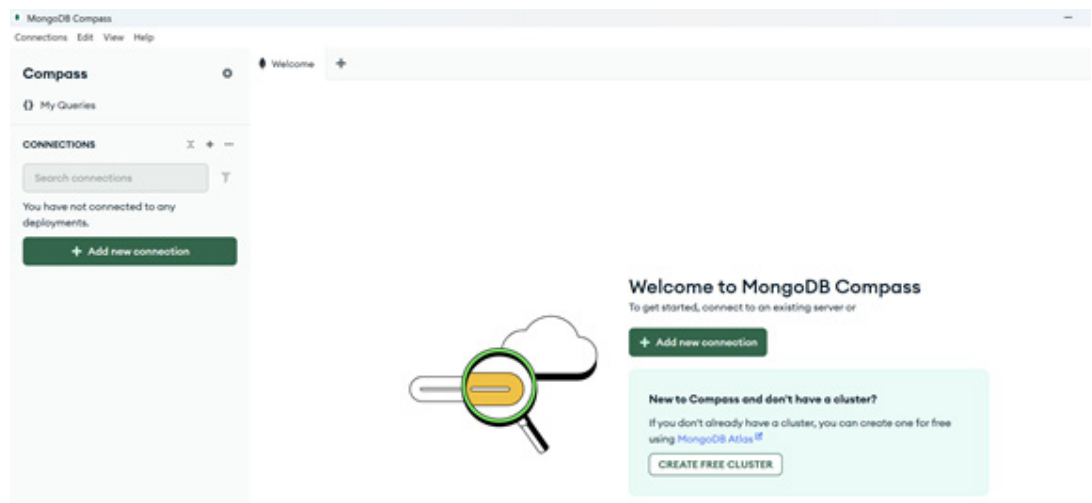
The password for **mccermak_db_user** is included in the connection string for your first time setup. This password will not be available again after exiting this connect flow.

Fuente: Elaboración propia.

En esta pantalla, instalaremos MongoDB Compass, el administrador de bases de datos de MongoDB. Seleccionaremos el sistema operativo que tengamos y le daremos click a "Download Compass", una vez descargado, procedemos a instalarlo.

En la sección 2, podemos ver que nos dan un string de conexión, ese string es importante ya que nos permitirá conectarnos a nuestra instancia de MongoDB a través de MongoDB Compass. Por lo tanto, copie ese string y guárdelo en un lugar seguro. Ahora seguiremos el desarrollo en la aplicación que acabamos de instalar.

Cuando iniciemos la aplicación, veremos una pantalla parecida a la siguiente:

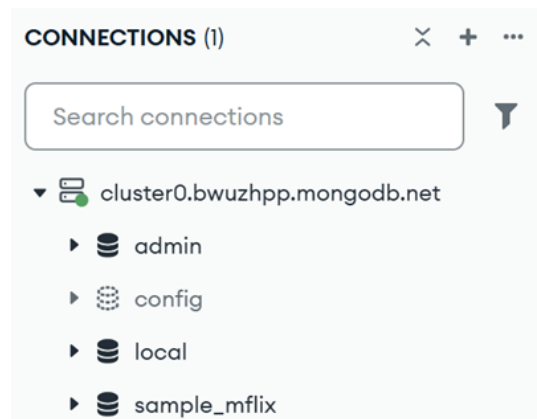


Fuente: Elaboración propia.

Haremos clic en el botón de "Add new connection" para registrar una nueva conexión. Nos aparecerá una pantalla como la siguiente:

Fuente: Elaboración propia.

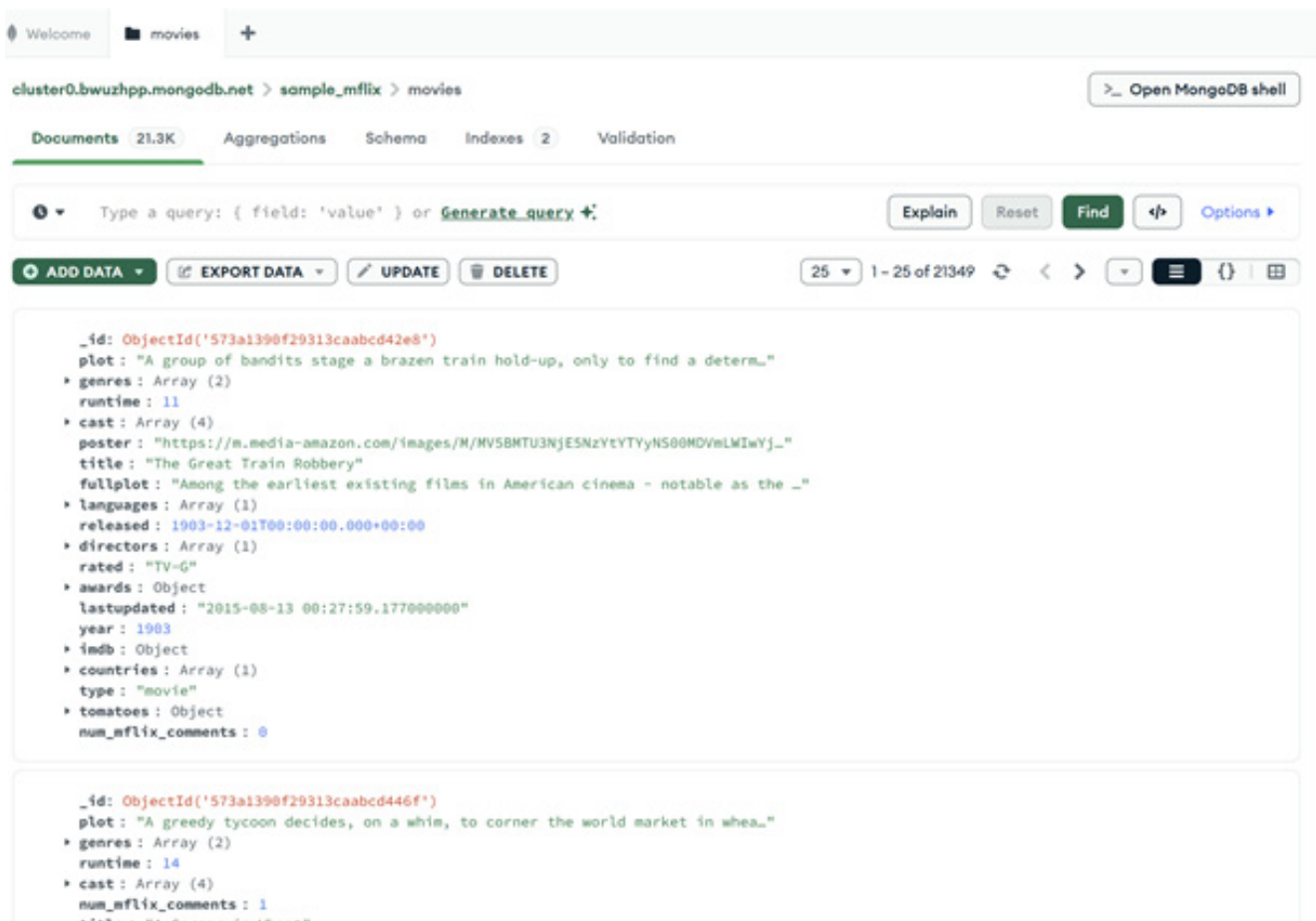
En la sección URI, deberá pegar el connection string que copió y guardó previamente. Después de hacerlo, presione "Save & Connect". Al finalizar, verá la conexión creada en el panel izquierdo de la pantalla.



Fuente: Elaboración propia.

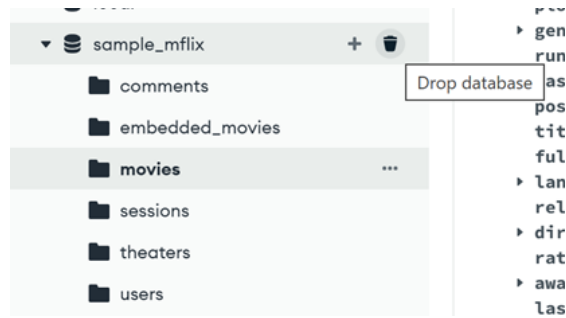
Verá cuatro bases de datos. Para nuestras pruebas utilizaremos “sample_mflix”, creada al activar la opción “Preload sample dataset”. Esta base contiene información sobre películas.

Si hace clic en alguna de sus colecciones, se abrirá una pantalla como la siguiente:



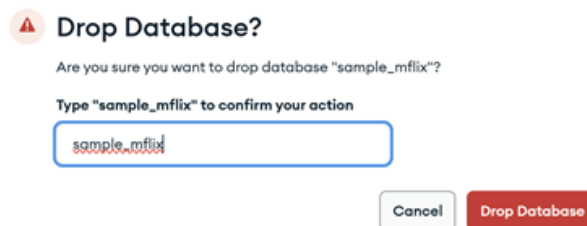
Fuente: Elaboración propia.

Si en algún momento realiza alguna modificación y su base de datos de prueba "sample_mflix" queda en un estado inconsistente, puede volver atrás de la siguiente manera:



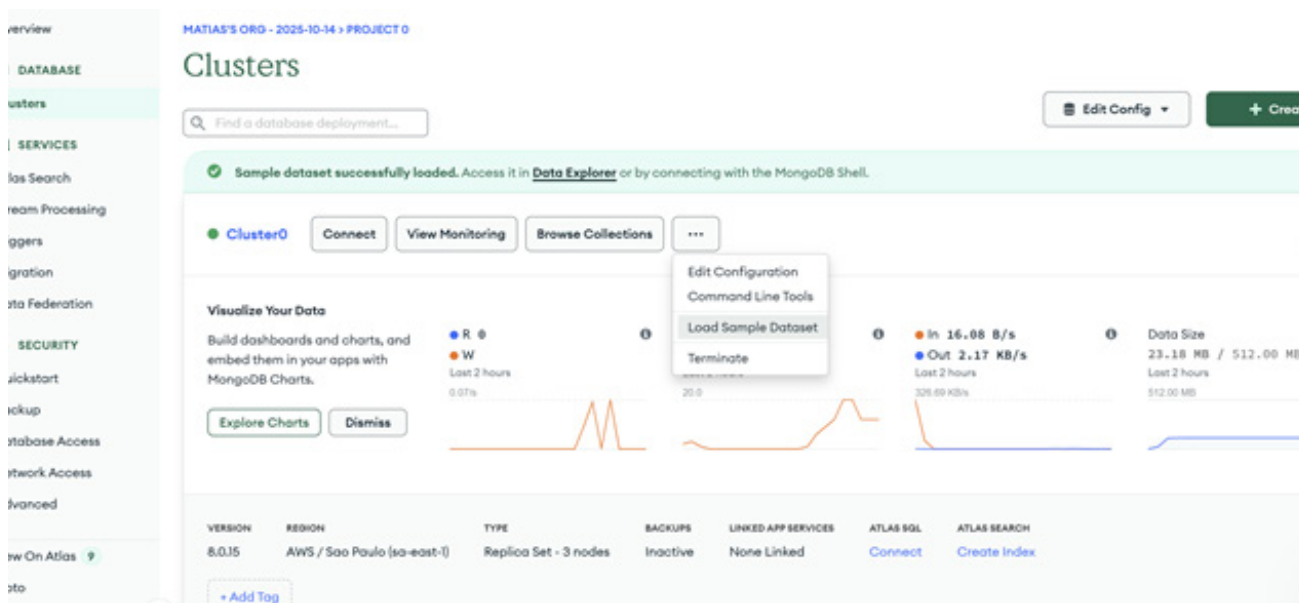
Fuente: Elaboración propia.

Vamos a hacer clic en “Drop Database”:



Fuente: Elaboración propia.

Ingresaremos el nombre de la base de datos que queremos eliminar y haremos clic en “Drop Database”. Seguidamente, iremos nuevamente al sitio de MongoDB Atlas:



Fuente: Elaboración propia.

- › Seleccionaremos "Load Sample Dataset".

Load sample data

Based on your data types and use case, Atlas found these recommended sample datasets (🔧) for you to explore.

Select sample datasets

sample_mflix ✕

Cancel

Load sample data

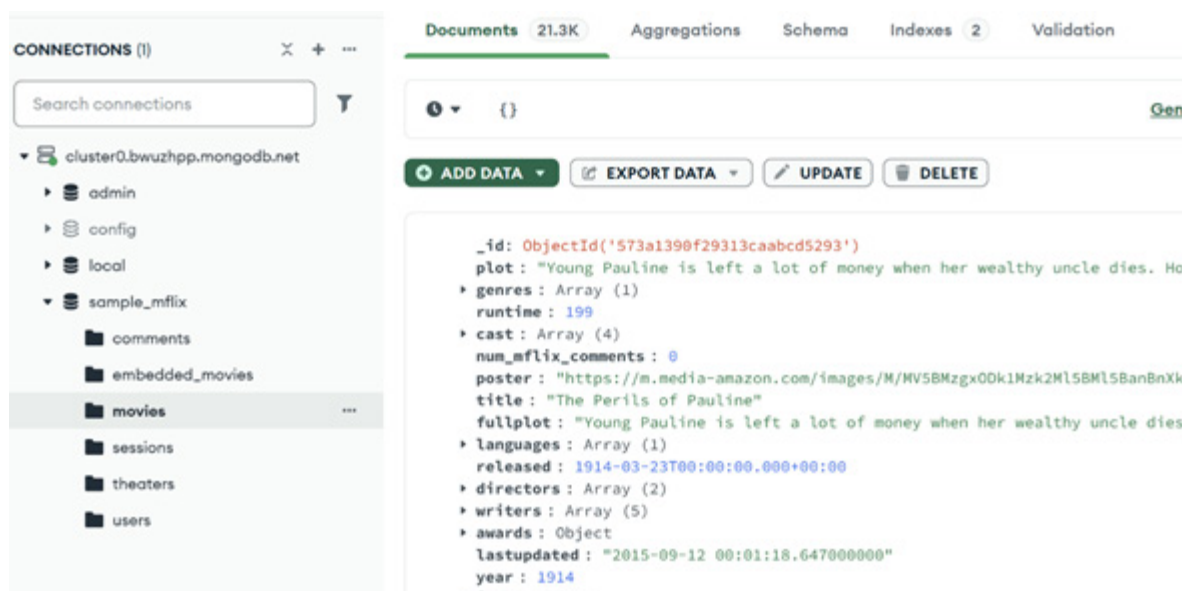
Fuente: Elaboración propia.

Y podremos ver la siguiente pantalla, que nos permite crear de nuevo la base "sample_mflix". Si le damos a "Load sample data", se creará nuevamente esta base de datos de prueba en nuestro sistema.

Ahora, ya podemos pasar a practicar las queries e inserciones en MongoDB.

- **Insertar datos en MongoDB**

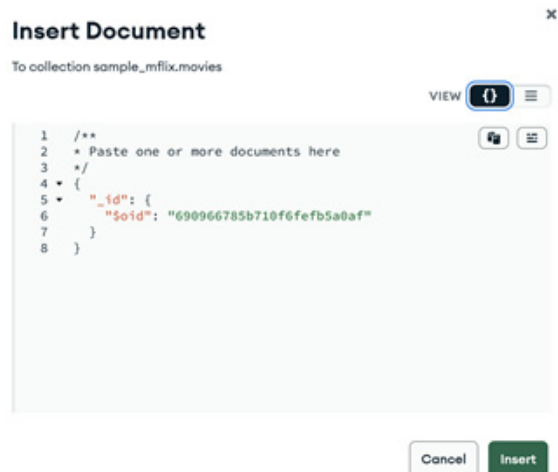
Para insertar datos en una colección, iremos hacia la colección de "movies"



The screenshot shows the MongoDB Atlas interface. On the left, the 'CONNECTIONS (1)' panel lists the 'sample_mflix' database, which contains several collections, including 'movies'. The 'movies' collection is selected. On the right, the 'Documents' tab is active, showing a sample document for the movie 'The Perils of Pauline'. The document includes fields like '_id', 'plot', 'genres', 'runtime', 'cast', 'num_mflix_comments', 'poster', 'title', 'fullplot', 'languages', 'released', 'directors', 'writers', 'awards', 'lastupdated', and 'year'.

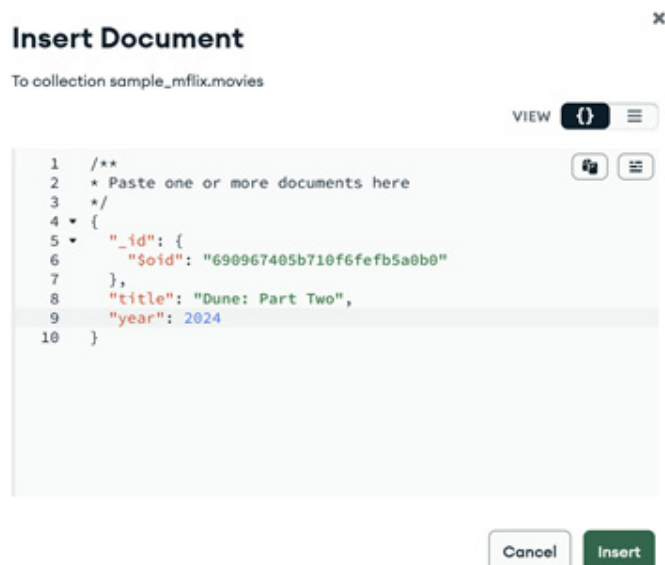
Fuente: elaboración propia.

Cuando estemos en la pantalla de la colección, haremos clic en el botón verde que indica "ADD DATA". Al pulsarlo aparecerán dos opciones, nosotros presionaremos en "Insert Document". Después de esto, veremos la siguiente pantalla:



Fuente: elaboración propia.

Aquí vemos la estructura básica de un documento en MongoDB, también podemos observar que el documento solo incluye un campo de id y nada más. Si quisiéramos, podríamos crear el documento de esta forma, vacío, ya que las colecciones de MongoDB no tienen una estructura predeterminada. Ahora Vamos a agregar la película “Dune: Part Two”:



Fuente: elaboración propia.

Hacemos clic en “Insert”. Si no aparece ningún error, significa que nuestra película se insertó correctamente. Más adelante aprenderemos cómo encontrarla.

- **Actualizar un documento**

Para actualizar un documento, seleccionaremos uno de la lista y haremos clic en el ícono del lápiz:



Fuente: Elaboración propia.

Esto nos permitirá editar el documento que hemos seleccionado:



Fuente: elaboración propia.

A los campos anidados, como "awards", podremos expandirlos y actualizarlos:

```
14 awards : Object
15   wins : 3
16   nominations : 0
17   text : "1 win."
18 lastupdated : "2015-09-12 00:01:18.647000000"
```

Fuente: elaboración propia.

- **Borrar un documento**

Para borrar un documento, simplemente seleccione uno de la lista y haga clic en el ícono del basurero:



Fuente: elaboración propia.

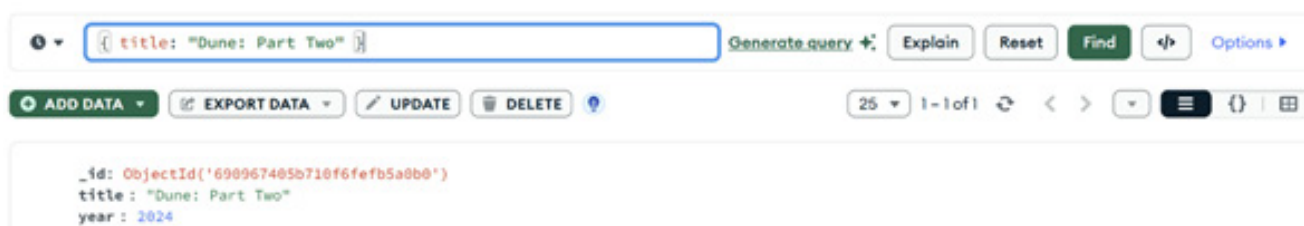
- **Queries en MongoDB**

Ahora aprenderemos a realizar las queries básicas en MongoDB. Estas queries se ejecutarán desde la barra de búsqueda:



Fuente: elaboración propia.

Esta barra de búsqueda funciona de manera similar a un WHERE en SQL, es decir, las condiciones que ingresemos serán utilizadas por MongoDB como filtros. Para empezar, busquemos la película que habíamos creado anteriormente:



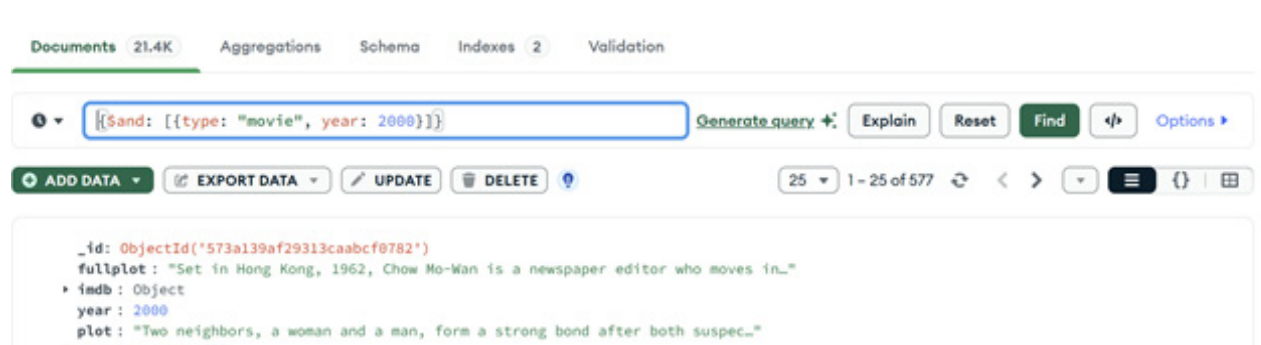
Fuente: elaboración propia.

Lo que hicimos fue solicitar a MongoDB que nos devolviera todos los documentos cuyo atributo título fuera exactamente igual a "Dune: Part Two". Traducido a SQL, sería igual que hacer "SELECT * FROM movies WHERE title = 'Dune: Part Two'" si la colección movies fuera una tabla de una base de datos relacional.

Para realizar una operación AND en MongoDB, podemos utilizar la siguiente consulta:

```
{ $and: [{ condicion_1, condicion_2 } ] }
```

Por ejemplo, si quisiéramos buscar todas las películas que salieron en el año 2000, podríamos hacer lo siguiente:

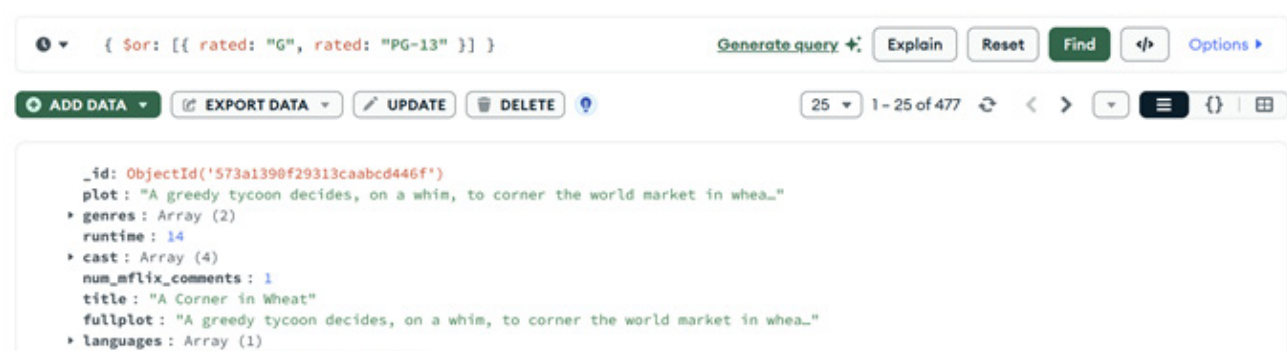


Fuente: elaboración propia.

Si queremos realizar una operación OR podemos utilizar la siguiente consulta:

```
{ $and: [{ condicion_1, condicion_2 } ] }
```

Por ejemplo, para buscar películas que tengan un rating de "G" o "PG-13" podríamos hacer lo siguiente:

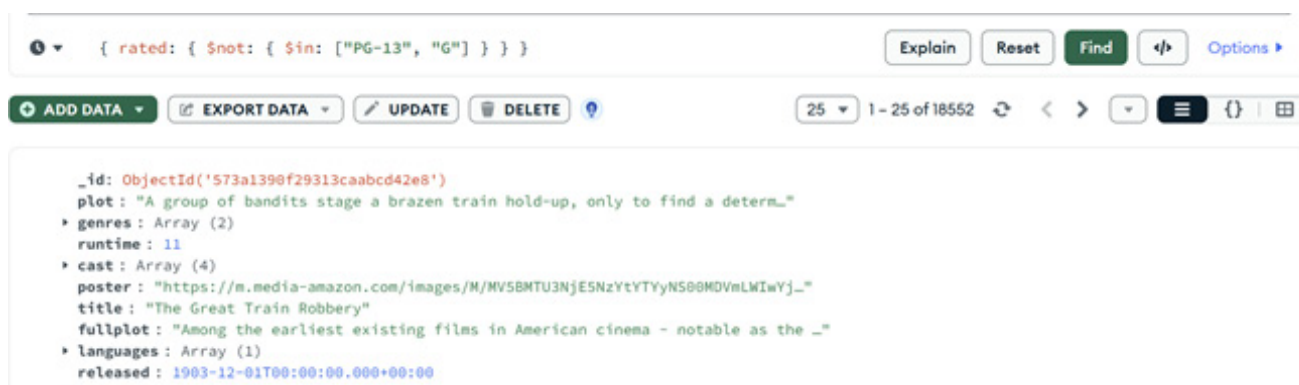


Fuente: elaboración propia.

Por otra parte, el operador de negación se utiliza de la siguiente manera:

```
{ atributo: { $not: { $eq: valor_a_negar } } }
```

La palabra clave `$not` indica que buscamos lo contrario de la condición que coloquemos dentro de ella. En este caso utilizamos el operador de igualdad `$eq`, pero podría emplearse cualquier otro operador. Por ejemplo, si ahora queremos encontrar las películas que NO sean "G" o "PG-13", podemos hacer algo como lo siguiente:



Fuente: elaboración propia.

En este caso, dentro de la negación utilizamos el operador `$in`, que funciona de manera equivalente al operador `IN` de SQL.

A continuación, mencionaremos otros operadores útiles:

- › `$eq`: Coincide con valores que son iguales a un valor especificado.
- › `$gt`: Coincide con valores que son mayores que un valor especificado.
- › `$gte`: Coincide con valores que son mayores o iguales a un valor especificado.
- › `$in`: Coincide con cualquiera de los valores especificados en un array.
- › `$lt`: Coincide con valores que son menores que un valor especificado.
- › `$lte`: Coincide con valores que son menores o iguales a un valor especificado.
- › `$ne`: Coincide con todos los valores que no son iguales a un valor especificado.
- › `$nin`: Coincide si el valor no es igual a ninguno de los valores de una lista dada.
- › `$all`: Requiere que el array contenga todos los valores dados.
- › `$exists`: Permite verificar que el atributo exista en el documento.

Para ilustrarlo con un ejemplo, podemos tomar el caso anterior con el operador `$not` y reformularlo utilizando ahora el operador `$nin`:

🔍 { rated: { \$nin: ["PG-13", "G"] } }

Explain Reset Find ↩️ Options ▶️

➕ ADD DATA ▾ 📄 EXPORT DATA ▾ ✎ UPDATE 🗑️ DELETE ⓘ

25 ▾ 1 - 25 of 18552 ↺ < > ▾ ☰ {} | 🔍

```

_id: ObjectId('573a1398f29313caabdc42e8')
plot: "A group of bandits stage a brazen train hold-up, only to find a determ_"
genres: Array (2)
runtime: 11
cast: Array (4)
poster: "https://m.media-amazon.com/images/M/MV5BMTU3NjE5NzYtYTYyNS00MDVmLWIwYj..."
title: "The Great Train Robbery"
fullplot: "Among the earliest existing films in American cinema - notable as the ..."
languages: Array (1)
released: 1903-12-01T00:00:00.000+00:00
directors: Array (1)

```

Fuente: elaboración propia.

Y si quisiéramos buscar películas del 2000 en adelante, podríamos hacer lo siguiente:

🔍 { year: { \$gte: 2000 } }

Explain Reset Find ↩️ Options ▶️

➕ ADD DATA ▾ 📄 EXPORT DATA ▾ ✎ UPDATE 🗑️ DELETE ⓘ

25 ▾ 1 - 25 of 12570 ↺ < > ▾ ☰ {} | 🔍

```

_id: ObjectId('573a1393f29313caabdc42e8')
plot: "Kate and her actor brother live in N.Y. in the 21st Century. Her ex-bo_"
genres: Array (3)
runtime: 118
metacritic: 44
rated: "PG-13"
cast: Array (4)
poster: "https://m.media-amazon.com/images/M/MV5BNmNlN2VlOTctYTdhMS00NzUxLTg0ZG..."
title: "Kate & Leopold"
fullplot: "Kate and her actor brother live in N.Y. in the 21st Century. Her ex-bo_"
languages: Array (2)

```

Fuente: elaboración propia.

En cambio, si queremos obtener todos los documentos en los que el atributo "poster" existe, podemos utilizar la siguiente consulta:

🔍 { poster: { \$exists: true } }

Generate query ↗️ Explain Reset Find ↩️ Options ▶️

➕ ADD DATA ▾ 📄 EXPORT DATA ▾ ✎ UPDATE 🗑️ DELETE ⓘ

25 ▾ 1 - 25 of 18044 ↺ < > ▾ ☰ {} | 🔍

```

_id: ObjectId('573a1398f29313caabdc42e8')
plot: "A group of bandits stage a brazen train hold-up, only to find a determ_"
genres: Array (2)
runtime: 11
cast: Array (4)
poster: "https://m.media-amazon.com/images/M/MV5BMTU3NjE5NzYtYTYyNS00MDVmLWIwYj..."
title: "The Great Train Robbery"
fullplot: "Among the earliest existing films in American cinema - notable as the ..."
languages: Array (1)
released: 1903-12-01T00:00:00.000+00:00
directors: Array (1)
rated: "TV-G"
awards: Object
lastupdated: "2015-08-13 00:27:59.177000000"
year: 1903
imdb: Object
countries: Array (1)
type: "movie"
tomatoes: Object
num_mflix_comments: 0

```

Fuente: elaboración propia.

También podemos verificar que un array contenga una cantidad determinada de elementos. Por ejemplo, si quisiéramos obtener todas las películas que tengan al menos dos géneros, podríamos hacerlo de la siguiente manera:

The screenshot shows a MongoDB query interface. The query bar contains the following JSON query: `{ "genres.1": { "$exists": true } }`. Below the query bar, there are buttons for "Generate query", "Explain", "Reset", "Find", and "Options". Below these buttons, there are buttons for "ADD DATA", "EXPORT DATA", "UPDATE", and "DELETE". The results section shows a single document for the movie "The Great Train Robbery". The document structure is as follows:

```
{
  "_id": ObjectId("573a1390f29313caabed42e8"),
  "plot": "A group of bandits stage a brazen train hold-up, only to find a determ...",
  "genres": Array (2),
  "runtime": 11,
  "cast": Array (4),
  "poster": "https://m.media-amazon.com/images/M/MvSBMTU3NjESNzYtYTYyNS06MDVhLWZwYj...",
  "title": "The Great Train Robbery",
  "fullplot": "Among the earliest existing films in American cinema - notable as the _",
  "languages": Array (1),
  "released": 1903-12-01T00:00:00.000+00:00,
  "directors": Array (1),
  "rated": "TV-G",
  "awards": Object,
  "lastupdated": "2015-08-13 00:27:59.177000000",
  "year": 1903,
  "imdb": Object,
  "countries": Array (1),
  "type": "movie",
  "tomatoes": Object
}
```

Fuente: elaboración propia.

Dado que los arrays empiezan en 0, al verificar que el item en la posición 1 exista, nos estamos asegurando de que el array tenga al menos 2 items.

Del mismo modo, tenemos el atributo `$regex`, que nos permite buscar coincidencias parciales de texto, de forma similar al operador LIKE en SQL.

Si se desea buscar películas que contengan la palabra "Star", podríamos hacer la siguiente consulta:

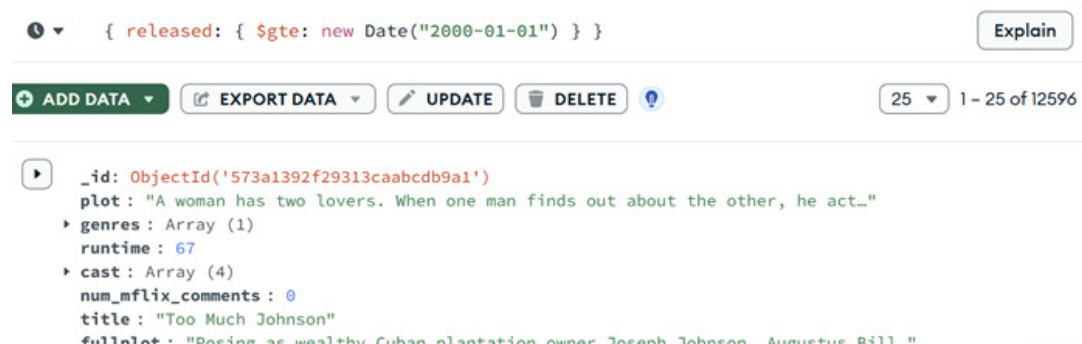
The screenshot shows a MongoDB query interface. The query bar contains the following JSON query: `{ "title": { "$regex": "Star", "$options": "i" } }`. Below the query bar, there are buttons for "Generate query", "Explain", "Reset", "Find", and "Options". Below these buttons, there are buttons for "ADD DATA", "EXPORT DATA", "UPDATE", and "DELETE". The results section shows a single document for the movie "The Stars Look Down". The document structure is as follows:

```
{
  "_id": ObjectId("573a1392f29313caabedbd3"),
  "plot": "Davey Fenwick leaves his mining village on a university scholarship in...",
  "genres": Array (1),
  "runtime": 110,
  "cast": Array (4),
  "num_flix_comments": 1,
  "poster": "https://m.media-amazon.com/images/M/MV5BNzI0ODk3NzU4Nl5BMl5BanBnXkFtZT...",
  "title": "The Stars Look Down",
  "fullplot": "Davey Fenwick leaves his mining village on a university scholarship in...",
  "languages": Array (1),
  "released": 1940-09-16T00:00:00.000+00:00,
  "directors": Array (1)
}
```

Fuente: elaboración propia.

El atributo `$options: "i"` indica que la coincidencia no va a discriminar entre mayúsculas y minúsculas. Esto significa que 'Star Wars' y 'star wars' serán consideradas equivalentes y aparecerán en los resultados. Si se omite este atributo, la búsqueda solo coincidirá con 'Star Wars' tal como está escrita.

En caso de necesitar trabajar con fechas en las consultas, puede hacerlo de la siguiente manera:



Fuente: elaboración propia.

Para buscar fechas, tendremos que crear un objeto Date con la fecha que queremos encontrar. En este caso, buscamos todas las películas cuyo campo released sea mayor o igual que el 1 de enero del año 2000.

Estas mismas operaciones también pueden aplicarse a objetos anidados dentro del documento. Si el objetivo es encontrar las películas que hayan ganado más de 35 premios, podríamos usar una consulta como la siguiente:



Fuente: elaboración propia.

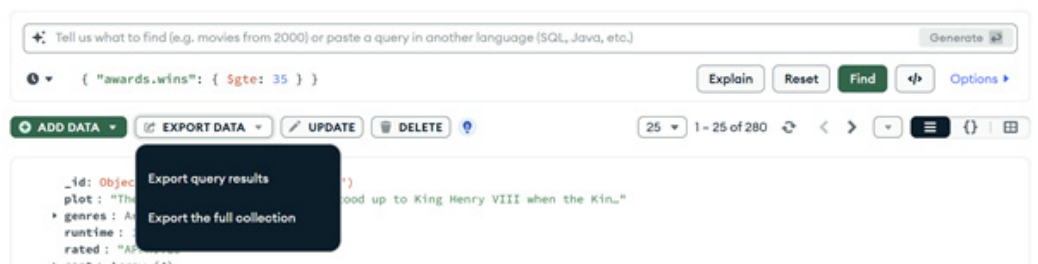
Esto es posible porque el atributo “awards” de cada documento contiene un objeto que, a su vez, incluye el campo “wins”.

```
▼ awards : Object
  wins : 35
  nominations : 6
  text : "Won 6 Oscars. Another 29 wins & 6 nominations."
```

Fuente: elaboración propia.

- **Exportar Datos**

Para cerrar la sección de MongoDB, vamos a aprender cómo exportar datos de MongoDB a formato JSON y CSV.



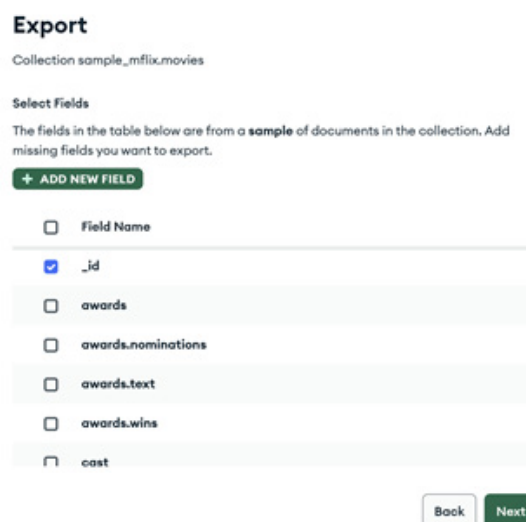
Fuente: elaboración propia.

Para hacerlo, utilizaremos el botón "Export Data" que nos dará a elegir entre dos opciones: exportar los resultados de nuestra query o exportar los datos de toda nuestra colección. En este caso, exportaremos solo los resultados de nuestra query y se nos abrirá la siguiente ventana:



Fuente: elaboración propia.

En esta ventana, podremos elegir si queremos exportar todos los campos o seleccionar los campos específicos que queremos exportar. Iremos por la segunda opción:



Fuente: elaboración propia.

A continuación, se abrirá una nueva pantalla donde podremos seleccionar los atributos que desee incluir en la exportación. Finalmente, llegará a la siguiente vista:

Export

Collection sample_mflix.movies

Export results from the query below

```
db.getCollection('movies').find(
  { 'awards.wins': { '$gte': 35 } },
  {
    _id: 1,
    directors: 1,
    genres: 1,
    rated: 1,
    title: 1,
    year: 1
  }
);
```

Export File Type

JSON CSV

> Advanced JSON Format

Back Export...

Fuente: elaboración propia.

En esta pantalla podremos elegir si exportar en formato JSON o CSV. En este caso, primero seleccionaremos JSON. Luego, solo deberá indicar la ubicación donde quiere guardar el archivo y se generará un documento como el siguiente:

```
{
  "_id": {
    "$oid": "573a1395f29313caabce2ce5"
  },
  "genres": [
    "Biography",
    "Drama",
    "History"
  ],
  "rated": "APPROVED",
  "title": "A Man for All Seasons",
  "directors": [
    "Fred Zinnemann"
  ],
  "year": 1966
},
{
  "_id": {
    "$oid": "573a1396f29313caabce496c"
  },
  "year": 1972,
  "genres": [
    "Drama",
    "Musical",
    "Romance"
  ],
  "rated": "PG",
  "title": "Cabaret",
  "directors": [
    "Bob Fosse"
  ]
}
```

Fuente: elaboración propia.

Este documento contiene todos los campos seleccionados. Si exportamos un CSV, veremos un archivo con el siguiente formato:

A	B	C	D	E	F	G	H	I
_id	genres[0]	genres[1]	genres[2]	title	directors[0]	directors[1]	year	
573a1395f29313caabce2ce	Biography	Drama	History	A Man for All Seasons	Fred Zinnemann		1966	
573a1396f29313caabce49e	Drama	Musical	Romance	Cabaret	Bob Fosse		1972	
573a1397f29313caabce5c1	Drama			One Flew Over the Cuckoo's Nest	Milos Forman		1975	
573a1397f29313caabce68f	Action	Adventure	Fantasy	Star Wars: Episode IV - A New Hope	George Lucas		1977	
573a1397f29313caabce731	Drama			Kramer vs. Kramer	Robert Benton		1979	
573a1397f29313caabce80e	Biography	Drama	History	Reds	Warren Beatty		1981	
573a1397f29313caabce841	Family	Sci-Fi		E.T. the Extra-Terrestrial	Steven Spielberg		1982	
573a1397f29313caabce84e	Biography	Drama	History	Gandhi	Richard Attenborough		1982	
573a1398f29313caabce8dc	Comedy	Drama		Terms of Endearment	James L. Brooks		1983	
573a1398f29313caabce8f7	Biography	Drama	History	Amadeus	Milos Forman		1984	
573a1398f29313caabcea7f	Biography	Drama	History	The Last Emperor	Bernardo Bertolucci		1987	
573a1398f29313caabcebe4	Adventure	Drama	Western	Dances with Wolves	Kevin Costner		1990	
573a1398f29313caabcebf8	Biography	Crime	Drama	Goodfellas	Martin Scorsese		1990	
573a1399f29313caabcecbf	Crime	Drama	Thriller	The Silence of the Lambs	Jonathan Demme		1991	

Fuente: elaboración propia.



Recursos

Si quisiera aprender un poco más sobre MongoDB, les compartimos el enlace a la documentación oficial, donde podrán explorar con mayor detalle sus funcionalidades y los distintos tipos de consultas que ofrece.

- **MongoDB Query Language**



Actividad

En este momento, lo/la invitamos a realizar la primera actividad vinculada con los contenidos trabajados hasta aquí en el módulo.

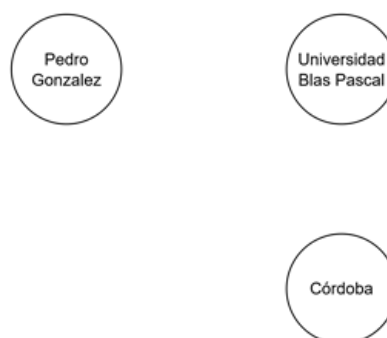
- › **Base de datos de grafos (Neo4J)**

Ya aprendimos sobre nuestra primera base de datos NoSQL, ahora investigaremos sobre las BD de grafos. Las bases de datos de grafos son muy útiles cuando las conexiones entre los datos son tan importantes como los mismos datos.

Neo4j almacena y organiza los datos mediante un sistema de grafos compuesto por cuatro elementos fundamentales: los nodos, que representan las entidades básicas en el grafo; las relaciones, que establecen conexiones dirigidas entre estos nodos; las etiquetas, que actúan como categorías o clasificaciones para agrupar nodos similares; y las propiedades, que almacenan atributos descriptivos tanto para los nodos como para las relaciones.

- › **Nodos**

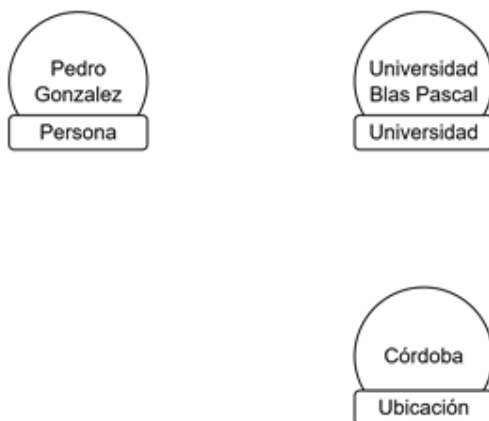
Los nodos son los círculos en un gráfico y representan las entidades en nuestro modelo de datos.



Fuente: elaboración propia.

› Etiquetas

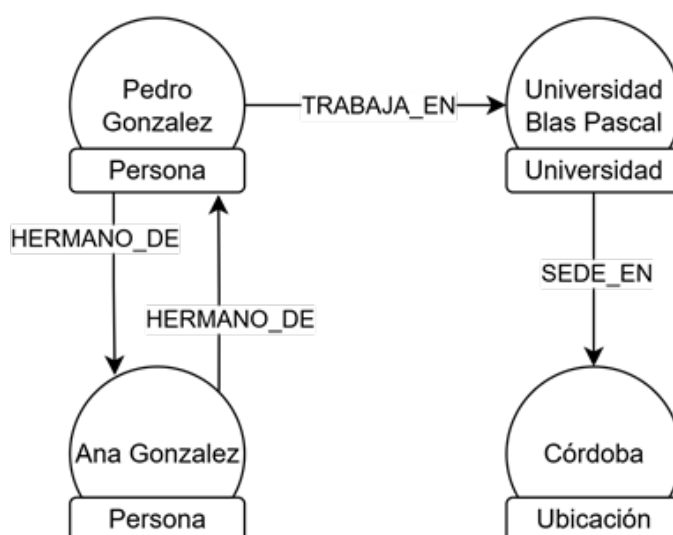
A su vez, los nodos son agrupados y categorizados con etiquetas. En nuestro ejemplo, podríamos poner las siguientes etiquetas:



Fuente: elaboración propia.

› Relaciones

Por otra parte, las relaciones son las líneas en el gráfico que representan cómo los nodos se conectan entre sí. Las relaciones tienen dirección, es decir, tienen un “nodo inicio” y un “nodo fin”. Volvamos al ejemplo anterior: Pedro González trabaja en Universidad Blas Pascal, pero Universidad Blas Pascal no trabaja en Pedro González. En cambio, Pedro González es hermano de Ana González, y Ana González también es hermano de Pedro, para describir esta relación bidireccional utilizamos dos relaciones HERMANO_DE.

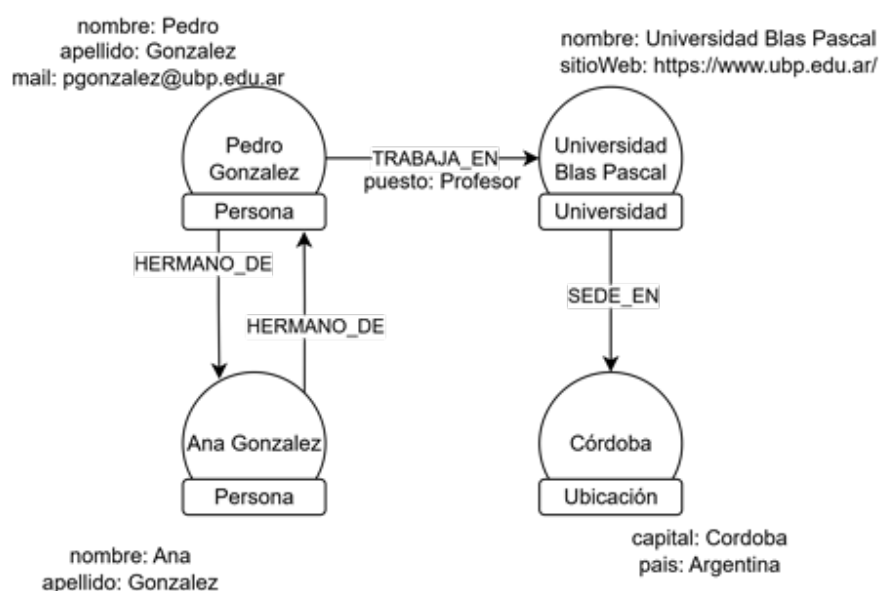


Fuente: elaboración propia.

› Propiedades

Finalmente, las propiedades son los datos que se almacenan en los nodos o en las relaciones en formato “clave: valor”. Cada propiedad cuenta con un tipo de dato asociado (integer, boolean, string, list, entre otros).

Ejemplo:



Fuente: elaboración propia.

› Neo4J en la práctica

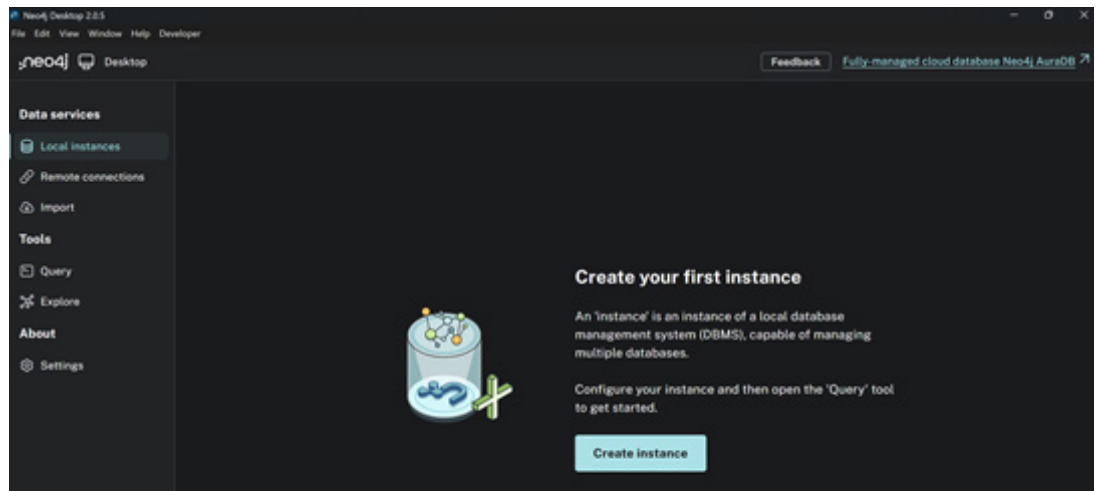
En esta sección, nos adentraremos en una introducción a Neo4j y veremos cómo es trabajar con una base de datos de grafos.



En este sentido, lo primero que haremos será instalar Neo4J Desktop, la aplicación de escritorio de Neo4J.

› *Link de descarga: Neo4j*

Una vez descargado, procedemos a instalar la aplicación. Una vez finalizada la instalación, al abrirla veremos la siguiente pantalla:



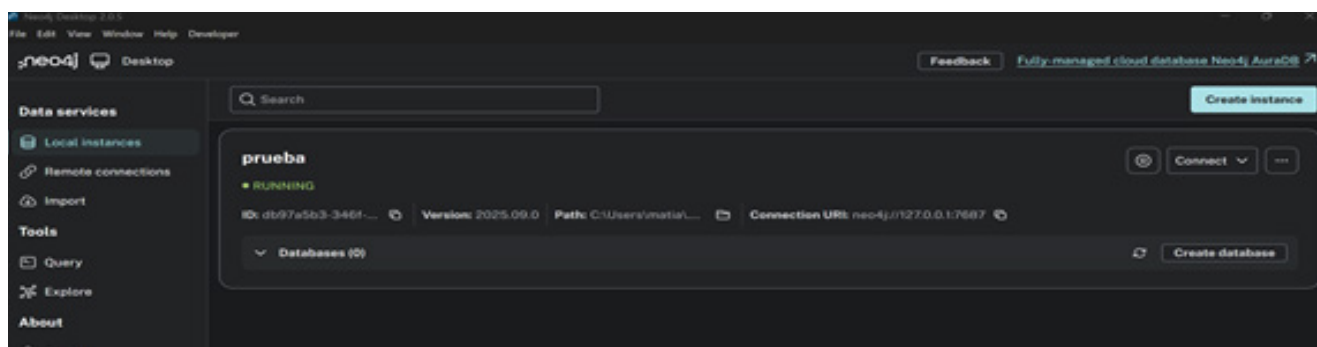
Fuente: elaboración propia.

En esta pantalla, le daremos clic a "Create Instance" y, al hacerlo, se abrirá una nueva ventana:

Fuente: elaboración propia.

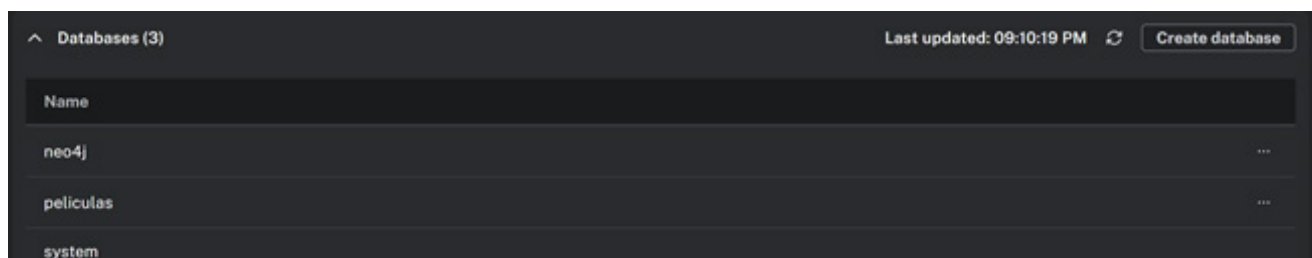
Aquí vamos a ingresar el nombre de nuestra instancia de Neo4j; en nuestro caso, utilizaremos "prueba". Luego, debemos establecer una contraseña, que puede ser la que deseemos. Una vez terminadas las configuraciones, presionaremos en "Create" y comenzará el proceso de creación de la base de datos.

› Cuando finalice, veremos la instancia creada:



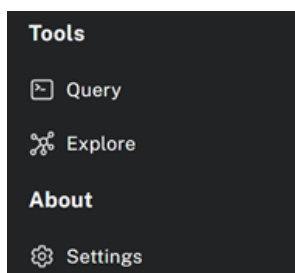
Fuente: elaboración propia.

Para crear una base de datos, daremos clic en “Create database” y generaremos una base de datos llamada "películas". Una vez hecha, podremos verla en la lista de bases de datos de la instancia:



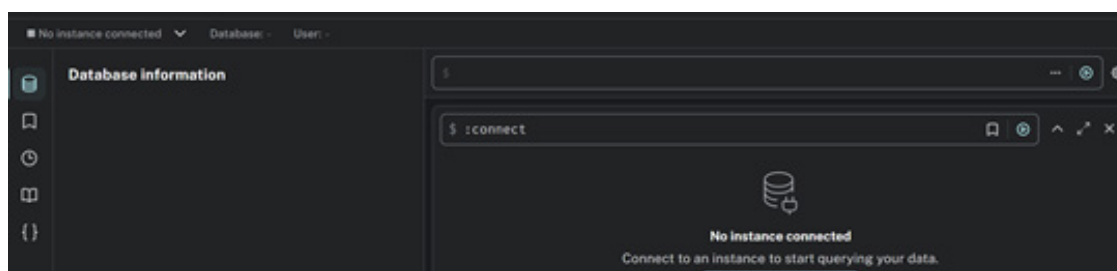
Fuente: elaboración propia.

A continuación, veremos cómo interactuar con nuestra base de datos. Para hacerlo, utilizaremos el lenguaje de consulta de grafos CYPHER. Para ingresar a la consola y ejecutar consultas, haremos clic en el botón de la barra de la izquierda "Query".



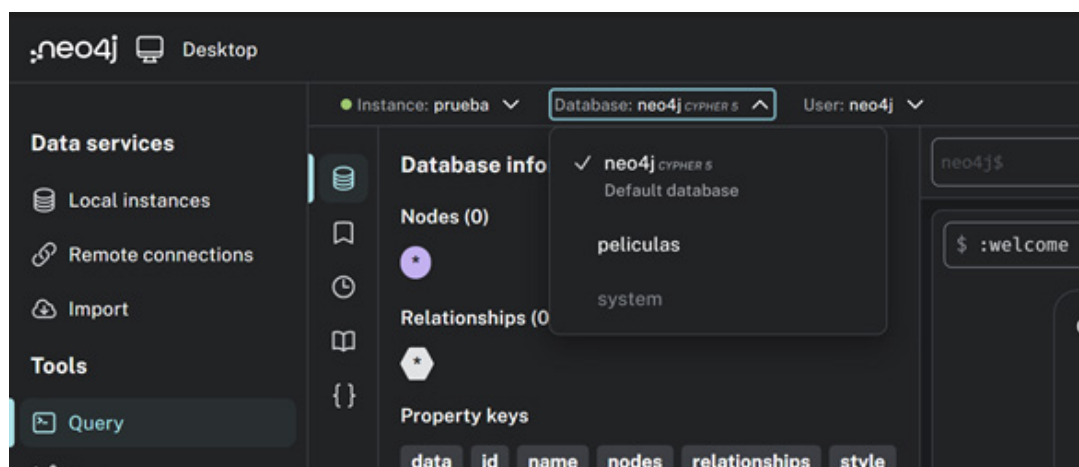
Fuente: elaboración propia.

Al presionar el botón, se abrirá esta pantalla:



Fuente: elaboración propia.

Presionando en "Connect to instance" podremos conectarnos a la instancia "prueba" que habíamos creado anteriormente. Una vez conectado, podemos hacer clic en el selector de bases de datos y seleccionar nuestra base de datos "películas".



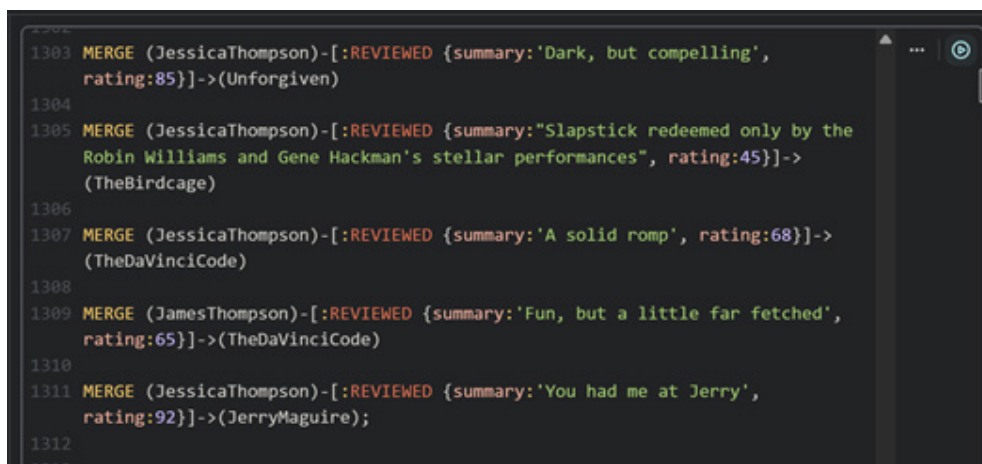
Fuente: elaboración propia.



Una vez seleccionada, cargaremos la base de datos "películas" con información de prueba para poder aprender a utilizar CYPHER.

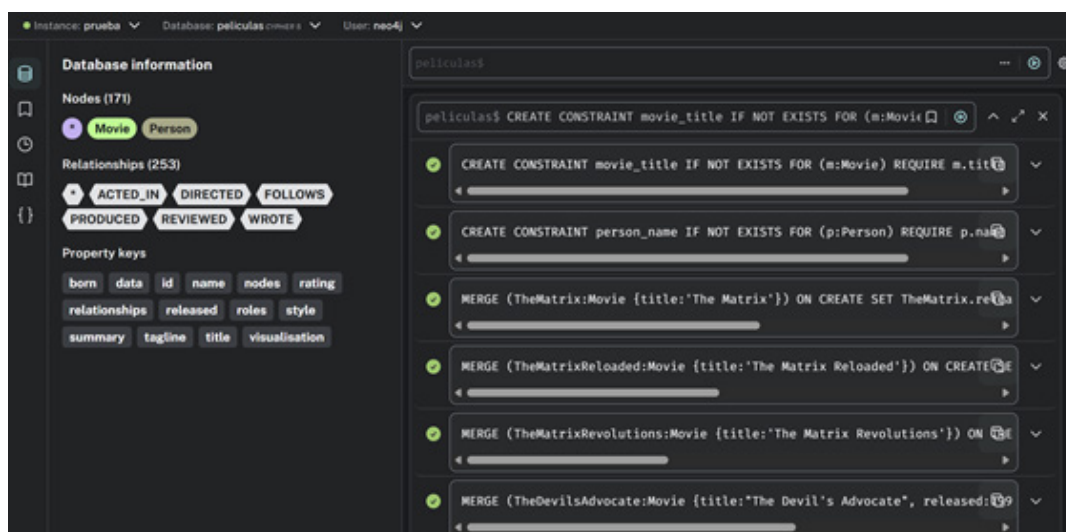
› `{{base_de_datos_películas_con_informacion_de_prueba_pdf}}`

Copiaremos el contenido del archivo (CTRL + A, CTRL+C) y lo pegaremos en el campo de texto de la consola (CTRL+V).



Fuente: elaboración propia.

Una vez pegado, le daremos clic al botón "Run" que se encuentra arriba a la derecha para ejecutar los comandos. Cuando se terminen de ejecutar, nuestra consola se verá algo así:



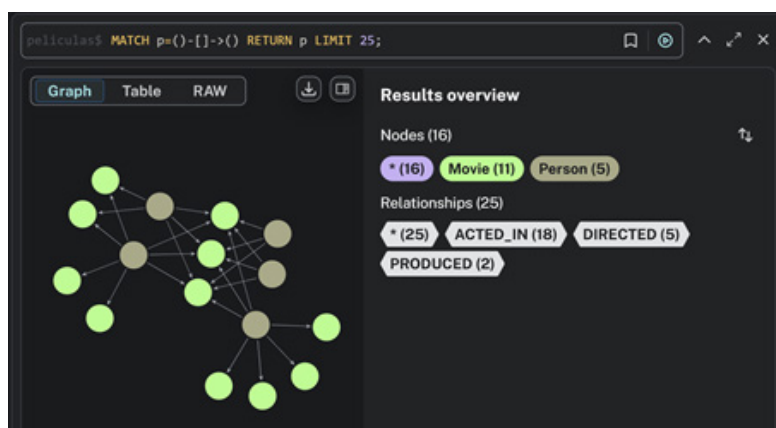
Fuente: elaboración propia.

La sección de "Database Information" nos brinda un buen vistazo de los tipos de nodos, relaciones y propiedades que existen en nuestra base de datos. Siguiendo con el ejemplo que venimos trabajando, podemos ver que existen 2 tipos de nodos: "Movie" y "Person". También se identifican seis tipos de relaciones que pueden surgir entre estos nodos. Además, se observan los nombres de las propiedades con las que nos encontraremos al revisar la información. A simple vista, podemos apreciar que se trata de una base de datos que relaciona películas con personas y que estas relaciones no simplemente vinculan las películas con sus actores, sino también con sus productores, directores, críticos, etc.

Antes de empezar a indagar en CYPHER, lo/a invito a que copie y pegue este comando en la caja de comandos y lo ejecute:

```
MATCH p=()-[]->>() RETURN p LIMIT 25;
```

Podrá ver lo siguiente:



Fuente: elaboración propia.

Este es el visualizador de grafos de Neo4j. Puede hacer clic en los nodos y en las relaciones, y así descubrir qué películas se conectan con qué personas, cuál es la relación entre ellas y cuáles son sus propiedades.

En general, los comando de CYPHER tienen la siguiente estructura:

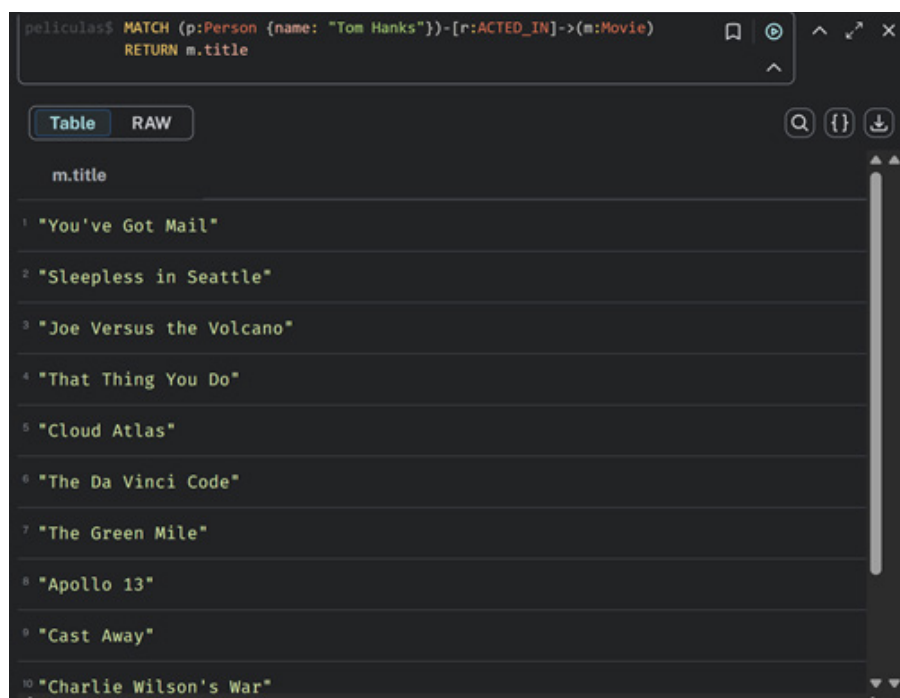
```
MATCH (nodo:Etiqueta {propiedad: "valor"})-[r:RELACION]
->(nodo2:Etiqueta2 {propiedad2: "valor2"})
```

Esto le indica a Neo4j qué patrones debe devolvernos. Por ejemplo, si quisiéramos saber en qué películas actuó Tom Hanks, podríamos ejecutar la siguiente consulta:

```
MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN]->
(m:Movie) RETURN m.title
```

Leyéndolo de izquierda a derecha, primero tenemos la palabra clave MATCH, que le indica a Neo4j que estamos por definir un patrón que queremos que encuentre. Luego aparece el primer nodo, "p", con etiqueta Person, cuya propiedad "name" es "Tom Hanks". A continuación, se indica la relación "-[r:ACTED_IN]->", que especifica que la relación "r" tiene que ser del tipo "ACTED_IN". Lo que sigue es "(m:Movie)", que indica que, del otro lado de la relación, esperamos encontrar un nodo "m" con la etiqueta "Movie". Por último, aparece RETURN. Si ha trabajado con otros lenguajes de programación, sabrá que RETURN se utiliza para devolver un valor; en este caso, estamos devolviendo el título de la película, haciendo referencia al nodo m y accediendo a su atributo "title".

Si ejecutamos el comando, veremos una respuesta como la siguiente:

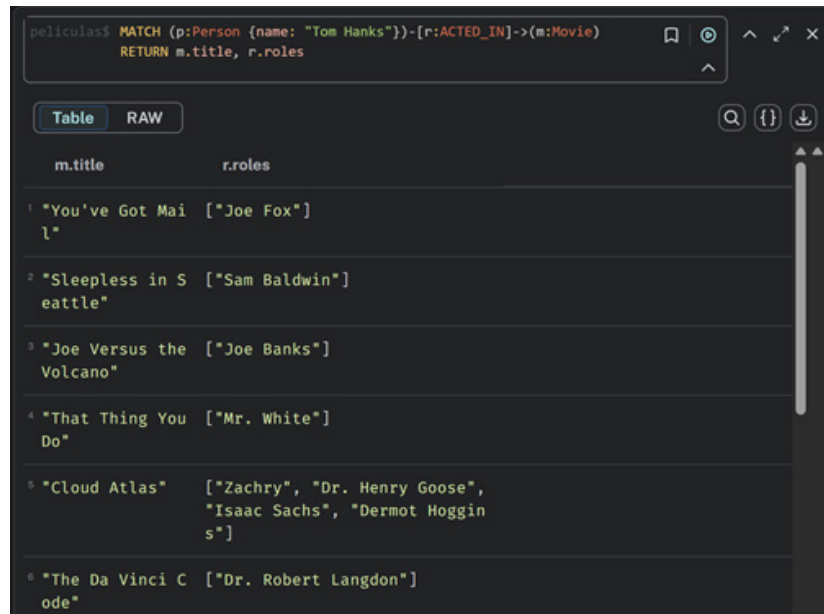


Fuente: elaboración propia.

Podríamos también ver el rol en el que actuó dentro de la película, el atributo que representa el rol está guardado dentro de la relación, por lo que podemos hacer lo siguiente:

```
MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN]->(m:Movie)
RETURN m.title, r.roles
```

Ahora no solo devolvemos el título de la película, sino también los roles en los que actuó:



The screenshot shows a Cypher query interface with a dark theme. The query entered is: `MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN]->(m:Movie) RETURN m.title, r.roles`. Below the query editor, there are tabs for 'Table' and 'RAW', with 'Table' selected. The results are displayed in a table with two columns: 'm.title' and 'r.roles'. The table contains six rows of data, each representing a movie and the roles Tom Hanks played in it.

m.title	r.roles
"You've Got Mail"	["Joe Fox"]
"Sleepless in Seattle"	["Sam Baldwin"]
"Joe Versus the Volcano"	["Joe Banks"]
"That Thing You Do"	["Mr. White"]
"Cloud Atlas"	["Zachry", "Dr. Henry Goose", "Isaac Sachs", "Dermot Hoggins"]
"The Da Vinci Code"	["Dr. Robert Langdon"]

Fuente: Elaboración propia.

Ahora, si quisiéramos ver solo las películas en las que actuó que fueron estrenadas a partir del año 2000 podemos hacer lo siguiente:

```
MATCH (p:Person {name: "Tom Hanks"})-[r:ACTED_IN]->(m:Movie)
WHERE m.released > 2000
RETURN m.title, r.roles
```

Seguro puede reconocer la cláusula WHERE de la sección de SQL. En CYPHER lo usamos para lo mismo: filtrar los datos y especificar más nuestra búsqueda. Es más, si quisiéramos, en vez de aplicar el filtro del "Tom Hanks" en el nodo, podríamos aplicarlo en la cláusula WHERE:

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE m.released > 2000 AND p.name = "Tom Hanks"
RETURN m.title, r.roles
```

```

1 MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
2 WHERE m.released > 2000 AND p.name = "Tom Hanks"
3 RETURN m.title, r.roles

```

	m.title	r.roles
1	"Cloud Atlas"	["Zachry", "Dr. Henry Goose", "Isaac Sachs", "Dermot Hoggins"]
2	"The Da Vinci Code"	["Dr. Robert Langdon"]
3	"Charlie Wilson's War"	["Rep. Charlie Wilson"]
4	"The Polar Express"	["Hero Boy", "Father", "Conductor", "Hobo", "Scrooge", "Santa Claus"]

Fuente: Elaboración propia.

En este punto, podemos complejizarlo un poco más. Si deseamos conocer a los directores con los que trabajó Tom Hanks, podríamos realizar una consulta como la siguiente:

```

MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)-[r2:DIRECTED]-(p2:Person)
WHERE p.name = "Tom Hanks"
RETURN m.title, p2.name

```

En esta consulta integramos una segunda relación y un segundo nodo, este patrón se extiende y nos permite conocer a los directores con los cuales Tom Hanks trabajó.

```

1 MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)-[r2:DIRECTED]-(p2:Person)
2 WHERE p.name = "Tom Hanks"
3 RETURN m.title, p2.name

```

	m.title	p2.name
1	"You've Got Mail"	"Nora Ephron"
2	"Sleepless in Seattle"	"Nora Ephron"
3	"Joe Versus the Volcano"	"John Patrick Shanley"
4	"That Thing You Do!"	"Tom Hanks"
5	"Cloud Atlas"	"Lilly Wachowski"
6	"Cloud Atlas"	"Lana Wachowski"

Fuente: Elaboración propia.

Como último ejemplo, imagine que queremos encontrar recomendaciones de actores con los cuales Tom Hanks podría trabajar en un futuro. Podemos definir a un posible actor como alguien con quien Tom Hanks aún no haya trabajado, pero que sí haya actuado junto a alguno de sus co-protagonistas.

Nuestro primer paso será conseguir los co-protagonistas de Tom Hanks:

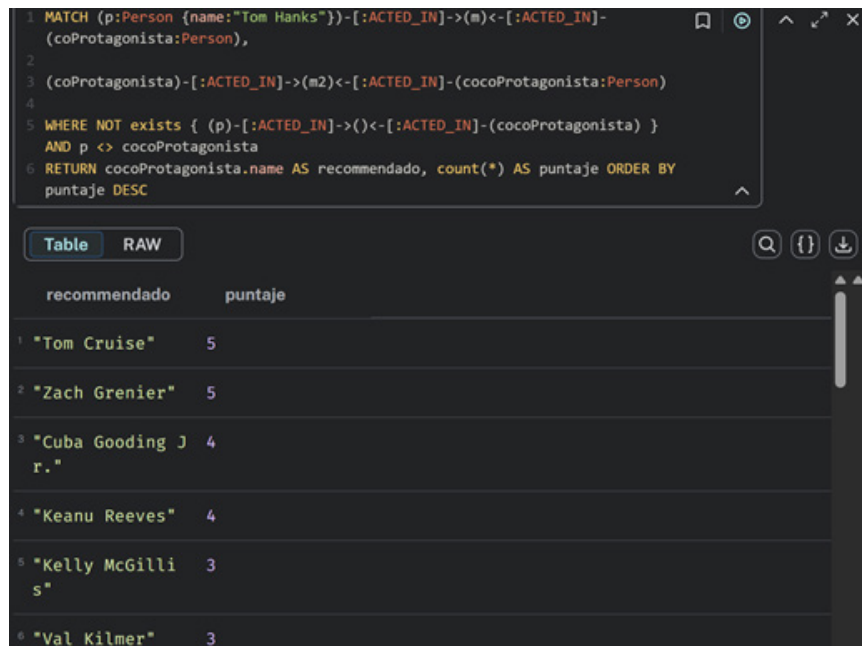
```
MATCH (p:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)
<-[:ACTED_IN]-(coProtagonista:Person) RETURN coProtagonista.name
```

Ya con los co-protagonistas encontrados, podemos extender la query a los que serían co-protagonistas de los co-protagonistas de Tom Hanks, que son las personas que estamos buscando:

```
MATCH (p:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coProtago-
nista:Person),
(coProtagonista)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoProtagonista:Person)
WHERE NOT exists { (p)-[:ACTED_IN]->()<-[:ACTED_IN]-(cocoProtagonista) } AND p <>
cocoProtagonista
RETURN cocoProtagonista.name AS recomendado, count(*) AS puntaje ORDER BY
puntaje DESC
```

Con esta query expandida, accedemos a través del nodo "coProtagonista" que trabajó con Tom Hanks a cada uno de esos actores que nos sirven para nuestra recomendación. En la cláusula WHERE agregamos dos condiciones, la primera evita recomendar actores que ya hayan trabajado con Tom Hanks; la segunda busca que esta posible recomendación no sea el mismo Tom Hanks.

Por último, en el return devolvemos el nombre del actor recomendado y utilizamos una función llamada COUNT(*), que cuenta todas las veces que aparece cada nodo en el patrón. En este sentido, la función COUNT(*) funciona como un GROUP BY de SQL pero implícito, ya que agrupa cada aparición de cada nodo y los cuenta. El resultado termina siendo algo así:



```

1 MATCH (p:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-
  (coProtagonista:Person),
2
3 (coProtagonista)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cocoProtagonista:Person)
4
5 WHERE NOT exists { (p)-[:ACTED_IN]->()-[:ACTED_IN]-(cocoProtagonista) }
  AND p <> cocoProtagonista
6 RETURN cocoProtagonista.name AS recomendado, count(*) AS puntaje ORDER BY
  puntaje DESC

```

	recomendado	puntaje
1	"Tom Cruise"	5
2	"Zach Grenier"	5
3	"Cuba Gooding Jr."	4
4	"Keanu Reeves"	4
5	"Kelly McGillis"	3
6	"Val Kilmer"	3

Fuente: Elaboración propia.

Podemos ver que la persona más recomendada para trabajar con Tom Hanks fue Tom Cruise.



Recursos

Con este primer acercamiento, damos por terminada la sección de Neo4j y las bases de datos de grafos. Para aprender más sobre ellas, y para consultar a la hora de realizar los ejercicios, les recomiendo los siguientes materiales:

- › Documentación Neo4j
- › Guía Rápida de CYPHER



Actividad

Lo/la invitamos a realizar la segunda actividad, que le permitirá continuar trabajando sobre los contenidos abordados en este módulo y poner en práctica lo aprendido.

- › **CosmosDB: Bases de Datos Multi-Modelo**

Para cerrar este recorrido, pasamos a la última sección de este módulo 3. En esta sección, exploraremos qué son las bases de datos multi-modelo, y cómo se aplica en CosmosDB.

Hasta ahora, hemos visto 3 tipos de bases de datos:

1. SQL: Para datos estructurados y relaciones fijas (modelo relacional).
2. MongoDB: Para datos semiestructurados en forma de documentos JSON (*modelo documental*).
3. Neo4j: Para datos interconectados y relaciones complejas (modelo de grafos).

CosmosDB nos permite emular estos modelos dentro de una sola base de datos. Esto es posible porque todos los modelos que implementa CosmosDB se apoyan, en el fondo, en un mismo formato de datos.

Podemos interactuar con CosmosDB usando la API de SQL (propia de Microsoft), tratándolo como si fuera una base de datos relacional. También podemos usar la API para MongoDB, permitiendo que las aplicaciones diseñadas para MongoDB se conecten a CosmosDB como si fuera un clúster de MongoDB, aunque en el fondo utilice la tecnología de CosmosDB. Además, es posible usar la API de Gremlin para trabajar con él como si fuera una base de datos de grafos, al estilo de Neo4j.

Este sistema multimodelo lo posiciona como una solución teórica ideal para entornos que requieren flexibilidad en el manejo de datos bajo una única infraestructura gestionada.



**Lectura
complementaria**

Para saber más sobre CosmosDB, puede acceder al siguiente artículo: [Technical Overview of Azure MongoDB](#)

MÓDULO 3 | GLOSARIO

API Gremlin (CosmosDB): Permite trabajar con CosmosDB como una base de grafos.

API MongoDB (CosmosDB): Emula el funcionamiento de MongoDB dentro de CosmosDB.

API SQL (CosmosDB): Permite usar CosmosDB como si fuera una base relacional.

Base de datos NoSQL: Bases de datos no relacionales que pueden almacenar datos en distintos modelos como documentos, grafos o multimodelo.

BSON: Formato binario basado en JSON utilizado por MongoDB para almacenar documentos.

Colección (MongoDB): Conjunto de documentos, similar a una tabla, pero sin esquema fijo.

CosmosDB: Base de datos NoSQL multimodelo de Azure que permite trabajar con APIs de SQL, MongoDB y Gremlin.

Cypher: Lenguaje de consulta de Neo4j para buscar patrones y relaciones en grafos.

Documento (MongoDB): Unidad básica de datos equivalentes a una fila, representada en formato JSON/BSON y con estructura flexible.

Etiqueta (Label): Categoría que clasifica un nodo dentro del grafo.

_id / ObjectId: Identificador único y automático asignado a cada documento en MongoDB.

MongoDB: Base de datos NoSQL de tipo documental que organiza la información en colecciones y documentos BSON.

MongoDB Atlas: Servicio en la nube para crear, administrar y desplegar instancias de MongoDB.

MongoDB Compass: Interfaz gráfica que permite conectarse a MongoDB, ejecutar consultas y manipular documentos.

Neo4j: Base de datos NoSQL basada en grafos, ideal para datos altamente conectados.

Nodo: Entidad dentro del grafo, como personas, películas o productos.

Operadores (MongoDB): Instrucciones para filtrar datos, como \$eq, \$gt, \$lt, \$in, \$regex o \$exists.

Propiedades: Atributos clave:valor que describen nodos o relaciones.

Relación: Conexión dirigida entre nodos, que puede incluir propiedades.

MÓDULO 3 | ACTIVIDADES



ACTIVIDAD N° 1: Queries en MongoDB

Utilizando la colección "movies" de la base de datos "sample_mflix" en MongoDB Compass, resuelva los siguientes ejercicios.

1. Busque todas las películas que tengan una duración mayor a 120 minutos.
2. Encuentre películas lanzadas antes del año 1990.
3. Muestre las películas que tengan el género "Action".
4. Busque películas donde el campo "rated" no exista.
5. Encuentre películas en las que el campo "cast" exista.
6. Busque películas donde el campo "genres" tenga al menos tres elementos.
7. Muestre las películas que tengan más de 10 nominaciones en la sección "awards".
8. Encuentre películas que tengan una calificación menor a 5 en IMDb.
9. Busque películas con una calificación IMDb entre 6 y 8 (ambos inclusive).
10. Encuentre películas que tengan el rating "PG" y duración mayor a 90 minutos.
11. Busque películas que sean de los géneros "Horror" o "Thriller".
12. Muestre películas que no sean de los géneros "Comedy" ni "Romance".
13. Busque películas que contengan la palabra "War" en el título (*sin distinguir mayúsculas o minúsculas*).
14. Encuentre películas cuyo título comience con la palabra "The". Nota: Investigar más a fondo la funcionalidad de \$regex
15. Busque películas cuyo argumento (plot) contenga la palabra "family", sin importar mayúsculas o minúsculas.
16. Encuentre películas lanzadas entre el 1 de enero de 2005 y el 31 de diciembre de 2010.
17. Busque películas donde el campo "imdb.votes" exista y sea mayor a 5000.
18. Muestre películas que tengan el campo "directors" con más de un nombre.
19. Encuentre películas en las que el campo "countries" incluya solamente "USA" y "UK", indistinto del orden.
20. Busque películas donde el campo "languages" incluya al menos 2 idiomas.

ACTIVIDAD 1 | Clave de corrección CC 1

› Ejercicio 1

```
{ runtime: { $gt: 120 } }
```

› Ejercicio 2

```
{ year: { $lt: 1990 } }
```

› Ejercicio 3

```
{ genres: "Action" } o { genres: { $in: ["Action"] } }
```

› Ejercicio 4

```
{ rated: { $exists: false } }
```

› *Ejercicio 5*

{ cast: { \$exists: true } }

› *Ejercicio 6*

{ "genres.2": { \$exists: true } }

› *Ejercicio 7*

{ "awards.nominations": { \$gt: 10 } }

› *Ejercicio 8*

{ "imdb.rating": { \$lt: 5 } }

› *Ejercicio 9*

{ "imdb.rating": { \$gte: 6, \$lte: 8 } }

› *Ejercicio 10*

{ \$and: [{ rated: "PG" }, { runtime: { \$gt: 90 } }] }

› *Ejercicio 11*

{ \$or: [{ genres: "Horror" }, { genres: "Thriller" }] } o { genres: { \$in: ["Horror", "Thriller"] } }

› *Ejercicio 12*

{ genres: { \$nin: ["Comedy", "Romance"] } }

› *Ejercicio 13*

{ title: { \$regex: "War", \$options: "i" } }

› *Ejercicio 14*

{ title: { \$regex: "^The", \$options: "i" } }

› *Ejercicio 15*

{ plot: { \$regex: "family", \$options: "i" } }

› *Ejercicio 16*

{ released: { \$gte: new Date("2005-01-01"), \$lte: new Date("2010-12-31") } }

› *Ejercicio 17*

{ \$and: [{ "imdb.votes": { \$exists: true } }, { "imdb.votes": { \$gt: 5000 } }] }

› *Ejercicio 18*

{ "directors.1": { \$exists: true } }

› *Ejercicio 19*

{ countries: { \$all: ["USA", "UK"] } }

› *Ejercicio 20*

{ "languages.1": { \$exists: true } }



ACTIVIDAD N° 2: Queries en Neo4J con Cypher

A partir de la base de datos indicada, resuelva los siguientes ejercicios:

1. Listar todas las películas con su año de lanzamiento, ordenadas por año.
2. Encontrar todos los actores que participaron en Top Gun.
3. Obtener todas las películas dirigidas por Nora Ephron.
4. Mostrar las personas nacidas antes de 1960.
5. Listar las películas donde participó Meg Ryan, junto con los roles que interpretó.
6. Encontrar las películas en las que actuaron tanto Tom Hanks como Meg Ryan.
7. Listar los directores que también hayan actuado en alguna película.
8. Contar cuántas películas produjo Joel Silver.
9. Encontrar películas (1990–2000) donde actuó algún actor nacido antes de 1950.
10. Listar los actores que interpretaron más de un personaje en la misma película.
11. Encontrar pares de actores que hayan trabajado juntos en más de una película.
12. Obtener las películas donde Tom Cruise actuó antes del año 2000.
13. Encontrar todas las personas que hayan trabajado (*cualquier rol*) junto a Keanu Reeves.
14. Listar las películas con más de 8 actores.
15. Encontrar actores que solo hayan actuado, y nunca dirigido, escrito ni producido.

ACTIVIDAD 2 | *Clave de corrección* CC1

› Ejercicio 1

```
MATCH (m:Movie)
RETURN m.title AS titulo, m.released AS anio
ORDER BY anio;
```

› Ejercicio 2

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie {title: 'Top Gun'})
RETURN p.name AS actor;
```

› Ejercicio 3

```
MATCH (n:Person {name:'Nora Ephron'})-[:DIRECTED]->(m:Movie)
RETURN m.title AS pelicula;
```

› Ejercicio 4

```
MATCH (p:Person)
WHERE p.born < 1960
RETURN p.name, p.born
ORDER BY p.born;
```

› *Ejercicio 5*

```
MATCH (meg:Person {name:'Meg Ryan'})-[:ACTED_IN]->(m:Movie)
RETURN m.title, a.roles;
```

› *Ejercicio 6*

```
MATCH (t:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)-[:ACTED_IN]-(meg:Person {name:'Meg Ryan'})
RETURN m.title;
```

› *Ejercicio 7*

```
MATCH (p:Person)-[:DIRECTED]->(m:Movie)
WITH p
MATCH (p)-[:ACTED_IN]->(m:Movie)
RETURN DISTINCT p.name;
```

› *Ejercicio 8*

```
MATCH (:Person {name:'Joel Silver'})-[:PRODUCED]->(m:Movie)
RETURN COUNT(m) AS total_peliculas;
```

› *Ejercicio 9*

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE m.released >= 1990 AND m.released <= 2000
AND p.born < 1950
RETURN DISTINCT m.title, m.released;
```

› *Ejercicio 10*

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE size(a.roles) > 1
RETURN p.name, m.title, a.roles;
```

› *Ejercicio 11*

```
MATCH (p1:Person)-[:ACTED_IN]->(m:Movie)-[:ACTED_IN]-(p2:Person)
WHERE p1 <> p2
WITH p1, p2, COUNT(m) AS veces
WHERE veces > 1
RETURN p1.name, p2.name, veces;
```

› *Ejercicio 12*

```
MATCH (tc:Person {name:'Tom Cruise'})-[:ACTED_IN]->(m:Movie)
WHERE m.released < 2000
RETURN m.title, m.released;
```

› *Ejercicio 13*

```
MATCH (keanu:Person {name:'Keanu Reeves'})-[r]->(m:Movie)<-[r2]-(p:Person)
WHERE p <> keanu
RETURN DISTINCT p.name AS colaborador;
```

› *Ejercicio 14*

```
MATCH (m:Movie)<-[:ACTED_IN]-(p:Person)
WITH m, COUNT(p) AS total_actores
WHERE total_actores > 8
RETURN m.title, total_actores
ORDER BY total_actores DESC;
```

› *Ejercicio 15*

```
MATCH (p:Person)-[:ACTED_IN]->(:Movie)
WHERE NOT (p)-[:DIRECTED]->(:Movie)
AND NOT (p)-[:PRODUCED]->(:Movie)
AND NOT (p)-[:WROTE]->(:Movie)
RETURN DISTINCT p.name;
```

MÓDULO 4

Microobjetivos

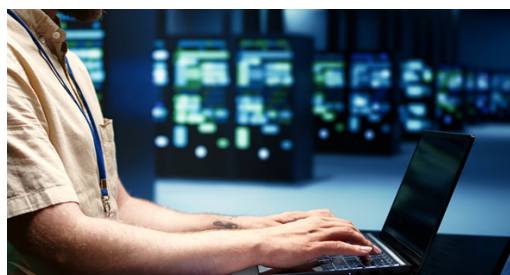
- › Identificar las operaciones fundamentales de análisis de datos con la biblioteca Pandas de Python, manejando las estructuras de Series y DataFrames a través de Jupyter Notebooks en Kaggle, para saber manipular y transformar eficientemente datos provenientes de diversas fuentes.
- › Aplicar el entorno de Jupyter Notebooks en la plataforma Kaggle para la ejecución de código Python, integrando celdas de Markdown para documentación y celdas de código para análisis interactivo, con el objetivo de desarrollar flujos de trabajo reproducibles y bien documentados en ciencia de datos.



VIDEO DOCENTE

Contenidos

- **Pandas Y Notebooks En Python**



Fuente: <https://www.freepik.es/>

Bienvenido/a al último módulo de la materia. En esta etapa final, nos adentraremos en Pandas, la biblioteca de Python fundamental para la ciencia de datos. Pandas nos permite manipular y analizar datos de forma fácil e intuitiva, trabajando con fuentes diversas como archivos CSV, tablas de bases de datos y mucho más.

Para trabajar con Pandas utilizaremos Jupyter Notebooks, un entorno interactivo que nos permitirá ejecutar código, visualizar resultados y documentar nuestro proceso de análisis todo en un mismo lugar, facilitando el aprendizaje y la experimentación. Esta experiencia práctica la llevaremos a cabo en Kaggle, una plataforma líder en la comunidad de ciencia de datos. Kaggle nos ofrece un entorno ideal con todo lo necesario: acceso inmediato a Notebooks, conjuntos de datos diversos y la posibilidad de explorar ejemplos de la comunidad.

Una vez empezemos a utilizar Pandas en este entorno, se darán cuenta de que muchos conceptos como filtrado de tablas, agrupaciones o JOINS (Merge) les resultarán familiares, ya que los hemos visto anteriormente en SQL.

- *Creación de cuenta en Kaggle*

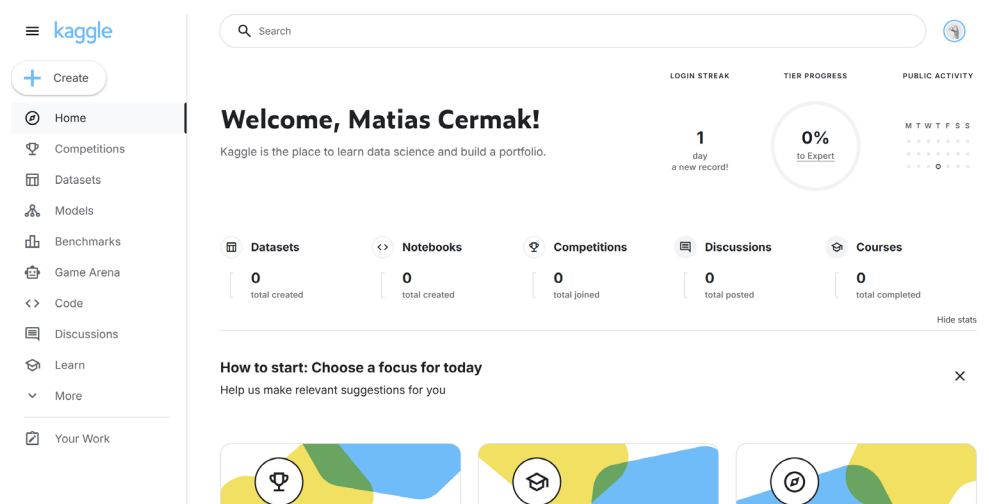
Como mencionamos anteriormente para trabajar con Pandas es necesario que tenga acceso a Kaggle.



Recursos

Para crear su cuenta en esta plataforma, ingrese a la siguiente página web: [Registro Kaggle](#)

Una vez ingresado, verá una página como la siguiente:

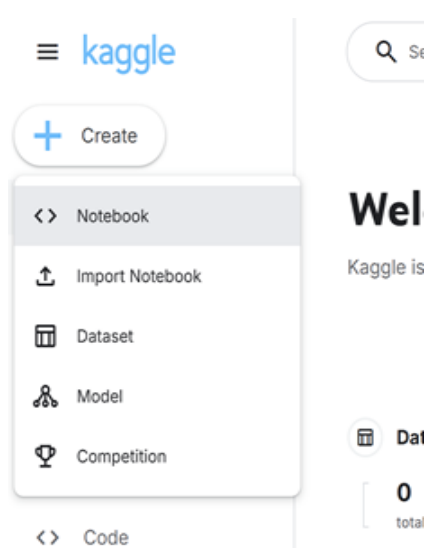


Recursos

En este módulo, utilizaremos Python como base para aprender Pandas, si necesita reforzar conceptos de Python, lo/a invito a realizar este curso gratuito en Kaggle que le va a enseñar todos los conceptos básicos: [Learn Python](#)

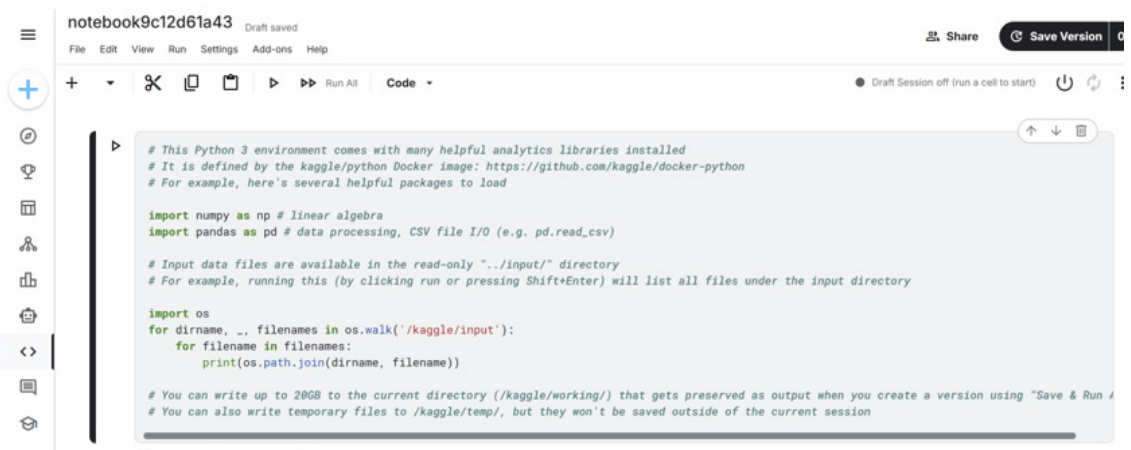
- *Explorando los Notebooks*

Si en la página que vimos arriba le damos al botón de “Create”, veremos las siguientes opciones



Fuente: Elaboración propia.

Aquí, le daremos clic a “Notebook” y se abrirá una ventana como la siguiente:

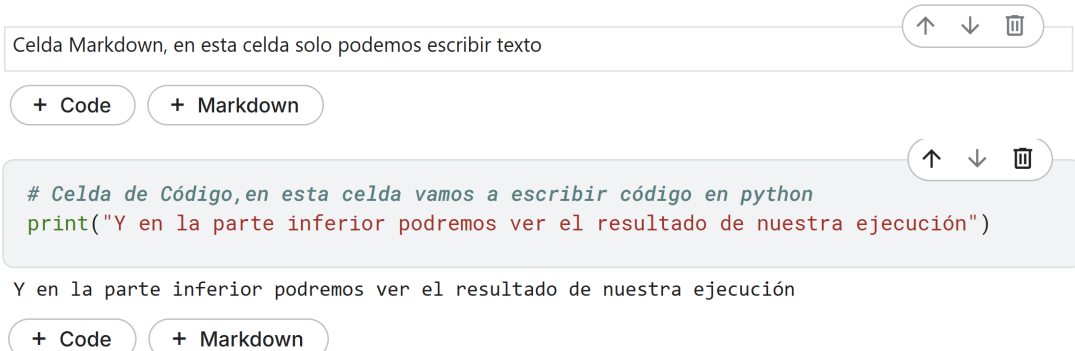


Fuente: Elaboración propia.

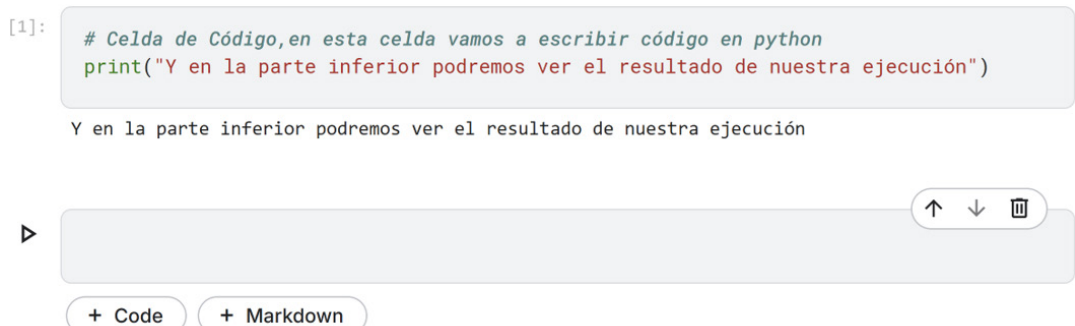
Este es nuestro primer Notebook.



Un Notebook es un entorno interactivo de código. Los notebooks tienen 2 tipos de “Celdas” donde suceden diferentes cosas: la celda Markdown y la celda de código.

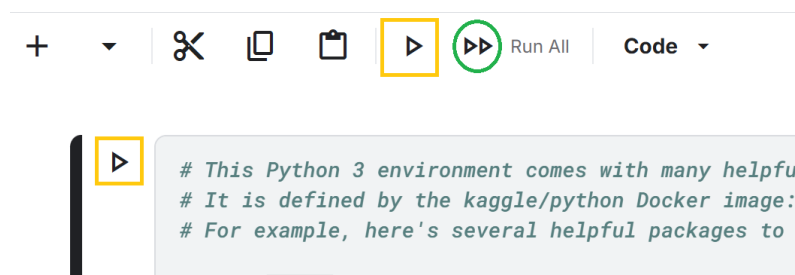


La celda Markdown se utiliza para agregar comentarios o añadir documentación extra relacionada con lo que nuestra notebook está haciendo. Mientras que las celdas de código son las que contienen el código que se ejecuta. Haciendo clic en las píldoras que dicen “+ Code” o “+ Markdown” crearemos una nueva celda de su respectivo tipo abajo. Por ejemplo, si apretamos “+ Code” sucederá lo siguiente:



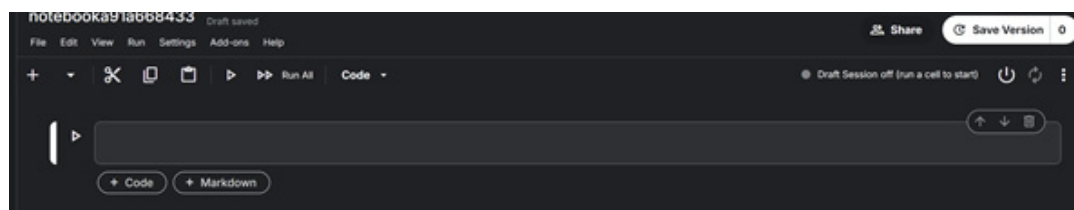
Vemos cómo una nueva celda de código se ha creado en nuestro notebook. Del lado derecho aparecen tres íconos: una flecha hacia arriba, una hacia abajo y el ícono de basura. La primera sirve para mover esta celda arriba de la celda anterior; la segunda para mover la celda más hacia abajo (*en este caso, como la celda actual es la última de nuestro notebook, no nos deja presionarla*). Por último, si apretamos el ícono de basura, eliminaremos la celda.

El modo de ejecución de un notebook es de arriba hacia abajo. Podemos ejecutar celdas individualmente, pero, para mantener un buen orden, se recomienda siempre organizar el código de manera que fluya de arriba hacia abajo.



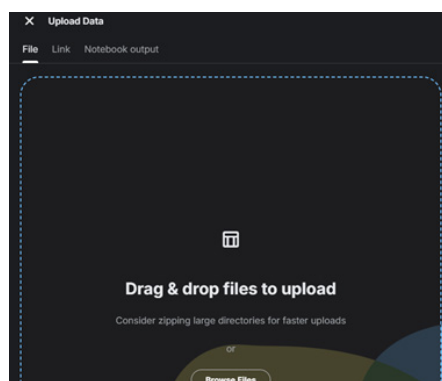
Fuente: Elaboración propia.

Si observamos en la parte superior del notebook, veremos los siguientes botones de “Play”. Los botones en amarillo permiten ejecutar el código que se encuentra en la celda seleccionada. El botón en verde ejecuta todo el código del notebook en orden, de arriba hacia abajo. Por último, si quisiéramos agregar un archivo (ejemplo: un csv para analizar) a nuestro entorno de trabajo, podríamos hacerlo de la siguiente manera:



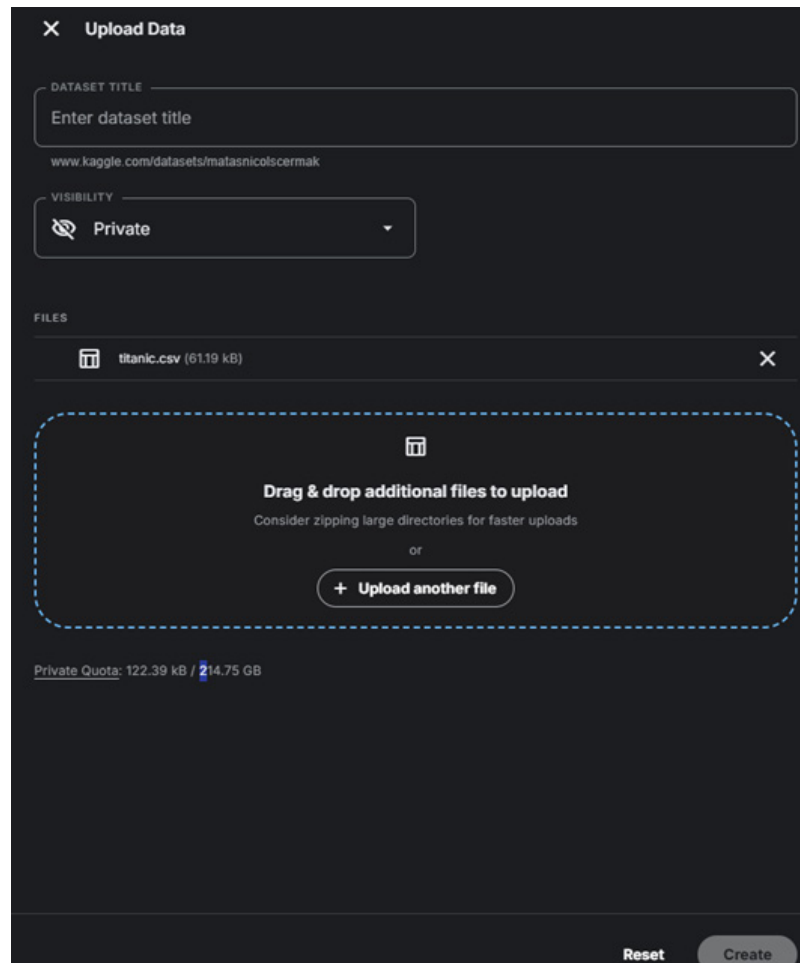
Fuente: Elaboración propia.

Al hacer clic en la solapa marcada en rojo, nos saldrá una barra lateral donde podremos cargar archivos, haremos clic en “Upload” y luego en “New Dataset”, nos aparecerá una pantalla como la siguiente donde podremos cargar nuestros archivos:



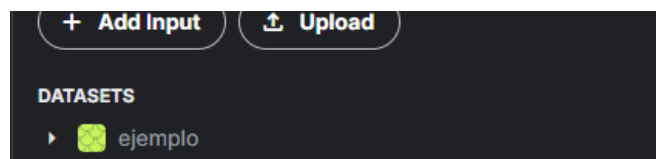
Fuente: Elaboración propia.

Una vez subido, debemos ponerle un nombre al archivo donde nos dice “Dataset Title” y luego darle al botón de “Create”



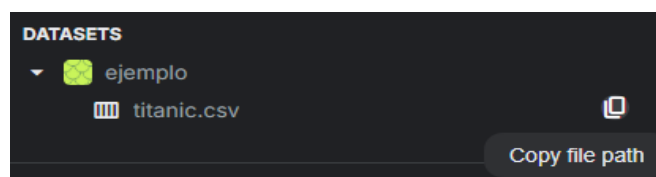
Fuente: Elaboración propia.

Una vez creado, lo veremos en la barra lateral de la siguiente forma:



Fuente: Elaboración propia.

Ahora ya estará disponible para nuestro uso, podremos referenciarlo de la siguiente manera:



Fuente: Elaboración propia.

Hacemos clic en el símbolo de copiar “Copy file path”, con esta referencia al archivo, podremos utilizar Pandas para cargarlo (*no se preocupe por el código ahora, lo aprenderemos en la siguiente sección de este módulo*)

```
import pandas as pd

# Ejemplo mostrando como leer datos de archivos que hayamos cargado nosotros
prueba = pd.read_csv('/kaggle/input/ejemplo/titanic.csv', index_col=0)
```

[9]: `prueba.head()`

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Fuente: Elaboración propia.

Ya con esto podemos iniciar la exploración de Pandas.

• Introducción a Pandas



Recursos

Para trabajar con Pandas en el análisis y manipulación de datos, lo/a invito a visitar la documentación oficial de la biblioteca, donde encontrará guías, tutoriales y ejemplos. <https://pandas.pydata.org/>



Énfasis

Según la web de Pandas, “Pandas es una herramienta de análisis y manipulación de datos de código abierto, rápida, potente, flexible y fácil de usar, construida sobre el lenguaje de programación Python”.

Existen dos estructuras de datos principales:

- › Series: estructura de una dimensión.
- › DataFrame: estructura de dos dimensiones (tablas).

Lo que diferencia a estas estructuras de datos de las estructuras nativas de Python (como listas, diccionarios o tuplas) es que, en el fondo, tanto Series como DataFrame están implementadas sobre NumPy.

NumPy es una librería de computación científica en Python que traduce el código que escribimos a funciones y procesos del lenguaje C, lo cual permite realizar cálculos de manera mucho más eficiente. Gracias a esto, Pandas logra una velocidad de procesamiento muy alta.

Anteriormente existía una tercera estructura llamada Panel, que era una estructura tridimensional, pero actualmente se encuentra discontinuada.

› Pandas en Notebooks

Utilizando el Notebook que creamos anteriormente, eliminaremos todo el contenido creado para empezar desde cero, y crearemos una nueva celda de código que contendrá lo siguiente:

```
import pandas as pd
```

Con este comando lo que hacemos es importar la librería de pandas a nuestro ambiente.



Con lo trabajado hasta ahora, puede resultarles útil profundizar en el uso de los comandos de Pandas. Para ello, lo/a invito a explorar el siguiente enlace: [Tutorial Pandas](#)



Para una experiencia más intuitiva, lo/a invito también a realizar el curso de Pandas que la plataforma Kaggle ofrece. Explore el siguiente enlace: [Curso Pandas](#)

El código que se presenta en los ejemplos del Tutorial puede ejecutarse directamente en el Notebook creado previamente. La intención es que el aprendizaje se desarrolle en simultáneo con la práctica: probar, observar qué sucede y, en ese movimiento, ir comprendiendo cómo opera Pandas sobre los datos.

Por ejemplo, el primer código que vemos es el siguiente:

```
Creación de una serie a partir de una lista
```

- `Series(data=lista, index=indices, dtype=tipo)`: Devuelve un objeto de tipo Series con los datos de la lista `lista`, las filas especificados en la lista `indices` y el tipo de datos indicado en `tipo`. Si no se pasa la lista de índices se utilizan como índices los enteros del 0 al $n - 1$, donde n es el tamaño de la serie. Si no se pasa el tipo de dato se infiere.

```
>>> import pandas as pd
>>> s = pd.Series(['Matemáticas', 'Historia', 'Economía', 'Programación', 'Inglés'],
>>> print(s)
0    Matemáticas
1      Historia
2     Economía
3   Programación
4        Inglés
dtype: string
```

Fuente: Elaboración propia.

Para trasladar este código a un Notebook podemos analizarlo de la siguiente manera.

La primera línea (`import pandas as pd`) ya se encuentra en nuestro archivo, por lo que no es necesario incluirla nuevamente. En cambio, la segunda y la tercera línea pueden agregarse de este modo:

```
▶ s = pd.Series(['Matemáticas', 'Historia', 'Economía', 'Programación', 'Inglés'], dtype='string')
print(s)

0    Matemáticas
1      Historia
2      Economía
3  Programación
4        Inglés
dtype: string
```

+ Code + Markdown

Fuente: Elaboración propia.

De esta manera, es posible llevar a la práctica los conceptos que se van incorporando durante el estudio.



Actividad

Una vez estudiados los contenidos desarrollados, Usted puede realizar las actividades del módulo.

Hemos llegado al final del último módulo de la materia. A lo largo de este recorrido, trabajamos una introducción a las bases de datos relacionales y no relacionales, así como un primer acercamiento al análisis de datos utilizando Pandas y Notebooks. Estos contenidos constituyen un punto de partida: son habilidades que usted seguirá puliendo y entrenando a medida que avance en su trayectoria académica y profesional, y que volverán a aparecer en distintos proyectos, materias y desafíos futuros.

Con esto concluido, lo/a dejo con la segunda parte de la evaluación integradora. Les deseo el mayor de los éxitos en lo que viene y que todo lo aprendido aquí les sirva como base para seguir explorando, creando y resolviendo problemas en su camino formativo. ¡Muchos éxitos!



Evaluación

Una vez finalizado el módulo y las actividades, Usted podrá pasar a realizar la segunda parte de la Evaluación Integradora.



VIDEO DOCENTE

MÓDULO 4 | GLOSARIO

Celda de código: sección del Notebook donde se escribe y ejecuta código Python.

Celda Markdown: sección del Notebook utilizada para documentación, comentarios y texto formateado.

CSV: formato de archivo de texto plano utilizado para almacenar datos tabulares, comúnmente usado con Pandas.

DataFrame: estructura de datos bidimensional en Pandas, equivalente a una tabla.

Import pandas as pd: comando para importar Pandas en un entorno Python y usar la abreviatura pd.

Jupyter Notebook: entorno interactivo que permite escribir código, visualizar resultados y documentar análisis en un mismo archivo.

Kaggle: plataforma de ciencia de datos que ofrece Notebooks, datasets y recursos para practicar análisis y modelado.

NumPy: librería de computación científica en Python que permite cálculos eficientes; base interna sobre la que funciona Pandas.

Pandas: biblioteca de Python para análisis y manipulación de datos, rápida y flexible, utilizada ampliamente en ciencia de datos.

Series: estructura de datos unidimensional en Pandas, similar a una columna.

MÓDULO 4 | ACTIVIDADES



Actividad

ACTIVIDAD N° 1: Ejercicios en Pandas



Recursos

Para esta actividad, realizaremos los ejercicios del 1 al 7 de la siguiente página:

- › Ejercicios Pandas



Recomendaciones

Utilizar los Notebooks en Kaggle para resolverlos. Para ver la solución, puede darle clic al botón de "Solución" luego de cada ejercicio.